

## Structural Testing

### *Module Six*

*Let us change our traditional attitude to the construction of programs. Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer to do..*

Donald Knuth

*When debugging, novices insert corrective code; experts remove defective code.*

Richard Pattis

*Testing by itself does not improve software quality. Test results are an indicator of quality, but in and of themselves, they don't improve it. Trying to improve software quality by increasing the amount of testing is like trying to lose weight by weighing yourself more often. What you eat before you step onto the scale determines how much you will weigh, and the software development techniques you use determine how many errors testing will find. If you want to lose weight, don't buy a new scale; change your diet. If you want to improve your software, don't test more; develop better.*

Steve McConnell



## 6.1 White Box Testing

White box testing, or structural testing, is a methodology which uses a knowledge of the internal workings of a system to develop test cases. Test cases are developed based on an inspection of the actual code that makes up the system with the objective of “exercising” segments of the code by supplying test cases that cause that segment of code to be executed.

White box testing can be done without reference to the specifications of the system since the cases are developed from a structural analysis of the code or logical structure and without reference to any specified function. But white box testing generally requires programming skills.

The two testing strategies – white box and black box – should never be seen as mutually exclusive. Each strategy has its strengths and weaknesses. The effective tester uses the two strategies to complement each other and produce better overall coverage for his or her test suite.

One important point, the internal workings of a system may mean more than just code. It may include knowledge of hardware interfaces, SQL requests and XML schema specifications.

In general, structural testing involves:

1. Analyzing the code or implementation of the SUT
2. Determine paths of logical flows through the SUT.
3. Test cases are chosen by the methods covered in this module to cause the SUT to execute selected paths. This is called path sensitization. When the path is executed, the test case is said to “exercise” the underlying code. Expected results for those inputs are determined, if possible, from an analysis of the code or other means.
4. The tests are run.
5. Actual outputs are compared with the expected outputs.

There are two main branches of white box testing (that was a software testing joke) that we will be looking at in this module; control flow testing and data flow testing.

### ***6.1.1 Applicability of White Box Testing***

While white box testing can be applied at all levels of testing from unit to systems testing, white box testing tends to be utilized more at the smaller component level where structural testing is primarily used to explore the flow of control thorough the code.

But structural testing is actually logical path testing and applying it to code is just one possible use of the techniques. We can test flows on control in a component but we can also test paths between modules within subsystems, between subsystems within systems, and even between entire systems.

In a later module, we will see that we can even apply these techniques to testing logic and various kinds of models and their representations.

### Advantages of White Box Testing

1. White box testing can be done if the specifications are incomplete or are ambiguous.
2. Testers have a greater level of confidence that the whole SUT is being tested than if they relied on black box methods alone.
3. However, one important advantage that white box testing has over black box testing is the ability to test for surprise functionality. This is where a piece of code, when executed, does something that is not documented in the specification.

Where does surprise functionality come from?

1. Obsolete code that was not removed from application when its functionality was no longer required.
2. Third party modules that were incorporated to provide certain functionality but also contain other functionality that is not needed in the system and is not documented for the SUT.
3. Malicious code that is put inside the application with the intent to subvert its functioning.

### Surprise functionality example.

This example demonstrates the surprise functionality from incorporating a third party module. The following text and photos are taken from the website <http://midnightspaghetti.com/news.htm> and are copyright 2004 by Midnight Spaghetti and The Chocolate G-Strings and are used by permission.

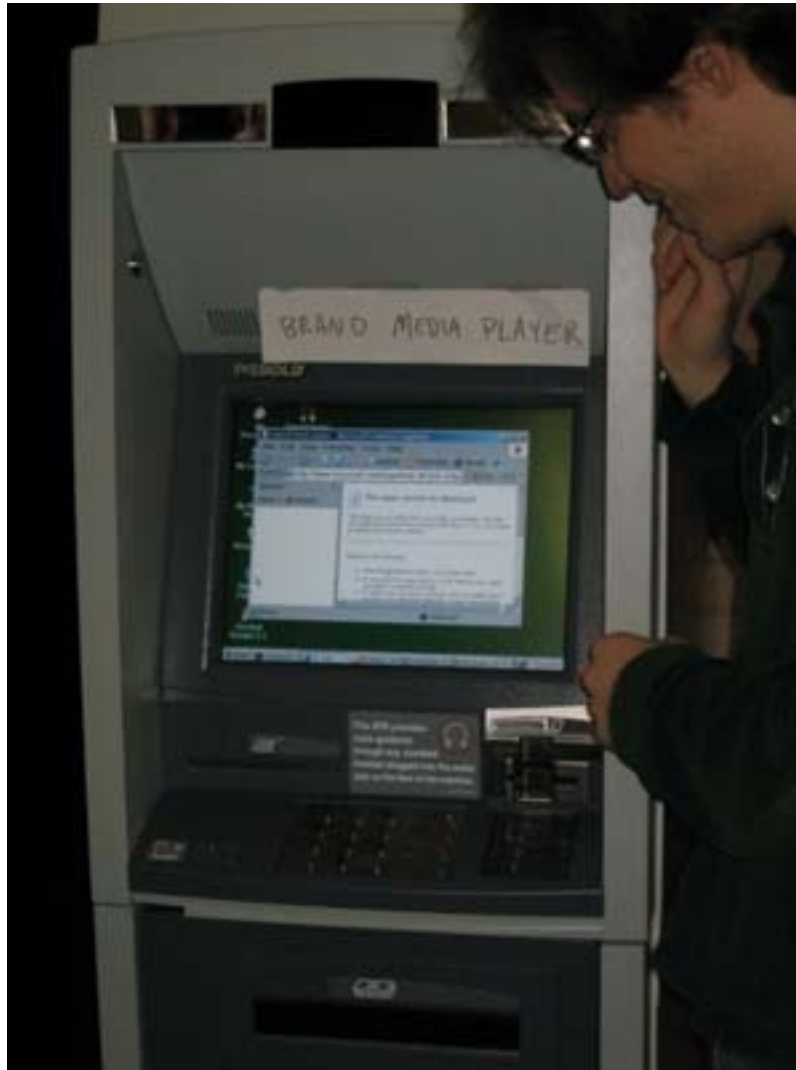
***3-17-2004 Midnight Spaghetti causing a ruckus as always.***

The Scene: Carnegie Mellon University

The Event: A newly installed Diebold Opteva 520 ATM crashes, then reboots. Surprisingly, it's vanilla-style Windows XP operating system initialized without the actual ATM software.

The Result: A desktop computer with only a touch screen interface is left wide open for the amusement of the most wired university in the U.S.





Eschewing more malicious schemes, the first move was to connect to the Internet. This plan proved unsuccessful as there seemed to be no network capability. The situation was complicated in that even typing proved extremely difficult due to the lack of a keyboard. The Character Map program was used to enter text by copy-and-pasting, yet the most that was accomplished by doing so was making the text-to-voice program say, "What, do you think I'm made of money?" Windows Media Player was set up to loop a series of Beethoven, Jazz, and Talking Heads (the sample sound files included with XP) while running a full screen visualization. Finally, an annoyed faculty member in an adjacent office unplugged the machine and dispersed the crowd. The story is humorous until one realizes that Diebold is the leading producer of electronic voting machines. We can all look forward to playing Minesweeper while exercising our citizenship.

## Disadvantages to White Box Testing

Structural testing has three primary disadvantages, aside from needing programming knowledge or skills.

1. The number of execution paths in any code is too large to test. We look at this in more detail in the next topic.
2. Structural testing is based on examination of the existing implementation. Missing paths, components or logic cannot be identified through structural testing.
3. Test cases chosen on the basis of path analysis may not detect data sensitivity errors. For example: consider a piece of code in a banking application that determines if a withdrawal should be allowed on a bank account.

```
bool withdraw(int amount) {  
    if (amount - balance < 0) return false;  
    balance = balance - amount;  
    return true;  
}
```

If the account is overdrawn, then all withdrawals  $> 0$  will be allowed. The error is “sensitive” to the data. However, most of the time, we can pick up these data sensitivity issues in our functional testing

### 6.1.2 Complexity and White Box Testing

An ongoing problem that software testers identified early on was that it is a lot more complicated to test a program than writing it. If we look back at our triangle challenge, we cannot possibly test all possible inputs to the program because the potential input space is infinite. While this was a functional programming issue, a similar one exists in structural testing.

Designing test based on exercising paths of execution faces a similar complexity problem. Consider the following code fragment from Robert Binder

```
for (int i = 0; i < n; ++i) {  
    if ( a.get(i) == b.get(i) )  
        x[i] = x[i]+100;  
    else  
        x[i] = x[i]/2;  
}
```

To fully test all paths through this code, would require testing  $2n + 1$  paths for each value of  $n$  in the loop. Assuming that we started testing 18 billion years ago just after the start of the big bang and were able to test 1,000 cases per second, we would be at  $n=36$  today.



## The logarithmic approach

The combinatorial explosion comes from the fact that paths through software increase geometrically with the addition of new code. For example, we have three segments of code immediately following one another. For each segment there are eight possible paths through the segment. This means that there are  $8 \times 8 \times 8$  or 512 possible paths.

Software testing tries to employ strategies to replace the geometrical progression with a linear progression. In other words, there should only be  $8+8+8$  or 24 cases to test. The whole point of the structural methods we will be looking at is to try and achieve this reduction of cases to a manageable level while maintaining reasonable coverage.

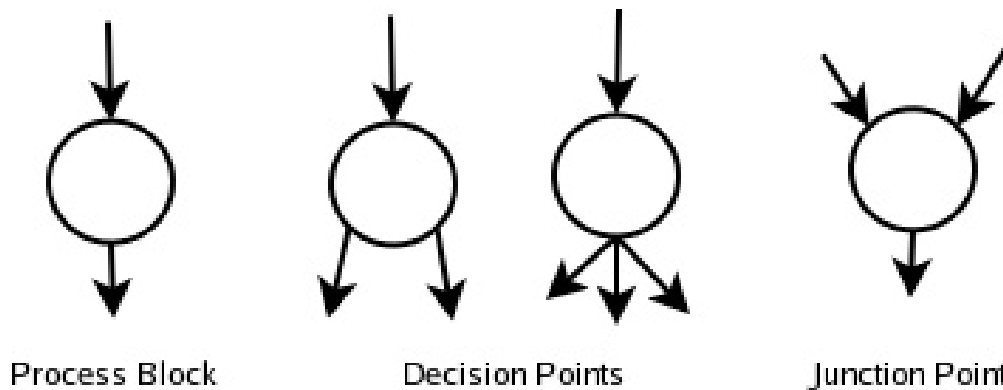
## 6.2 Control Flow Testing

Control flow testing looks at various kinds of paths through a module of program code and uses techniques to create test cases to cover those paths to various degrees of completeness. There are eight levels of “coverage” defined, each level attempting to improve the coverage provided by the test cases while still controlling the growth in the number of test cases.

### Some Notation: Flow Control Graphs.

Control flow graphs are used to represent a module's control flow structure. Lines of code are transformed into graphs (in the mathematical sense). The resulting paths through the graphs are analyzed in order to produce test cases

There are three basic nodes that are used in a control flow graph.



#### *Process Blocks*

A process block is a sequence of program statements that execute sequentially from beginning to end. No entry into the block is permitted except at the beginning. No exit from the block is permitted except at the end. Once the block is initiated, every statement within it will be executed sequentially. Process blocks are represented in control flow graphs by a bubble with one entry and one exit.

#### *Decision Point*

A decision point is where the control flow can change. Decision points can be binary when derived from if-then-else statements. Multi-way decision points are derived from multi switch statements like case statements.

#### *Junction Point*

A junction point is where several control flows join together.

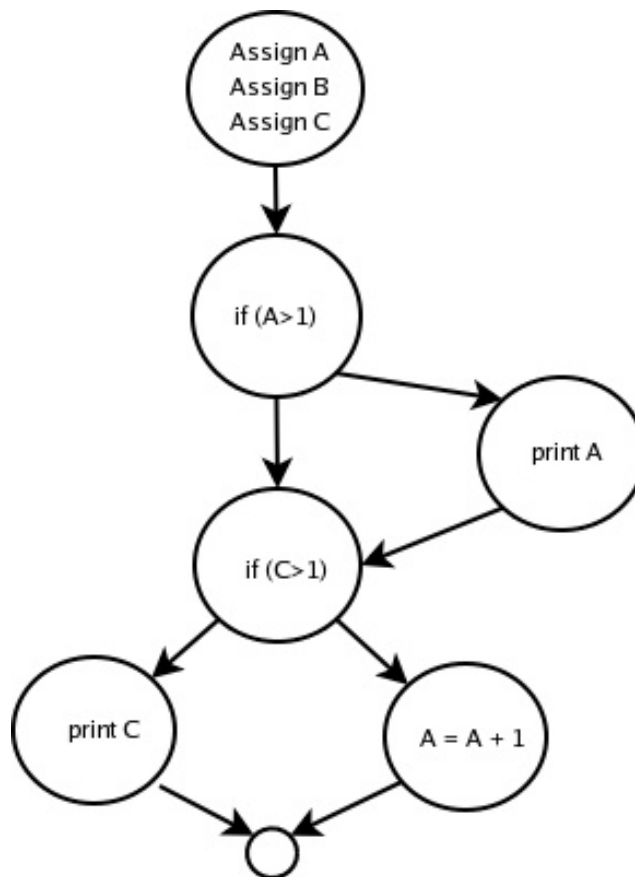


**A control flow example.**

Consider the following code fragment in an imaginary programming language..

```
function testit(integer A, integer B, integer C)
  if (A>1)
    then
      print A;
    endif
  if (C>1)
    then
      print C;
    else
      A = A + 1;
    endif
  return
```

The associated control flow graph is



### 6.2.2 Coverage Level 0: Random Testing

This level of coverage can be described as “testing by poking around.” It tends to be the most common form of unit testing done by programmers in which a few test cases are generated randomly (usually garden path type) and, as long as the program does not crash, it is considered tested. This form of “testing” accomplishes nothing except to instill a false sense of security about the reliability of the code.

### 6.2.3 Coverage Level 1: Statement Coverage

Statement coverage means that every executable statement within the module is executed. Statement coverage is popular with programmers under the belief it is an effective strategy, however while it does exercise the code, it does little to check that the flow of control within the program is correctly written.

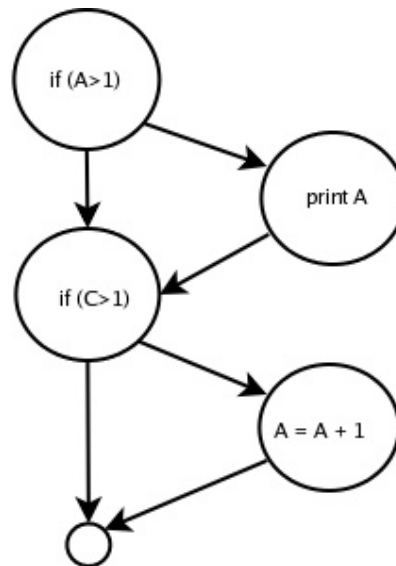
While statement coverage seems like a reasonable goal, defects can be missed when relying only on statement coverage.

Consider the following code:

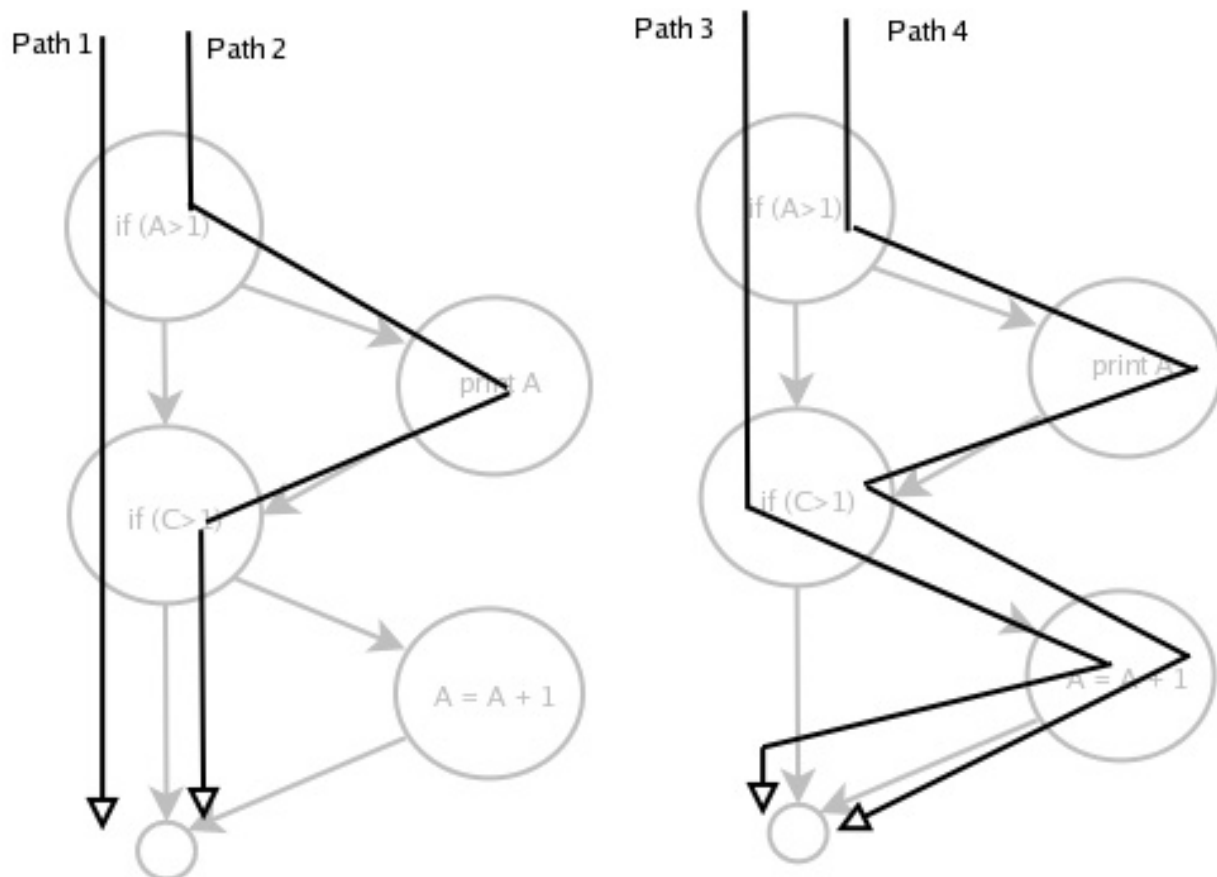
```
if (A>1)
    then
        print A;
endif
if (C>1)
    then A = A + 1;
endif
```

We can get full statement coverage with the single test case (A=2,C=2). This test case will exercise every executable statement in the sample. This code can be represented in graphical form as:





If we think in term of paths through the code, there are four distinct paths.



Our statement coverage only provides coverage of path 4. From this perspective, statement coverage is totally inadequate for producing anything approximating path coverage.

### 6.2.4 Level 2: Decision Coverage

We can get a better approximation to path coverage by using what is called decision coverage or branch coverage. The idea behind decision coverage is to produce test cases so that each decision that has a TRUE and FALSE outcome is evaluated at least once as TRUE and once as FALSE.

For those of you comfortable with graph concepts, statement coverage means that we visit each node while decision coverage means we traverse each edge in the flow control graph.

In the previous example, duplicated below, we can get decision coverage with the two test cases (a=2, c=2 and a=0, c=0).

```

if (A>1)
    then
        print A;
endif
if (C>1)
    then A = A + 1;
endif

```

How do we decide on these test cases? One way to be systematic about it is to write down the decisions and determine what test values we need to get both a TRUE and FALSE decision for that condition.

| Condition | TRUE | FALSE | Test Case    |
|-----------|------|-------|--------------|
| If (A>0)  | A>0  |       | 1. (A=2,C=2) |
|           |      | A<=0  | 2. (A=0,C=0) |
| If (C>0)  | C>0  |       | 1. (A=2,C=2) |
|           |      | C<=0  | 2. (A=0,C=0) |

For true/false decisions, we must provide test cases that trigger both a true and false evaluation of the decision. For multiple branches, like switch or case statements, we need to provide a test case to ensure that every branch of the switch has to be invoked.



### 6.2.5 Level 3: Condition Coverage

Decision coverage works well when decisions are simple or what we call an elementary conditional expression. A decision can be a compound conditional expression made up of elementary conditional expressions joined together with logical operators. For example in

```
if (a==0 && c>1) { printf("ok");}
```

The decision is (a==0 && c>1) while the elementary conditions are a==0 and c>1.

In order for the decision to be TRUE, it requires a greater than 0 and c greater than 1. .

If the value of a were set to 1 for testing purposes then the c>1 part of the condition would not be tested. (In most programming languages the second expression would not even be evaluated.)

We compensate for this by using condition coverage. This is done by providing test cases are written so that each condition makes up a decision is evaluated at least once as TRUE and once as FALSE.

#### Writing condition coverage test cases

1. List every elementary condition in the program under review.
2. From this list create an expanded list which also contains the negation of each condition.
3. Construct a test set that causes each of the expanded conditions to be true at least once.

#### Condition coverage example

Consider the code sample:

```
if (a>1 && b==0) {
    printf("Some message");
}
if (a==2 || c>1) {
    printf("Some other message");
}
```

The following set of test cases provide condition coverage.

| Condition       | Value | Test Case        |
|-----------------|-------|------------------|
| a>1             | a=2   | 1. (a=2,b=0.c=2) |
| b==0            | b=0   | 1. (a=2,b=0.c=2) |
| a==2            | a=2   | 1. (a=2,b=0.c=2) |
| c>1             | c=2   | 1. (a=2,b=0.c=2) |
| <b>Expanded</b> |       |                  |
| a <= 1          | a=0   | 2. (a=0,b=1,c=1) |
| b != 0          | b=1   | 2. (a=0,b=1,c=1) |
| a != 2          | a=0   | 2. (a=0,b=1,c=1) |
| c <= 1          | c=1   | 2. (a=0,b=1,c=1) |

### 6.2.6 Level 4: Condition/Decision Coverage

Condition coverage does not imply decision coverage. It is possible to select a set of test cases that provides condition coverage but not decision coverage. For the fragment of code below the test cases (a=1, c=2) and (a=0,c=0) provide condition coverage but not decision coverage. (You should verify this for yourself and not fall into the Mind Forged Manacle of accepting the instructor's word for it)

```
if (a==0 && c>1) { printf("ok");}
```

Note that (a=1, c=0) and (a=0,c=2) provides both decision and condition coverage. At this level test cases are created for every condition and every decision. The usual process is to create condition coverage test cases and then either tweak them or add more cases to ensure that you have decision coverage.

| Decision    | TRUE    | FALSE   | Test Case        |
|-------------|---------|---------|------------------|
| a>1 && b==0 | a=2,b=0 |         | 1. (a=2,b=0.c=2) |
|             |         | a=0     | 2. (a=0,b=1,c=1) |
| a==2    c>1 | a=2     |         | 1. (a=2,b=0.c=2) |
|             |         | a=0,c=1 | 2. (a=0,b=1,c=1) |



So in our example from the last section, we can verify that we actually got decision coverage as well by seeing how the decisions evaluate with our choice of condition coverage test cases.

### 6.2.7 Level 5: Multiple Condition Coverage.

While condition coverage theoretically tests all of the elementary conditions, in practice it may miss testing some conditions. In certain cases, an outcome in the early part of a compound condition may prevent the rest of the compound condition from being evaluated.

This tends to happen when the outcome is predictable from the evaluation already done. Therefore

1. If two conditions are ANDed together and the first one is false, then the second one will not be evaluated since, no matter what it is, the whole expression will be false.
2. If two conditions are ORed together and the first one is true, the second will not be evaluate since result will be true no matter what value the second condition has.

In order to implement multiple condition coverage, for each compound condition, compile a new list of all possible combinations of its elementary conditions. Construct test cases that cases each of the lists to be true at least once.

#### Multiple condition coverage example

Consider the code fragment below.

```
if (a==0 && b<1) {
    x=x+1;
}
if (c==3 || d)<0) {
    y=0;
}
```

In the first decision, if a is not 0, then the compiler will never evaluate b<1. Similarly if c is 3, then d<0 will never be evaluated. While there appear to be no particular problems with this on first glance consider the following somewhat contrived code.

```
if ( a >= 0 && abs(b/a) < 2) { ... }
```

If we are doing condition/decision coverage with cases:

| Condition                | TRUE       | Case          | Decision |
|--------------------------|------------|---------------|----------|
| $a \geq 0$               | $a=1$      | $(a=1, b=1)$  | TRUE     |
| $\text{abs}(b/a) < 2$    | $a=1, b=1$ | $(a=1, b=1)$  | TRUE     |
| $a < 0$                  | $a=-1$     | $(a=-1, b=4)$ | FALSE    |
| $\text{abs}(b/a) \geq 2$ | $a=1, b=4$ | $(a=-1, b=4)$ | FALSE    |

The test cases provide level 4 coverage but they do not pick up the error that the case  $(a=0, b=0)$  would. The reason is that the cases that cause the decision to be FALSE all cause the decision to be made on the the first condition. The case error is picked up only when we create a FALSE decision so that the FALSE decision is made on the second condition.

### 6.2.8 Level 6: Basic Loop Coverage

Testing loops is a real challenge in white box testing. Loops can produce an infinite number of code paths in a module. However we can exercise a loop using some ideas borrowed from out boundary value analysis in black box testing.

There are several different kinds of loops that we encounter in code:

1. Simple loops,
2. Nested loops,
3. Concatenated loops, and
4. Unstructured loops.

#### Simple Loops

Simple loops are the basic loop constructs we find in code, like while and for loops. The loop is said to be simple since there is only one control mechanism that “runs” the loop.

When we have a simple loop, we should try, if possible, to execute the following tests.

1. One case that causes flow to skip the loop entirely.
2. A case where we pass through the lop only once.
3. A case where me make  $m$  passes through the loop where  $m < n$  and  $n$  is the maximum times the loop should iterate.
4. Cases with  $n - 1$ ,  $n$ ,  $n + 1$  passes through the loop.



Consider the following loop in a C++ module that iterates over a data array of size 10.

```
int data[10];    // the data array.
int reps = 0;    // our iteration number
cin > reps;      // input from console.
for (int i = 0; i < reps; ++i) {
    if (data[i] == 13) cout << "unlucky";
    else cout << "lucky";
}
```

The test cases that we need are:

1. reps=0 This should cause the loop to not execute.
2. reps=1: This should cause the loop to execute once.
3. reps=4: This is our  $m < n$  test.
4. reps=9, reps=10, reps=11: These are the boundary value test at  $n-1$ ,  $n$  and  $n+1$

## Nested loops

Nested loops cannot simply repeat the technique of simple loops at each level since this would result in a combinatorial explosion in the number of test cases. A more practical approach for nested loops is:

1. Start at the innermost loop. Set all other loops to iterate once.
2. Conduct simple loop tests for the innermost loop.
3. Work outward, conducting simple loop tests for the next loop while keeping all other outer loops to iterate once and the tested nested loops iterating at their typical values.
4. Continue until all loops have been tested.

## Concatenated loops

Concatenated loops can be tested as simple loops if each loop is independent of the others. If they are not independent (e.g. the loop counter for one is the loop counter for the other), then the nested approach can be used.

## Unstructured loops

Don't even try.

### ***6.2.9 Level 7: Total Path Coverage***

When loops are in a module, we run into the problem of combinatorial explosion of paths. It has long been known in the mathematical study of programs, that all programs are reducible to a set of if-then and while loop constructs.

For code modules without loops the number of paths is generally small enough that a test case can be constructed for each path using the techniques up to level 5.. For modules with loops, we can achieve full path coverage on the non-loop portions of the module and adequate coverage of the loops using the level 6 techniques of loop testing,



## 6.3 Data Flow Testing

Consider the following two examples of C++ data errors:

```
int cal() {
    int value;
    cout << value; // prints out junk
    return value;
}

void recal() {
    MyOBJ * pointer1 = new MyOBJ;
    MyOBJ *pointer2;
    pointer2->equals(pointer1); // null pointer
    return; // memory leak
}
```

These errors are all due to data mismanagement. In the first case, value is not initialized so that the value printed will be whatever was "left over" in the memory location which value has been allocated. In the second case, we have a method call on a non-existent instance of a class, and we also fail to release memory for pointer1 which produces a "memory leak."

None of these errors can be picked up by control flow testing, so we turn to a form of structural testing called data flow testing.

### 6.3.1 Data Life Cycle

Variables that contain data values have a life cycle: they are created, used, and destroyed. Different programming languages have different ways of handling a variable's life cycle. Older languages, like FORTRAN for example, create a variable the first time it is used, while more modern languages like C++ tend to take the approach that each variable must be created by the programmer explicitly (also known as defining the variable) before they can be used.

Variables generally have a scope which is the extent of code over which they exist and can be referred to in the code. Normally there is a definition of the variable either at the beginning of its scope (as required in a language like C) or close to its point of first usage (like in Java and C++). Variables defined within a scope are usually destroyed automatically at the end of their scope.

```

{
int x;      // x is defined here
x = 0;      // x can be used here, y cannot
    {
        int y=1;;    // y is defined here
        x = x + y;    // both x and y can be used
    }        // y is destroyed
cout << x;  // x is still defined
}           // x is destroyed

```

Three possibilities exist for the first occurrence of a variable through a program path:

1. **~d** - the variable does not exist (that is what the ~ means), and then it is defined (d). This is correct program flow. The d stands for “defined.”
2. **~u** - the variable does not exist, and then it is used (u). This is incorrect since a variable may not be used before it is defined. Notice that just because we can do this in some programming languages does not mean we should do this. The u stands for “used.”
3. **~k** - the variable does not exist, and then it is killed or destroyed (k). This is incorrect. Destroying a variable before it is created is the result of a programming error. The k stand for “killed.”

Once a variable exists, consider the following sequences. Now consider the following time-sequenced pairs of defined (d), used (u), and killed (k):

1. **dd** Defined and defined again. This is probably a programming error. Most compilers should catch this unless we have a scoping problem like variable shadowing.
2. **du** Defined and used. The normal case.
3. **dk** Defined and then killed. Probably a programming error.
4. **ud** Used and defined. This may be an error but may not be.
5. **uu** Used and used again. Normal usage.
6. **uk** Used and killed. Normal usage.
7. **kd** Killed and defined. Normal usage. A variable is killed and then redefined.
8. **ku** Killed and used. Error, using a variable that does not exist.
9. **kk** Killed twice. Programming error.

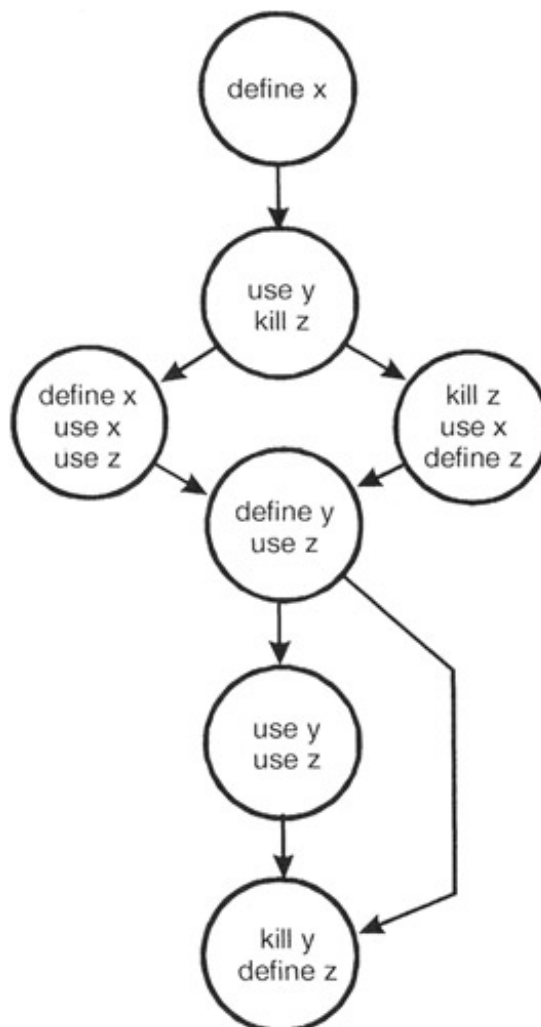


## Data flow graphs

A data flow graph, like a control flow graph, shows the control flow through a module and it documents the definition, use, and destruction of each of the module's variables.

These diagrams allow us to examine define-use-kill patterns for a variable fare valid or indicate the presence of an error.

When the diagram is first prepared, we do static tests of the diagram, which is just a fancy way of saying we look at the graph. Next we perform dynamic tests on the module by constructing and executing test cases.





## Test Management

### *Module Seven*

*The real complexity in our jobs is that all planning is done under conditions of uncertainty and ignorance. The code isn't the only thing that changes. Schedules slip. New milestones are added for new features. Features are cut from the release. During development, everyone - marketers, developers and testers - comes to understand better what the product is really for."*

Brian Marick

*But there's more to defining processes and coordinating people than assigning someone to dream up a checklist and get it blessed in a staff meeting.*

James Bach

*If we deliver on time, but the product has defects, we have not delivered on time.*

Phil Crosby



## 7.1 Defining Test Management

Test management is project management done in accordance with professional standards and practices. The ISTQB defines test management to be:

*The planning, estimating, monitoring and control of test activities, typically carried out by a test manager.*

If you replace the word “test” with “project,” then the definition is essentially the same as the definition for project management – in this case the “project” being managed is to test a system or application. The basic project management concepts and practices – project planning, resource allocation, measurement and metrics – all appear within the test management vocabulary as well.

The primary difference between test management and project management in general is the more rigorous framework of standards and practices that test management must follow – the rationale for which will be explained in this module and subsequent modules.

### 7.1.1 Test Management Responsibilities

There is a lot more to being a test manager than just being able to manage a test process. According to the ISTQB, the responsibilities of a test manager include:

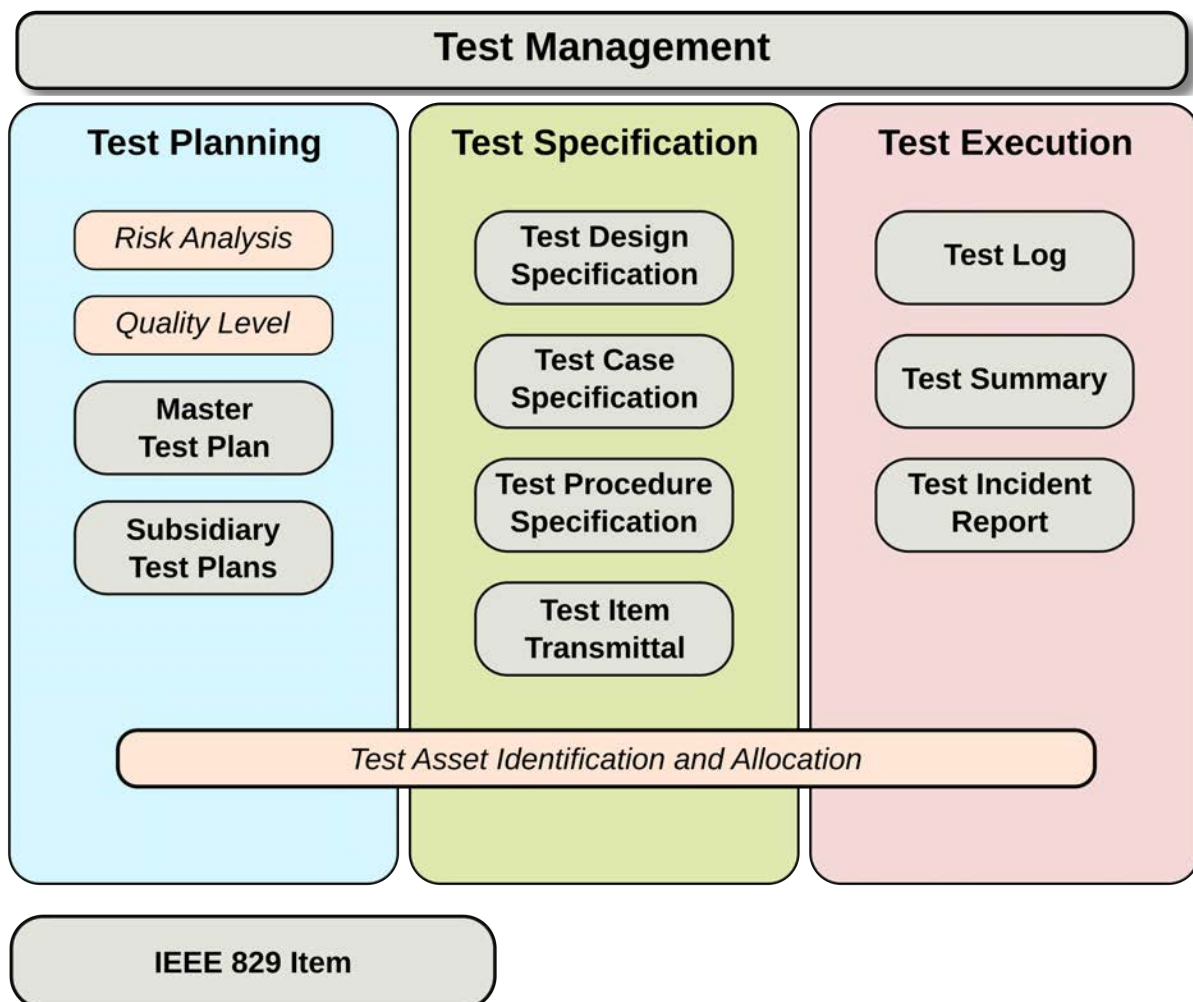
1. Lead the test management within an organization, project or program to identify and manage critical success factors with management commitment at CEO/Board level
2. Take appropriate business-driven decisions on a test management strategy and implement organization wide commitment and compliance based on quality KPIs
3. Assess the current status of the test management, propose step-wise improvements and show how these are linked to achieving business goals within the organizational context of test management organization or project/program
4. Set up a strategic policy for improving the test management and the testing, and implement that policy in an organization
5. Analyze specific problems with the test management and its alignment with other roles or management areas in the project/organization, and propose effective solutions
6. Create a master test plan with matching governance dashboard to meet or exceed the business objectives of the organization or a project/program
7. Develop innovative concepts for test management (project) organizations which include required roles, skills, methodologies (tools) and organizational structure
8. Establish a standard process for implementing test management in an organization (project/program) with standardized delivery based on quality KPIs
9. Lead an organization to improve the test management process and manage the introduction of changes

10. Understand and effectively manage the human issues associated with test-project management and implement necessary changes

Obviously these topics way to deep for what we will be discussing in this overview but it is important to keep in mind that being a good test manager does involved more than being able to manage the processes described in this module – in the same way that a good project manager also has to deal with people issues, organizational politics and those other “intangibles.”

### 7.1.2 The IEEE Test Process

The diagram below illustrates the basic processes that are standardized for test management. Most of the process definitions used in test management are taken from the IEEE 829 standard discussed in a bit later in this section. The processes in the grey boxes are specifically defined in the IEEE standard and are incorporated more or less directly into most test management proceeses. The other processes are outside the IEEE standard but are also needed for effective test management and are widely used.



## IEEE 829 Standard for Software and System Test Documentation

This standard, which is available from the IEEE, is the industry standard definition for test management core processes. This 2008 version of the standard is the third with previous versions being released in 1996 and 1983 (referred to as 820-2008, 829-1996 and 829-1983 respectively).

This standard is to be superseded by the ISO/IEC/IEEE 29119 five part standard which was initiated in 2007 and is made up of:

1. ISO/IEC/IEEE 29119-1: Concepts & Definitions (September 2013)
2. ISO/IEC/IEEE 29119-2: Test Processes (September 2013)
3. ISO/IEC/IEEE 29119-3: Test Documentation (September 2013)
4. ISO/IEC/IEEE 29119-4: Test Techniques (December 2015)
5. ISO/IEC/IEEE 29119-5: Keyword Driven Testing (November 2016)

## The Documentation Standard

For many years, test management was defined by the IEEE in terms of maintaining a set of standard test documentation. Since software testing is a field that relied heavily on documentation, the IEEE as developed the IEEE 829-1998 Standard for Software Test Documentation which was felt to be adequate because in order to use the documents correctly, testers had to implement a test management process. Or at least that was the theory.

The documentation standard is self described as:

The standard describes a set of basic software test documents and was developed for the following reasons:

1. A standardized test document can facilitate communication across groups, for example a customer contracting out software development and a supplier are both using the same definition for a test plan.
2. A standardized test document can function as a completeness checklist for the testing process.
3. A standardized set of documentation can also provide a baseline for the evaluation of an organization's existing test documentation practices.
4. In organizations without their own set of testing documents, the existence of these standardized documents provides a template from which an organization can develop its own documentation.

According to the specification:

The documents described in the standard cover test planning, test specification, and test reporting.

### ***Test Planning***

The test plan prescribes the scope, approach, resources, and schedule of the testing activities. It identifies the items to be tested, the features to be tested, the testing tasks to be performed, the personnel responsible for each task, and the risks associated with the plan.

### ***Test Specification***

Test specification is covered by three document types:

1. A test design specification refines the test approach and identifies the features to be covered by the design and its associated tests. It also identifies the test cases and test procedures, if any, required to accomplish the testing and specifies the feature pass/fail criteria.
2. A test case specification documents the actual values used for input along with the anticipated outputs. A test case also identifies constraints on the test procedures resulting from use of that specific test case. Test cases are separated from test designs to allow for use in more than one design and to allow for reuse in other situations.
3. A test procedure specification identifies all steps required to operate the system and exercise the specified test cases in order to implement the associated test design. Test procedures are separated from test design specifications as they are intended to be followed step by step and should not have extraneous detail.

### ***Test Reporting***

Test reporting is covered by four document types:

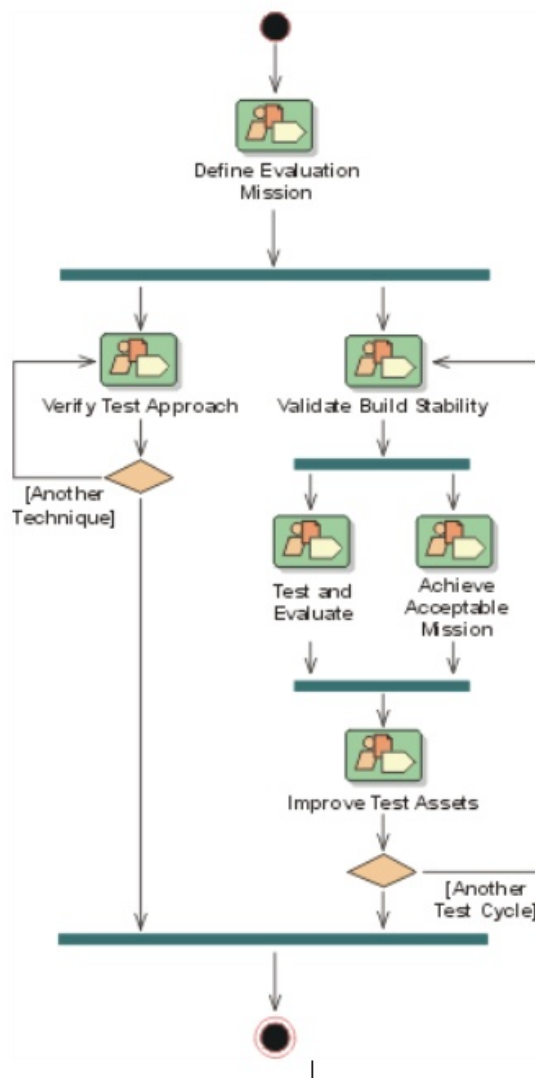
1. A test item transmittal report identifies the test items being transmitted for testing in the event that separate development and test groups are involved or in the event that a formal beginning of test execution is desired.
2. A test log is used by the test team to record what occurred during test execution.
3. A test incident report describes any event that occurs during the test execution which requires further investigation.
4. A test summary report summarizes the testing activities associated with one or more test design specifications.



### 7.1.2 The RUP Model

The IEEE documentation based approach was not the only one being taken by software test managers. Out of the Rational Unified Process and other similar approaches, a more activity oriented view of test management was in competition with the IEEE approach. Just as a point of terminology, processes in RUP are called "disciplines."

Most of the content in this section has been adapted from Rational Edge article Testing: The RUP Philosophy by Paul Szymkowiak and Philippe Kruchten.



### The Test Discipline

The Test discipline acts in many respects as a service provider to the other disciplines. Testing focuses primarily on the evaluation or assessment of product quality realized through a number of core practices:

1. Finding and documenting defects in software quality.
2. Generally advising about perceived software quality.
3. Proving the validity of the assumptions made in design and requirement or specifications through concrete demonstration.
4. Validating the software product functions as designed.
5. Validating that the requirements have been implemented appropriately.

An interesting difference between Test and the other disciplines in RUP is that Test is essentially tasked with finding and exposing weaknesses in the software product. That is interesting in that, to yield the most benefit, it necessitates a different general philosophy to that used in the Requirements, Analysis and Design and Implementation disciplines. The somewhat subtle difference is that while those other disciplines focus on completeness, Test focuses on incompleteness. A good test effort is driven by questions such as "How could this software break?" and "In what possible situations could this software fail to work predictably?".

### RUP Test Philosophy

The test philosophy in the RUP takes a different approach and can be summarized as a small set of principles:

1. **Iterative development.** Testing does not start with just test plans. The tests themselves are developed early and conducted early, and a useful subset of them is accumulated in regression suites, iteration after iteration. This enables early feedback of important information to the rest of the development team, permits the tests themselves to mature as the problem and solution spaces are better understood, and enables the inclusion in the evolving software design of required testability mechanisms. To account for the changing tactical objectives of the tester throughout the iterative lifecycle, we will introduce the concept of mission.
2. **Low up-front documentation.** Detailed Test planning is defined iteration by iteration, based on a governing master Test plan, to meet the needs of the team and match the objectives of the iteration. For example, during the Elaboration phase, we focus on architecture, and so the test effort should focus on testing the key architectural elements as they evolve in each iteration. The performance of key end-to-end scenarios should be assessed -- typically under load -- even though the user interface may be rudimentary. But beyond this semiformal artifact that defines the test plan for each iteration, there is not a lot of upfront specification paperwork developed.
3. **Holistic approach.** The approach to identifying appropriate tests is not strictly and solely based on deriving tests from requirements. After all, the requirements rarely specify what the system should not do; they do not enumerate all the possible crashes and sources of errors. The tests for these issues have to come from elsewhere. Tests in the RUP are derived from the requirements and from other sources.



4. **Automation.** Testing starts early in, and continues throughout, the RUP lifecycle; it is impractical to work on many aspects of testing without appropriate tool support. In particular, tools can help generate test data conditions, run tests, and analyze results.

A test cycle is a period of independent test activity that includes, among other things, the execution and evaluation of tests. Each iteration can contain multiple test cycles -- the majority of iterations contain at least one. Each test cycle starts with the assessment of a software build's stability before it's accepted by the test team for more thorough and detailed testing.

The RUP recommends that each build be regarded as potentially requiring a new cycle of testing (that is, a test cycle), but there is no strong coupling between build and test cycle. Typically, Inception iterations of new projects don't produce builds, but iterations in all other phases do. Although each build is a potential candidate for a cycle of testing, there are various reasons why you might not decide to test every software build.

For example, in situations where a build is created daily, a useful test cycle may take longer than the available time between builds. In this case it may be appropriate to test every nth software build.

The intent of the Test Discipline is RUP Test Discipline to define a systematic approach to ensure testing effectiveness and efficiency. The discipline does this by

1. Advocating iterative testing process.
2. Producing early concrete feedback.
3. Being scalable and customizable.
4. Being designed for flexibility.
5. Being focused on efficiency.

### ***Define Evaluation Mission***

For each iteration:

1. Identify the objectives for and deliverables of the testing effort.
2. Identify a good utilization strategy for test resources.
3. Define the scope and boundaries for the test effort.
4. Outline the approach that will be used.
5. Define how progress will be monitored and assessed.

### ***Verify Test Approach***

For each technique:

1. Verify early that the intended test approach will work and confirm its value.
2. Establish the basic infrastructure to enable and support the approach.
3. Obtain commitment from the development team to provide and support the required testability to support the approach.
4. Identify the scope, boundaries, limitations and constraints of each technique.

### ***Validate Build Stability***

For each test cycle:

1. Assess the stability and testability of the Build.
2. Gain an initial understanding, or confirm the expectation, of the development work delivered in the Build.
3. Make a decision to accept the Build and to use in further testing, or to conduct testing against a previous Build.

### ***Test and Evaluate***

For each test cycle:

1. Provide ongoing evaluation and assessment of the Target Test Items.
2. Record the appropriate information necessary to diagnose and resolve any issues identified.
3. Achieve suitable breadth and depth in the test and evaluation work.
4. Provide feedback on the most likely areas of potential quality risk.

### ***Achieve Acceptable Mission***

For each test cycle:

1. Add the minimal set of appropriate additional tests to the smoke test.
2. Assemble new test scripts into appropriate test suites.
3. Remove unproductive or uneconomic test assets.
4. Maintain test environment configurations and test data sets.
5. Conduct general maintenance and maintainability improvements.
6. Document lessons learned.



## Mission

An evaluation mission, as used in the RUP approach to is a simple heuristic model for test planning. This model recognizes that different missions govern the activities and deliverables of the testing effort, and that selecting an appropriate mission is a key aspect of test planning.

The evaluation mission identifies a simple statement the test team can remember in order to stay focused on their overall goal and appropriate deliverables for a given iteration. This is especially important in situations where the team is faced with a number of possibly conflicting goals. A test team without an evaluation mission often describes their goal with statements such as "We test everything" or "We just do testing." Such teams are concerned with simply performing the test activities and overlook how those activities should be adjusted to suit the current project context or iteration context to achieve an appropriate goal.

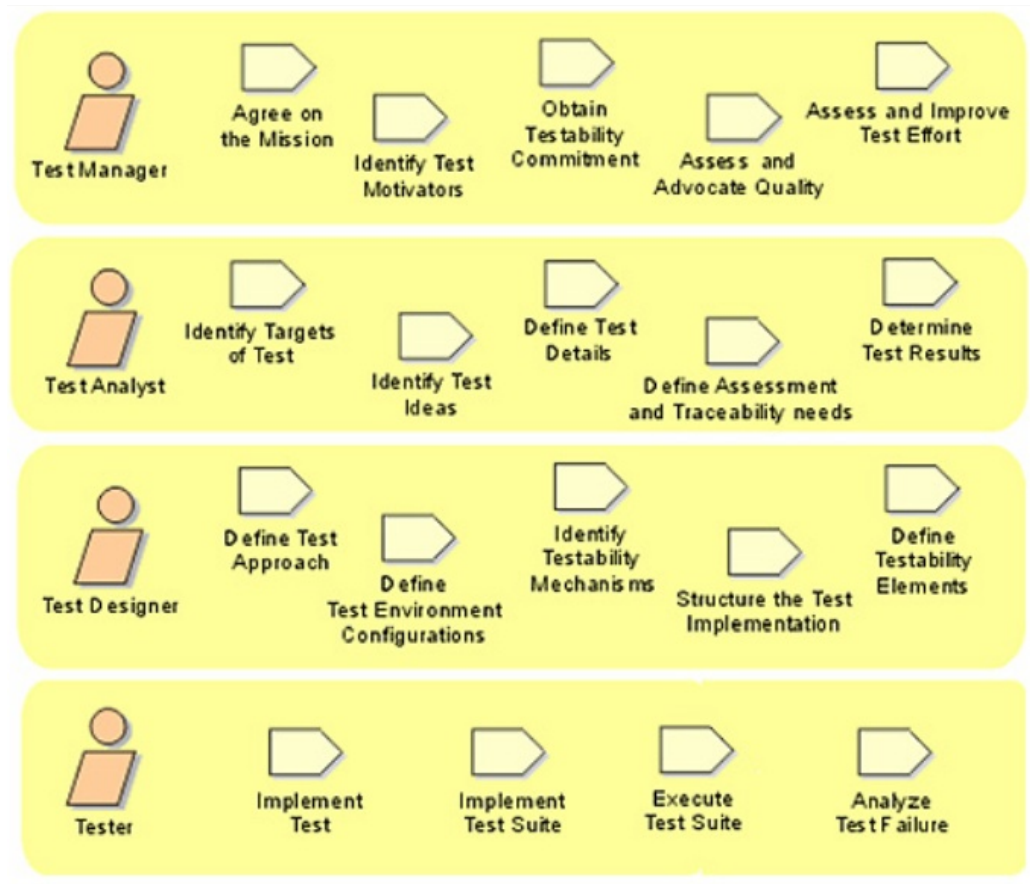
Mission statements shouldn't be too complex or incorporate too many conflicting goals. The best mission statements are simple, short, succinct and achievable. Here are some sample mission statements appropriate for a given iteration:

1. Find as many defects as possible.
2. Find important problems fast.
3. Assess perceived quality risks.
4. Advise about perceived project risks.
5. Advise about perceived quality.
- 6 . Certify to a given standard.
7. Assess conformance to a specification (requirements, design, or product claims).

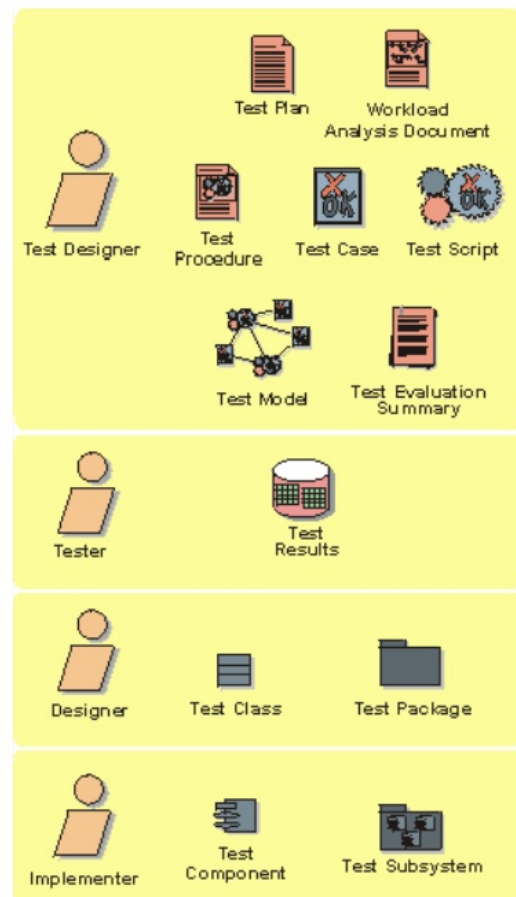
The RUP approach to testing incorporates the concept of Good Enough Quality (GEQ) which is the idea that we should be aiming for GEQ rather than perfection. We have encountered GEQ in a previous module.

## RUP Test Artifacts and Roles

One of the reasons for the Agile backlash against RUP was the formalized roles and responsibilities. The following diagram shows some of the testing roles and responsibilities. While I do not advocate following the RUP model, looking at both the roles and responsibilities can function as a completeness checklist for anyone doing test management.



Similarly, RUP also defined a number of test artifacts and documents just like the IEEE has. There is overlap between the two document sets, but being able to look at how different groups have approached the challenge of test documentation is illuminating.



## ***IEEE Change of Focus***

There were some significant changes that took place in the 2008 version of the IEEE 829 standard. According to the standard the ones that are important to us are:

1. Changed focus from being document-focused to being process-focused in keeping with IEEE/EIA Std 12207.0 TM-1996 a while retaining information on test documentation.
2. Added the concept of an integrity level to assist organizations in determining a recommended minimum set of testing tasks and concurrent selection of test documentation needed to support the tasks.
3. Added a Master Test Plan (MTP) for documenting the actual management of the total test effort.
4. Added a Level Interim Test Status Report to be issued during the test execution activity.
5. Added a Master Test Report for when there are multiple Level Test Reports that need consolidation. The Master Test Report may also summarize the results of the tasks identified in the Master Test Plan.

The two important implications of this are the codification of a risk or integrity level as part of the testing process and the development of an overall master test plan and a corresponding master test report.

The interrelationship between the different artifacts of the standard is depicted below.

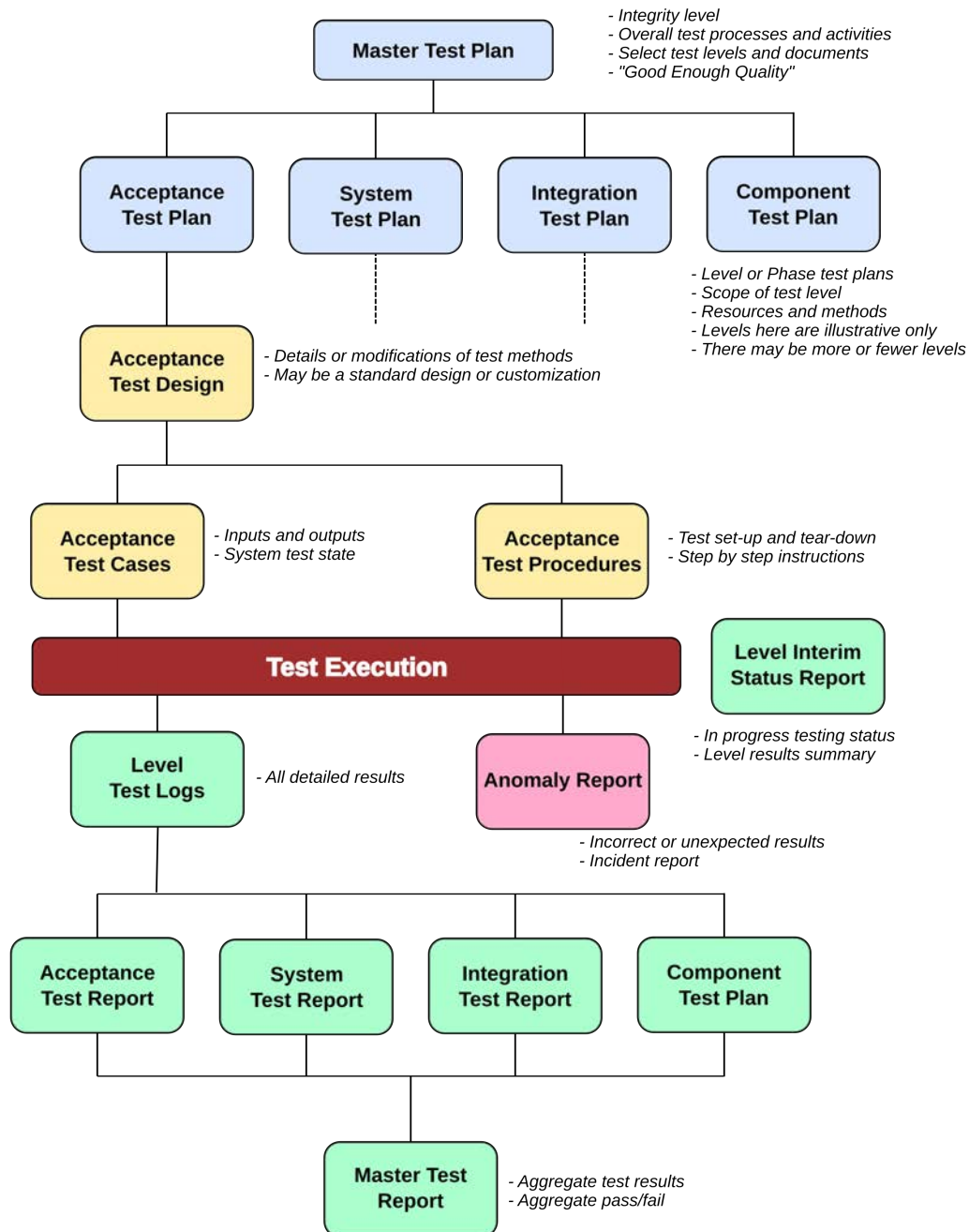
## **Standard as Best Practice**

The IEEE and other standards, like RUP, are derived by describing in a generic sense the practices and methods of expert testers and test managers. When we look at the concept of test maturity or professional test excellence we see that the documentation or processes described represent and describe the way these experts plan, design and execute their testing.

The standards were originally developed to create a common vocabulary and set of artifacts based on what most professional testers were using in some form. The goal was to create a paradigm for the testing profession to share expertise, insights and experiences – much in the same way the accounting profession developed the generally accepted accounting standards.

The next few sections will take a look at the underlying professional attitudes, mindset and approaches to testing and test management that the IEEE and similar standards are based on. Remember that the standards are not intended as a set of rules to be obeyed but rather a description of what the experts in the field do.





## 7.2 Application Lifecycle Management

Application Lifecycle Management (ALM) is a set of concepts that define a set of best practices in software development. ALM is not a methodology or a process – like Agile or waterfall – but focuses instead on broader management concerns that are common to all development process methodologies, whether Agile, waterfall or iterative.

When planning testing activities, as we shall see in various other modules, we have to work on a larger time frame and scope than just the development process – we have to manage our testing at the application life cycle level.

The primary difference between a software development process and ALM is that most development processes only deal with activities up to the completion of the development. ALM takes the broader perspective of “software as a product” which means managing not just the development phase but also the time the software is in production, being maintained and eventually removed from production.

The problem with most standard software development processes is that they often terminate as soon as the product is deployed and the product is handed over to a support team, or in the the worst case, the users are just left to fend for themselves. While there are a number of variations on how it is implemented, all flavors of software development processes have the same underlying flow of work, whether it is done over iteratively as in Agile methodologies where this cycle is repeated a number of times, or finishing each phase completely before moving on to the next as in the waterfall processes.

One of the more interesting aspect of Agile testing is that we not only work within the process but we also expand the scope of our testing into lifecycle activities.

### 7.2.1 *Software as Product*

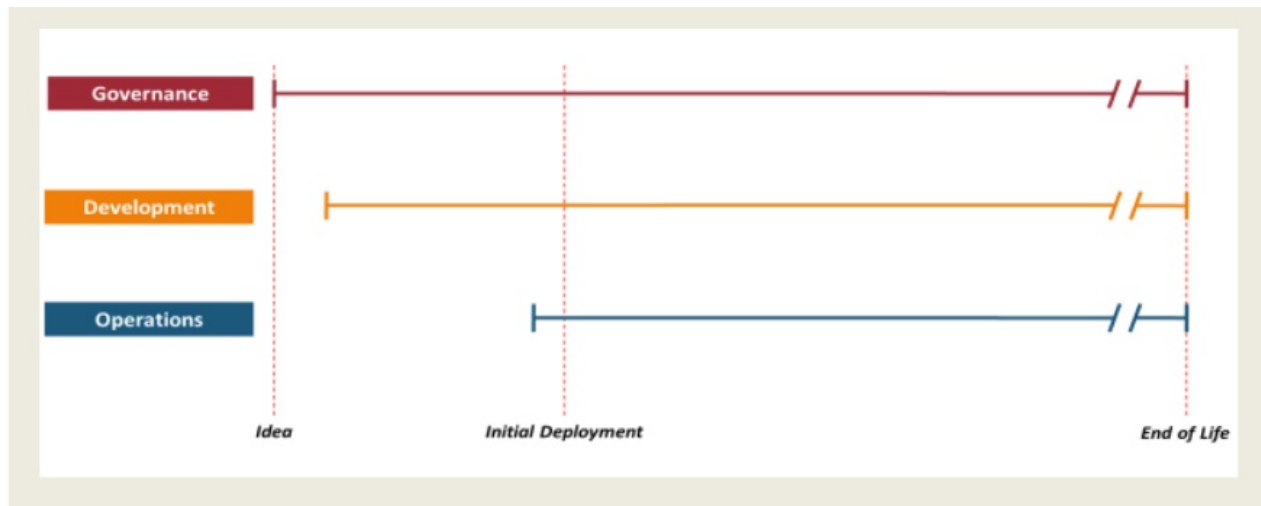
This material is derived from James Chappell's excellent summary of ALM from his 2014 paper sponsored by Microsoft.

A common description of what an application's lifecycle is “the entire time an organization spends money of the product from the initial idea to the end of the application's life when it is no longer in use.” The diagram following shows that there are three events that are considered milestones in an application lifecycle:

1. **Idea or Inception:** This occurs in advance of the start of the development process and usually involves looking at whether to develop an application, examination of the business case, deciding whether to build or buy and other sorts of portfolio management issues and feasibility considerations.
2. **Deployment:** This is where the actual product is moved into production and is maintained. This event marks usually marks some sort of hand-off between the development team and a support team.



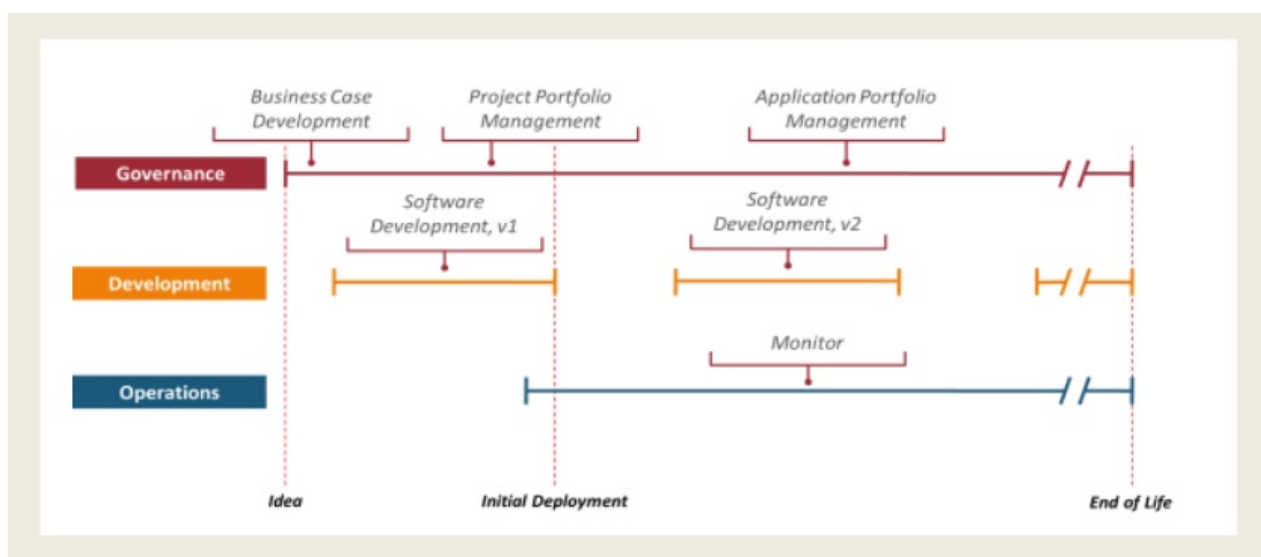
3. Retirement: At some point the application has to be retired, which may actually be a simple process – for example, it may involve transforming massive amounts of data or ensuring that automation support for some critical process is not interrupted.



Spanning these milestone events are three main areas of concern. Generally, these focus on aspects of a product lifecycle that do not usurp the issues and concerns of whatever software development process is being used but rather supplement and complement the process. As can be seen from the diagram each of ALM's three aspects has its own components.

## Governance

Governance starts with business case development and feasibility studies. Once the development begins, governance is now implemented through project portfolio management where the role of the application in the overall business context and IT architecture is continuously evaluated, often with respect to some organizational wide metrics, standards and valuations.



Once an application is deployed, it becomes part of the organization's portfolio of applications, and as such is an organizational asset like any other and is subjected to an ongoing assessment of its associated benefits and costs.

Some of the decisions that are the responsibility of portfolio management are deciding when updates and revisions, or even retirement, make business sense, and ensuring that the application does not either duplicate the function of other applications, or leaves gaps in functionality that could create business liabilities.

An aspect of governance that is recently becoming critical for many organizations is an assessment of security risks and liability arising from issues like hacked data, compliance with new regulatory frameworks and other similar issues. These issues may result in a decision to mitigate risk by retiring an application and migrating functionality to other applications.

### Development

ALM does not try and dictate the actual tasks done in a software development process, but takes a somewhat higher view of the development period. A very typical issue that ALM deals with is the choice of software development process that should be used both in development and during deployment for support and enhancement programming, which may be quite different.

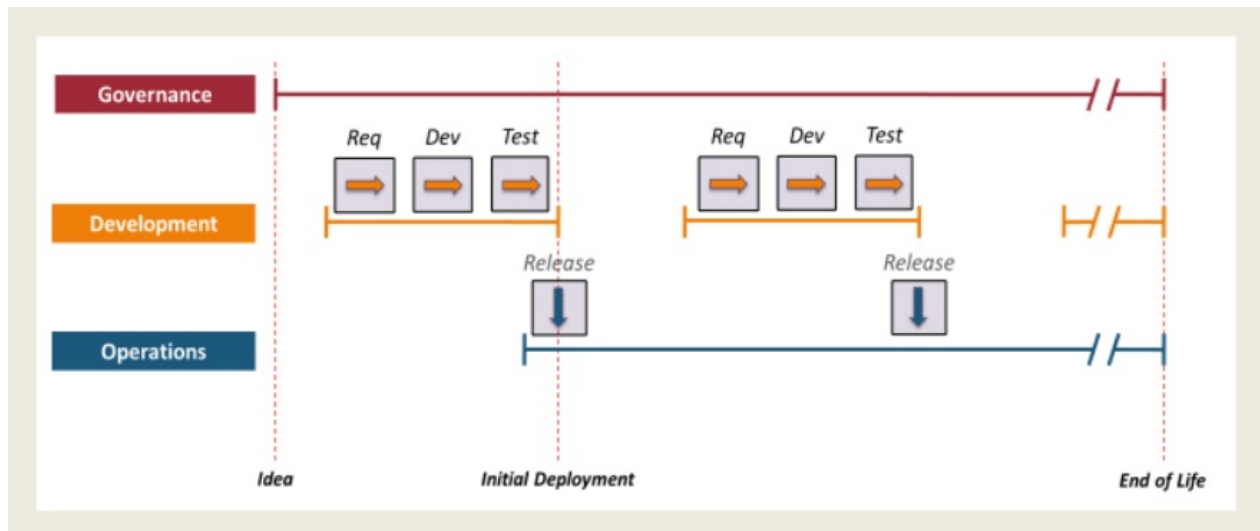
ALM also is concerned with the transmission of information and artifacts so that anyone working in development at any point in the product lifecycle has the correct and accurate information they need to accomplish work in a timely and cost effective manner.

The current updated version of ALM integrates with the three main SDLC approaches used today: traditional waterfall, Agile methods and DevOps style continuous development. Each of these models of integration breaks any SDLC down into three fundamental activities:

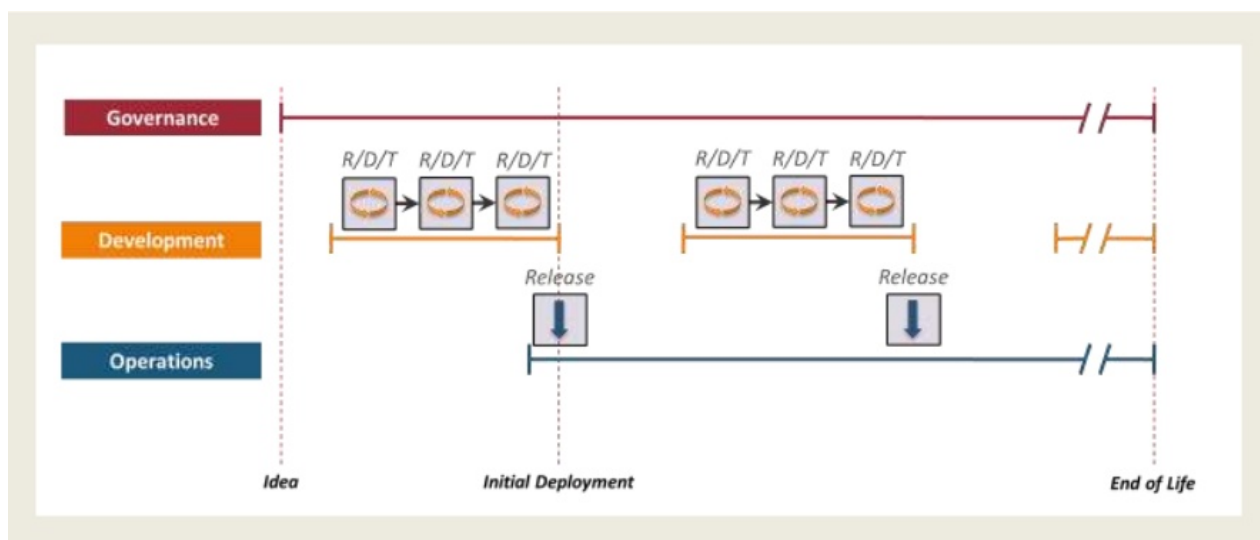
1. Requirements: working out in detail what the application should do.
2. Development: building the code and other software artifacts.
3. Test: ensuring that what was built is in fact what was supposed to be built.

Where the SDLCs differ is in the timing and organization of the activities . For example, the diagram following shows the ALM with a traditional waterfall SDL:

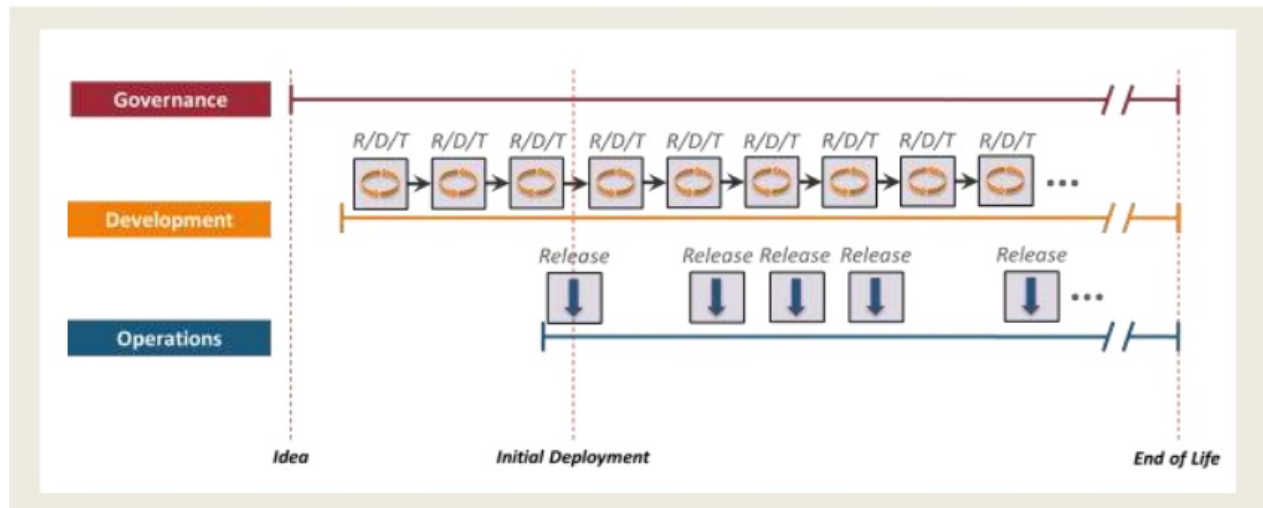




And the next diagram following shows the ALM aspects integrated with a standard Agile development where the three activities are cycled through iteratively.



The final diagram shows a continuous delivery (DevOps like) integrated into the ALM aspects.



In all of these models, especially the last two, requirements is an ongoing activity throughout the lifecycle of the product.

### Operations

During deployment, governance makes decisions and development makes changes. Operations provides the context for making those two threads work together. Operations monitors usage, performance, service level targets, change requests and bug reports. Operations also monitors changes in the application's technical environment, such as operating system changes or architectural changes, that could impact the application.

This data is used by governance to make a decision to effect some change, and by development to quantify and scope the needed changes. Operations also manages the integration of the product into its operational environment, including the installation of any upgrades and enhancements. We usually find that the feedback from operations is the inputs into the ongoing requirements activities.

## 7.3 Risk Analysis

The two ever constant concerns that feed into test planning are risk and quality. Generally, we are always concerned with different levels of both risk and quality, because these directly influence our test plans and quality control standards. And of course risk comes in different forms.

### Product Risk

Product risk is described by Beizer as the concern that the product will not work as it is supposed to or will may not perform correctly or robustly enough to result in a negative outcome. As an example of product risk consider the following case study of the USS Yorktown presented here in an abridged form.

#### ***The Navy's Smart Ship technology may not be as smart as the service contends.***

*Although PCs have reduced workloads for sailors aboard the Aegis missile cruiser USS Yorktown, software glitches resulted in system failures and crippled ship operations, according to Navy officials.*

*The Yorktown last September suffered a systems failure when bad data was fed into its computers during maneuvers off the coast of Cape Charles, Va. The ship had to be towed into the Naval base at Norfolk, Va., because a database overflow caused its propulsion system to fail.*

*The Yorktown lost control of its propulsion system because its computers were unable to divide by the number zero, the memo said. The Yorktown's Standard Monitoring Control System administrator entered zero into the data field for the Remote Data Base Manager program. That caused the database to overflow and crash all LAN consoles and miniature remote terminal units, the memo said.*

*The program administrators are trained to bypass a bad data field and change the value if such a problem occurs again, Atlantic Fleet officials said.*

This is product risk because the system was not robust enough to reject a bad input and continue functioning. If this sort of thing could happen in a combat situation, then the outcome might be very bad indeed. Of course if you are thinking like a tester you might ask "Why didn't anyone see what would happen if you put a 0 in that field?" And if you are thinking like an Agile tester you might ask "Why was it possible to even enter a 0 in that field?"

### Business Risk

This is where the product works perfectly but its use can have unintended consequences. Generally this sort of analysis is done during any sort of high level analysis which often focuses on questions like:

1. Can this product be used in ways it was not intended to be used that would have negative consequences?
2. Could the use of this product have a negative impact on the use of other products?
3. Could this product be modified or reverse engineered in a way that could have a negative impact on our business?

Just a quick example. Encryption technology has many positive purposes however we are seeing a rise in what is called ransom-ware – malicious actors encrypt an organization's data and refuse to decrypt it until a ransom is paid. When the encryption software was being developed, this was probably not a consideration but if this was recognized as a potential business risk, then ways of looking at mitigating this risk during design could have been considered.

### Project Risk

This is the risk that the project will not complete successfully. Part of the job we do as test testers is to look for the risk factors that might impact the project and ensure they have been mitigated. Some of the typical project risk factors are:

1. Reliance on a proprietary technology from a single vendor.
2. The need for a specialized skill set among developers that may not be readily available.
3. Unreliable infrastructure or possible resource problems.
4. Key components relying on buggy or poorly performing technology.
5. Possible legal issues such as patent infringements or non disclosure agreements.

### 7.3.1 Risk Matrix

#### A Critical Tool for Assessing Project Risk<sup>3</sup>

written by: Sidharth Thakur. edited by: Ginny Edwards 6/13/2015

*Confused about how to manage risks? Can't decide which to deal with first? Then the risk assessment matrix is the tool that you need to prioritize and develop an effective strategy. Find out more about this helpful tool.*

*A risk assessment matrix is a project management tool that allows a single page – quick view of the probable risks evaluated in terms of the likelihood or probability of the risk and the severity of the consequences.*

*A risk assessment matrix is easier to make, since most of the information needed can be easily extracted from the risk assessment forms. It is made in the form of a simple table where the risks are grouped based on their likelihood and the extent of damages or the*



kind of consequences that the risks can result in. A sample risk assessment matrix can be downloaded for free from [here](#).

Making a risk management matrix is the second step in the process of risk management, and it follows the first step of filling up a risk assessment form to determine the potential risks. The preparation of risk assessment forms is a more elaborate task and involves determining risks, gathering risk data, determining the probability and the impact levels of the risks, understanding consequences, assigning priorities and developing risk prevention strategies. On the other hand, a risk assessment matrix just provides the project team with a quick view of the risks and the priority with which each of these risks needs to be handled.

| Severity<br>Frequency | Catastrophic | Critical | Marginal | Negligible |
|-----------------------|--------------|----------|----------|------------|
| Frequent              | Extreme      | High     | Serious  | Moderate   |
| Probable              | High         | High     | Serious  | Moderate   |
| Occasional            | High         | Serious  | Moderate | Low        |
| Remote                | Serious      | Moderate | Moderate | Low        |
| Improbable            | Moderate     | Moderate | Moderate | Low        |
| Prevented             | None         | None     | None     | None       |

Also in project planning, a different type of risk assessment template can be created in Excel and used to assess the overall risk of initiating a project.

### *How to Place Risks in the Matrix*

As mentioned above, in a risk assessment matrix risks are placed on the matrix based on two criteria:

1. **Likelihood:** the probability of a risk
2. **Consequences:** the severity of the impact or the extent of damage caused by the risk.

### *Likelihood of Occurrence*

Based on the likelihood of the occurrence of a risk the risks can be classified under one of the five categories:

1. **Definite:** A risk that is almost certain to show-up during project execution. If you're looking at percentages a risk that is more than 80% likely to cause problems will fall under this category.
2. **Likely:** Risks that have 60-80% chances of occurrence can be grouped as likely.
3. **Occasional:** Risks which have a near 50/50 probability of occurrence.
4. **Seldom:** Risks that have a low probability of occurrence but still can not be ruled out completely.
5. **Unlikely:** Rare and exceptional risks which have a less than 10% chance of occurrence.

### *Consequences*

The consequences of a risk can again be ranked and classified into one of the five categories, based on how severe the damage can be.

1. **Insignificant:** Risks that will cause a near negligible amount of damage to the overall progress of the project.
2. **Marginal:** If a risk will result in some damage, but the extent of damage is not too significant and is not likely to make much of a difference to the overall progress of the project.
3. **Moderate:** Risks which do not impose a great threat, but yet a sizable damage can be classified as moderate.
4. **Critical:** Risks with significantly large consequences which can lead to a great amount of loss are classified as critical.
5. **Catastrophic:** These are the risks which can make the project completely unproductive and unfruitful, and must be a top priority during risk management.



### **Using the Risk Assessment Matrix**

Once the risks have been placed in the matrix, in cells corresponding to the appropriate likelihood and consequences, it becomes visibly clear as to which risks must be handled at what priority. Each of the risks placed in the table will fall under one of the categories, for which different colors have been used in the sample risk assessment template provided with this article. Here are some details on each of the categories:

1. The risks that fall in the cells marked with 'E' (red color), are the risks that are most critical and that must be addressed on a high priority basis. The project team should gear up for immediate action, so as to eliminate the risk completely.
2. High Risk: Denoted with 'H' with a pink background in the risk assessment template, also call for immediate action or risk management strategies. Here in addition to thinking about eliminating the risk, substitution strategies may also work well. If these issues cannot be resolved immediately, strict timelines must be established to ensure that these issues get resolved before they create hurdles in the progress.
3. Medium: If a risk falls in one of the orange cells marked as 'M', it is best to take some reasonable steps and develop risk management strategies in time, even though there is no hurry to have such risks sorted out early. Such risks do not require extensive resources; rather they can be handled with smart thinking and logical planning.
4. Low Risk: The risks that fall in the green cells marked with 'L', can be ignored as they usually do not pose any significant problem. However still, if some reasonable steps can help in fighting these risks, such steps should be taken to improve overall performance of the project.

## 7.4 The Documents

The IEEE 829 standard defines eight different documents each of which is composed of a number of sections. Since these specifications may not be available to us during class, I have given an overview in the to older set of documents so that the level of data and planning for test management can be appreciated.

### 7.4.1 Test Plan

The test plan is composed of the following sections:

1. *Test plan identifier* - A unique identifier so that this document can be distinguished from all other documents.
2. *Introduction* - A summary of the software to be tested. A brief description and history may be included to set the context. References to other relevant documents useful for understanding the test plan are appropriate. Definitions of unfamiliar terms may be included.
3. *Test items* - Identifies the software items that are to be tested. The word "item" is purposely vague. It is a "chunk" of software that is the object of testing.
4. *Features to be tested* - Identifies the characteristics of the items to be tested. These include functionality, performance, security, portability, usability, etc
5. *Features not to be tested* - Identifies characteristics of the items that will not be tested and the reasons why.
6. *Approach* - The overall approach to testing that will ensure that all items and their features will be adequately tested.
7. *Item pass/fail criteria* - The criteria used to determine whether each test item has passed or failed testing.
8. *Suspension criteria and resumption requirements* - The conditions under which testing will be suspended and the subsequent conditions under which testing will be resumed.
9. *Test deliverables* - Identifies the documents that will be created as a part of the testing process.
10. *Testing tasks* - Identifies the tasks necessary to perform the testing.
11. *Environmental needs* - Specifies the environment required to perform the testing including hardware, software, communications, facilities, tools, people, etc.
12. *Responsibilities* - Identifies the people/groups responsible for executing the testing tasks.
13. *Staffing and training needs* - Specifies the number and types of people required to perform the testing, including the skills needed.
14. *Schedule* - Defines the important key milestones and dates in the testing process.



15. *Risks and contingencies* - Identifies high-risk assumptions of the testing plan. Specifies prevention and mitigation plans for each.
16. *Approvals* - Specifies the names and titles of each person who must approve the plan.

### **7.4.2 Test Design Specification**

The test design specification identifies the features to be tested and describes a set of test cases that will adequately test those features. The test design specification is composed of the following sections:

1. *Test design specification identifier* - A unique identifier so that this document can be distinguished from all other documents.
2. *Features to be tested* - Identifies the test items and the features that are the object of this test design specification.
3. *Approach refinements* - Specifies the test techniques to be used for this test design.
4. *Test identification* - Lists the test cases associated with this test design. Provides a unique identifier and a short description for each test case.
5. *Feature pass/fail criteria* - The criteria used to determine whether each feature has passed or failed testing.

### **7.4.3 Test Case Specification**

The test case specification specifies in detail each test case listed in the test design specification.

1. *Test case specification identifier* - A unique identifier so that this document can be distinguished from all other documents.
2. *Test items* - Identifies the items and features to be tested by this test case.
3. *Input specifications* - Specifies each input required by this test case.
4. *Output specifications* - Specifies each output expected after executing this test case.
5. *Environmental needs* - Any special hardware, software, facilities, etc. required for the execution of this test case that were not listed in its associated test design specification.
6. *Special procedural requirements* - Defines any special setup, execution, or cleanup procedures unique to this test case.
7. *Intercase dependencies* - Lists any test cases that must be executed prior to this test case.

### **7.4.3 Test Procedure Specification**

The test procedure specification describes the steps for executing a test case and the process for determining whether the software passed or failed the test.

The test procedure specification is composed of the following sections:

1. *Test procedure specification identifier* - A unique identifier so that this document can be distinguished from all other documents.
2. *Purpose* - Describes the purpose of the test procedure and its corresponding test cases.
3. *Special requirements* - Lists any special requirements for the execution of this test procedure.
4. *Procedure steps* - Lists the steps of the procedure. Possible steps include: Set up, Start, Proceed, Measure, Shut Down, Restart, Stop, and Wrap Up.

### **7.4.4 Test Item Transmittal Report**

The identifies the test items being transmitted for testing. It includes the person responsible for each item, its physical location, and its status. Any variations from the current item requirements and designs are noted in this report.

The test item transmittal report s composed of the following sections:

1. *Transmittal report identifier* - A unique identifier so that this document can be distinguished from all other documents.
2. *Transmitted items* - Lists the items being transmitted for testing including their version or revision level.
3. *Location* - Identifies the location of the transmitted items.
4. *Status* - Describes the status of the items being transmitted. Include any deviations from the item's specifications.
5. *Approvals* - Specifies the names and titles of all persons who must approve this transmittal.

### **7.4.5 Test Log**

The test log provides a chronological record about relevant details observed during the test execution.

The test log is composed of the following sections:

1. *Test log identifier* - A unique identifier so that this document can be distinguished from all other documents.



2. *Description* - Identifies the items being tested and the environment under which the test was performed.
3. *Activity and event entries* - For each event, lists the beginning and ending date and time, a brief description of the test execution, the results of the test, and unique environmental information, anomalous events observed, and the incident report identifier if an incident was logged.

### **7.4.6 Test Incident Report**

The test incident report documents any event observed during testing that requires further investigation.

The test incident report is composed of the following sections:

1. *Test incident report identifier* - A unique identifier so that this document can be distinguished from all other documents.
2. *Summary* - Summarizes the incident.
3. *Incident description* - Describes the incident in terms of inputs, expected results, actual results, environment, attempts to repeat, etc.
4. *Impact* - Describes the impact this incident will have on other test plans, test design specifications, test procedures, and test case specifications. Also describes, if known, the impact this incident will have on further testing.

### **7.4.7 Test Summary Report**

The test summary report summarizes the results of the testing activities and evaluates those results.

The test summary report is composed of the following sections:

1. *Test summary report identifier* - A unique identifier (imagine that!) so that this document can be distinguished from all other documents.
2. *Summary* - Summarizes the evaluation of the test items.
3. *Variance* - Reports any variances from the expected results.
4. *Comprehensive assessment* - Evaluates the overall comprehensiveness of the testing process itself against criteria specified in the test plan.
5. *Summary of results* - Summarizes the results of the testing. Identifies all unresolved incidents.
6. *Evaluation* - Provides an overall evaluation of each test item including its limitations.
7. *Summary of activities* - Summarizes the major testing activities by task and resource usage.
8. *Approvals* - Specifies the names and titles of each person who must approve the report.



## Non- Functional Testing

### *Module Eight*

*Quality in a service or product is not what you put into it. It is what the customer gets out of it.*

Peter Drucker

*Quality - Those attributes of a product or service that the customer attaches value to. Depending on the customer's focus, "Quality" may include surface finish, timeliness, size, cost, reliability, or other factors*

Joseph R Jablonski

*Requirements capture is the process of finding out, usually under difficult circumstances, what is to be built. In fact, it is so difficult that it is still not uncommon for project teams to start writing code (which is fairly easy) before they have firmed up just what the code is supposed to do (which is difficult).*

Ivar Jacobson



## 8-1 Introduction

Non-functional testing has traditionally been sort of a "kitchen utility drawer" of testing where anything that wasn't actually functional testing was tossed into the bin labeled "non functional testing."

However, over the years, various parts of non-functional testing developed until they were sophisticated and comprehensive enough to be considered an independent testing type: this has happened with acceptance testing, usability testing, security testing and is still happening with other types of non functional testing.

For the purposes of this course, we focus on what has often been considered the core of what non-functional testing deals with which I will refer to as performance testing. In general, performance testing is concerned with how the system or application operates or performs along a number of operational dimensions such as capacity, reliability, throughput and others. The criteria I am using for calling something performance testing is based on the following characteristics.

### *Universality*

Functionality is specific to a system. No one expects a messaging app to work like the software in a nuclear reactor cooling system – they are designed to do different jobs.

However, no matter what its functionality every systems can be assessed by exactly the same set of performance indicators. No matter what they do, the non-functional properties of systems that we test are always the same. For example, every system performs with a certain response time no matter what it is doing.

### *Statistical Measurement*

Unlike functional quantitative testing, we cannot have a test case with an input and expected output. Instead, we usually have to describe the expected performance in statistical terms:

The system shall respond to a query withing 1ms 99% of the time and in no case shall have a response time greater than 20ms.

The system shall be able with handle up to 1 million simultaneous users anytime with no more than a 3% degradation in throughput and up to 2 million users for an average 10 minute period with no more than a 5% degradation in throughput.

In the event of a system crash, the system must be recovered and responsive within 1 second with a 99% confidence interval.

Performance measures tend to be described in terms of averages and allowable variances.

### ***Engineering Models***

Most of the work on developing non-functional performance testing models has been done by computer engineers who were concerned with performance issues of software like aircraft autopilots, power control systems and medical devices. In these sorts of mission critical applications, the performance of the software becomes often the most important aspect of the product in terms of quality and risk. As a result, we tend to have access to a large body of material from Quality Engineering that deals with performance testing.

### ***Specialized Test Environments***

Very often, we cannot do performance testing incrementally. We can on a component by component basis in some cases, but generally we have to do our performance testing only after all the functionality is operational and tested.

Even more critical in many cases is doing the performance in a test environment that simulates the actual production or operational environment as closely as possible. However, maintaining a clone of a production environment or close to production environment can be incredibly expensive.

One alternative that is often used, but only after an appropriate risk analysis is parallel testing where the system being tested works in parallel with the existing system. There are a variety of ways to do this sort of testing but it is not always possible.

### ***Automation Tools***

Almost every aspect of performance testing is done through the use of automated tools.



## 8-2 Types Performance Testing

These have all been traditionally lumped together under systems testing. It will be an exercise to identify other kinds of performance testing.

### Volume Testing

The system is subjected to heavy loads of data in order to test for failures at high volumes. This testing is intended to find out whether the system can or cannot handle the volume of data specified in the specification. Generally when we are concerned at the rate at which the data moves through the system to complete certain kinds of transactions, we often refer to this as transaction through-put testing.

### Stress Testing

The system is subjected to the maximum capacity it is specified to handle for a specified period of time. Peak capacities of systems are usually stated in terms of a percentage of normal or average loads. For example, the telephone system peaks at 125% of normal capacity in the period right after the start of business. Failure to test to an acceptable stress level allows for the possibility that a spike in system activity could cause a crash or other sort of failure.

### Performance Testing

Attempts to locate areas where the system does not meet performance requirements such as response time, simultaneous activities and throughput. A performance requirement might expressed "The system shall respond to all requests in under 1ms 99% of the time with a maximum response time of 5ms. The system shall be able to maintain this response time while there are up to 100,000 users online and processing up to 50,000 simultaneous transactions."

### Storage Testing

This tests data storage along several parameters: the amount of data that can be stored, the access rates when searching data and the transfer rate of data in and out of storage, normally broken down into read rates and write rates.

### Configuration Testing

Tests to ensure the system performs to spec under all of the combinations of hardware and software configurations allowed in the specification.

## Compatibility Testing

Checks to ensure that the system under development is compatible with other parts of the existing system or application. A classic case of the failure to do compatibility testing is the Mars Climate Orbiter failure<sup>1</sup>.

*1999: A disaster investigation board reports that NASA's Mars Climate Orbiter burned up in the Martian atmosphere because engineers failed to convert units from English to metric. The \$125 million satellite was supposed to be the first weather observer on another world. But as it approached the red planet to slip into a stable orbit Sept. 23, the orbiter vanished. Scientists realized quickly it was gone for good.*

*"It was pretty clear that morning, within half-an-hour, that the spacecraft had more or less hit the top of the atmosphere and burned up," recalled NASA engineer Richard Cook, who was project manager for Mars exploration projects at the time. A NASA review board found that the problem was in the software controlling the orbiter's thrusters. The software calculated the force the thrusters needed to exert in pounds of force. A separate piece of software took in the data assuming it was in the metric unit: newtons.*

*There had been warning signs, the mishap board found. En route to Mars, the spacecraft had to make 10 to 14 times more minor adjustments than engineers expected. And the last few signals from the orbiter indicated that it was dipping dangerously low into the Martian atmosphere, about 105 miles lower than it was supposed to go. Ultimately, the Mars Climate Orbiter came within 37 miles of the Martian surface. Simulations showed that, at any altitude lower than 53 miles, atmospheric friction would tear the fragile craft apart.*

*The whole thing could be written off as a miscommunication. Propulsion engineers, like those at Lockheed Martin who built the craft, typically express force in pounds, but it was standard practice to convert to newtons for space missions. One pound of force is about 4.45 newtons. Engineers at NASA's Jet Propulsion Lab assumed the conversion had been made, and didn't check.*

## Installability Testing

Tests for failures to meet specification regarding the installability of the system as well as the ability of the system to be upgraded as required.

## Reliability Testing

Tests for failures to meet specific reliability specifications. For example, a system may have a reliability specification of two hours down time per forty years of operation. Clearly this is difficult to test however there are mathematical models borrowed from engineering which can be used if the system has operational specification like mean-time-to-failure (MTTF = 20 hours), of no more than eight errors after being placed into production.

## Recovery Testing

Systems may fail for a variety of reasons and one important specification is how the system will recover from a disaster. A recovery test would check to see if the system recovered from a disaster according to spec. This is often called failover testing.

The next several types of testing are from Lisa Crispin.



## Scalability

Scalability testing verifies the application remains reliable when more users are added. What that really means is, “Can your system handle the capacity of a growing customer base?” It sounds simple, but really isn’t, and is a problem that an agile team usually can’t solve by itself.

## Maintainability

Maintainability is not something that is easy to test. In traditional projects, it’s often done by the use of full code reviews or inspections. Agile teams often use pair programming, which has built-in continual code review. There are other ways to make sure the code and tests stay maintainable.

We encourage development teams to develop standards and guidelines that they follow for application code, the test frameworks, and the tests themselves. Teams that develop their own standards, rather than having them set by some other independent team, will be more likely to follow them because they make sense to them.

What they are talking about here is what we start to see at the level four maturity models. The development of best practices and standards in both testing and development, and hopefully of the business analysis side too, that allow a smooth and consistent process for maintaining the velocity of the work even while designs and other artifacts have are in a state of flux.

Maintainable code supports shared code ownership. It is much easier for a programmer to move from one area to another if all code is written in the same style and easily understood by everyone on the team. Complexity adds risk and also makes code harder to understand. The XP value of simplicity should be applied to code. These same concepts apply to test frameworks and the tests themselves.

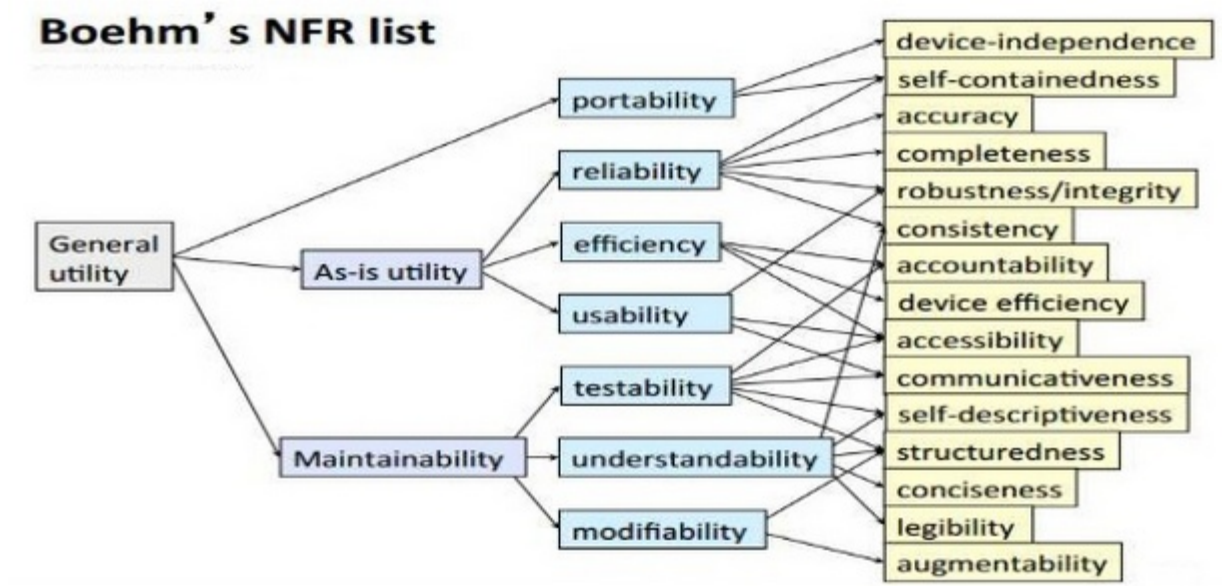
Maintainability is an important factor for automated tests as well. Database maintainability is also important. The database design needs to be flexible and usable. Every iteration might bring tasks to add or remove tables, columns, constraints, or triggers, or to do some kind of data conversion. These tasks become a bottleneck if the database design is poor or the database is cluttered with invalid data.

What is being stressed here are the same principles we see in XP with its concepts of robust design spikes, shared code base and coding standards. This sort of testing approach tends to integrate well with the development discipline of Software Craftsmanship.

## 8.3 Non-Functional Requirements

One of the most important tasks in performance testing is to ensure that we have adequate quality control in mapping the non-functional requirements into quantitative specifications. Recall the idea of testability from the IEEE checklist – we need to be able to write a pass or fail test – but one of the problem we often encounter is that stakeholders have problems quantifying their non-functional requirements. We really can't write a proper test to the requirement "It should be really fast even if there are lot of users on the system." Of course this problem is compounded when stakeholders have to deal with a statistical model as a quantification of their requirement.

It has been an ongoing engineering problem for decades to create comprehensive models of non-functional requirements like the Barry Boehm categorization shown below.



The conversion of qualitative non-functional requirements to quantitative specifications is hard often requires a lot of work on the part of the business analyst. Our primary job is to ensure that this is done, no non-functional requirements have been missed, the quantitative specification is correct and that there exists properly quantified tests for that specification.

One of the reasons this tends to be arduous for many analysts and stake holders is that the success criteria for a test often have to be expressed in statistical concepts such as mean values, confidence intervals and error ranges. These are topics we are not going to pursue in this class.

### Non-Functional Requirements Templates



Because non-functional requirements tend to be similar across systems, there have evolved a number of checklists or templates for documenting non-functional requirements. These templates often are a good starting point for developing non-functional requirements and associated tests during an Agile project.



### ***Furps+***

One of these standard templates is Furps+. This was originally developed by Robert Grady at HP and is based on a number of previous attempts to categorize non-functional requirements, including the Boehm model shown earlier.

The Furps+ checklist breaks down the requirements by category in the following manner:

#### **F. Functional Requirements**

##### **U. Usability**

- Human factors
- Aesthetics
- Interface consistency
- Documentation

##### **R. Reliability**

- Mean time to failure
- Failure severity
- Failure frequency
- Accuracy
- Predictability

##### **P. Performance**

- Speed
- Response time
- Throughput
- Efficiency
- Resource consumption
- Load and stress

### S. Supportability

- Testability
- Extensibility
- Adaptability
- Maintainability
- Compatibility
- Configurability
- Serviceability
- Installability
- Localizability

### +. Other factors

- Implementation constraints
- Interface constraints
- Design constraints
- Physical environment

## Early Non-functional Testing

While we may not be able to run non-functional tests until the latter part of the project, it is critical to ensure the quantitative specifications are developed early in the project, with an early version, perhaps not being full quantified, during iteration 0. The reason for this is that the non-functional requirements often play a major role in the choice of the candidate architectures.



## Reviews and Inspections

### *Module Nine*

*A computer lets you make more mistakes faster than any invention in human history - with the possible exceptions of handguns and tequila.*

Mitch Ratcliffe

*Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live*

Martin Golding

*Any code of your own that you haven't looked at for six or more months might as well have been written by someone else.*

Eagleson's Law



## 9.1 Introduction

This module is going to be more general than just reviews and inspections, but will cover a number of related sorts of what might be called human based techniques that are used in a number of different testing contexts.

One of the important things to remember is that these human based techniques are not a type of testing, like functional or non-functional, but are methods that are widely used in quality assurance, acceptance testing, usability testing, security testing and other test contexts.

These techniques are not native to software testing, but rather they are common methods that are used in a wide range of test environments – market research, engineering design and use interface design for example – but also are used to do exploratory testing: we use these techniques to try and identify potential risks, problems and errors that we might make but also to try and see if we can uncover issues or problems that we haven't yet identified.

### 9.1.2 Qualitative Measurement

One of the IEEE qualities that we identified as being desirable in a specification was "testability," which meant that each individual specification item had to be quantified. That is a desirable property for most of the testing we do, but when we are dealing with human based methods, we are not dealing with tests that can be objectively quantified.

Whenever we are doing human based testing, we are dealing with people's opinions, intuitions, gut feelings and value judgments – the measurements we use for testing are reports of subjective evaluations by users, developers, architects or designers.

We can "scale" these subjective measurements by using different ranking schemes:

1. "Compared to the previous interface, do you find the new interface easier or harder to use or about the same?"
2. "What are the three best features in our product? The three worst?"
3. "On a scale of 1 to 10, can you rank how likely you are to use each of the following features for our application where 1 means 'not at all' and 10 means 'always'?"

When we deal with data like this, we call it qualitative since it ranks, groups or categorizes subjective reports in a way that makes it easier for us to analyze it.

The other thing we have to keep in mind is that sometimes people's judgments and evaluations cannot be explained easily, especially with highly experienced subject matter experts. This issue is especially important because these sorts of expert intuitions are often some of our best qualitative testing resources.

## 9.2 Sources of Quality Baselines

A lot of what we will be doing in human based testing is developing best practices and learning from mistakes that have been made either in the past or by others doing similar things to what we are doing.

There is also a validity testing component that falls under the techniques of exploratory testing. In most cases, testers are not actually doing this activities, often it is the business analyst, but the tester plays the role of a quality engineer and helps assemble this data into standards, best practices and other sorts of guidelines that reduce the probability of the team making errors.

There are four main sources information that we will want to examine when developing our guidelines and doing our exploratory testing.

1. Comparative and historical data.
2. Observational data.
3. Research on passive stakeholder requirements.
4. Direct interaction with the active stakeholders.

### ***9.2.1 Historical Data:***

We can start to mitigate the potential errors that could occur in our project by looking at problems in past projects which were done within the same organization or with generally the same sets of stakeholders. Often we find that over time, the projects done by a specific organization tend to group naturally into generic project types. Once we have an understanding as to what type of project we are working on, then the past experience from other projects of the same type becomes a guideline for our current project.

This is particularly useful for non-user requirements and passive stakeholder requirements where errors tend to be easier to make. It is useful to look at the requirements that have changed as the previous projects were rolled out. This works best when the organization keeps historical data.

Think of this as learning from past mistakes.

### ***9.2.2 Comparative Data:***

We can also get insights into what might go wrong in our project by looking at what happened in similar projects elsewhere. Similar does not mean identical but it does mean that the projects can be thought of the same “type.” For example, did another project forget to look at the mandatory requirements of a regulatory agency and wind up doing massive rework to become compliant to the missed requirements?



Looking at similar projects helps us identify potentially similar sets of stakeholders and requirements, especially those stakeholders that are outside the other organization and would likely also be our stakeholders – government agencies for example.

Think of this as learning from other people's mistakes

### ***9.2.3 Observational Data***

Observational data is obtained by watching the stakeholders in action. This is useful when the stakeholders are unable to or are unwilling to describe their requirements or if there are political or other factors that may make the requirements unreliable. By watching them in action in their current situation we can see what sort of processes they follow and what sort of requirements can be empirically deduced from observation as opposed to those that are "officially" relayed to the BAs.

For example: In designing an ATM, we watch the customers interacting with the tellers to see what sort of functional requirements they might have as opposed to just accepting what the bank tells us what the customer requirements are. If the bank is wrong, our acceptance tests will probably fail.

Observational data can be collected overtly or covertly meaning that we can either let the stakeholders know we are collecting the data or we can do it on the sly. The problem with overt collection is that knowing that they are being observed may cause the stakeholders to change what they do so we wind up getting skewed or tainted data. The problem with covert collection of data is that there are often legal and privacy issues that have to be considered before we can embark on covert research.

A form of observational data collection is what we call historical observation. If we have access to enough historical information that we can search thorough the data to look for patterns of behaviour. These techniques are often lumped together under the general category of "data mining." The reliability of data mining is a topic that is currently hotly debated in the requirements community. Some suggest that patterns in historical data are indicators of what requirements actually are while others suggest that the patterns found may often be random events that only appear meaningful and, in turn, would skew requirements investigation in bogus directions.

This has also been complicated by the development of big data and analytics which seem to show that what people say they do and what they actually do is out of sync far more than we had previously thought. For example, a breakdown of Google searches by demographic groups show very different results of what people search for as opposed to what people say they search for.

## 9.3 Group Techniques

There are three kinds of group data collection we will touch on in this course.

1. Surveys.
2. Focus Groups.
3. JAD Sessions.

These represent three main types of techniques, but within each type are a number of variations.

Each technique has its own advantages and disadvantages, however as we will see, for testing work, the costs of these techniques may not be worth the value generated in certain types of testing, so they have to be used carefully. Some of these techniques can be resource intensive and it often takes some experience to determine the return on investment from a quality perspective as to how whether they should be used. All of these techniques are used extensively in other professions which engage in qualitative testing.

### 9.3.1 Surveys

Surveys are often perceived as the quick and fast way to get qualitative data but they are often the most expensive and difficult way to get accurate and usable data.

Survey upsides:

1. We can get data from a large number of stakeholders.
2. We can do all the stakeholders at the same time.
3. We can get a lot of data just by increasing the size of the survey.
4. Surveys can be effective if they are well done.

Survey downsides:

1. It can be very expensive and time consuming to do a survey well.
2. It is very difficult to discover actionable data using surveys – it's easy to miss things that should be asked when designing the survey.
3. It is very difficult to discover stakeholders using surveys.



## The Survey Cycle.

In order to be effective, surveys have to be developed in a specific way.

1. We meet with stakeholders to identify the kinds of data that should be collected in the survey.
2. We design the survey.
3. We test the survey on a small group.
4. We revise the survey.
5. We administer the survey.
6. We analyze the results and report.

### ***Planning the Survey***

The first step is identifying what we need to ask in the survey which means that we need to do some sort of initial analysis to identify the topics and focus of the survey. In other words, we need to do a “requirements analysis” for the survey.

### ***Designing the Survey***

We need to design the survey because not only does each question has to be designed correctly but the way that the questions are organized in the survey also has to be planned carefully. The survey result is unreliable if the survey “leads” the respondent to answer in a particular manner either by a specific question:

*“Would you support the continued corruption in the current government or vote for the opposition?”*

Or, for example, in multiple choice questions, the order of the answers will bias the respondents to tend to select the first option. We also have to ensure that the order of the questions is not introducing a bias. For example:

*“1. Do you think that those who support tax reform are fundamentally selfish?”*

*“ 2. Do you support tax reform?”*

Developing a survey is easy, but developing a survey that produces accurate results is hard.

### ***Testing the Survey***

Once the survey is developed, we need to test it to see that:

1. The questions are clear and understood.
2. There are no problems administering the survey (like a confusing sequence of questions or unanticipated answers.)

*“Have you ever been convicted of a crime for which you have not received a pardon?”*

Tests are usually done on a small sample of the target audience to ensure that we are measuring what we actually think we are measuring. After testing, the survey needs to be revised to take into account the results of the testing.

### ***Administering the Survey***

Now we have to decide how to get people to answer the survey. If we are not going to have everyone answer the survey, we need to survey a sample, which means we need a technique for sampling. However, the survey is useless if the sample is not representative.

Once we have the sample, we provide the survey to the sample. If we let people just fill in the survey and return it Our response rates are often very low; sometimes around 1%. Pestering people increases the response rates somewhat but not significantly.

Alternatively, we can use a forced technique, for example everyone is placed in a room and can't leave until the survey is complete. However, this may produce some poor quality results because a portion of the population will answer anything just to finish the survey.

The most effective technique is to have someone administer the survey to one respondent at a time. In other words, the survey becomes a structured interview. This is the approach generally taken in market research because it allows the administrator to identify problems and possible bogus responses while the survey is being administered. However, this is also the most expensive and time consuming way to administer a survey.

### ***Analyzing the Survey***

Once we have the results, the survey has to be analyzed by entering them into some data set that can be analyzed in a quantified manner, usually a form of statistical analysis.

The results are then interpreted, which can be quite difficult because it involves trying to understand what a statistical trend means. It may require a further survey or follow-up to clarify.



However, there are also confounding factors like people who deliberately answer in away to mislead (the “screw you” effect), or “outriders” or data points that are extreme and are usually the result of an incorrectly answered survey. These sorts of data have to be factored out of the results or the results will not be reliable. The technical term for this is “cleaning the data.”

### **When to use and not use surveys?**

1. Very brief and informal surveys are often good for quick and dirty data where a certain amount of inaccuracy can be tolerated.
2. Surveys are useful to identify trends that we may want to follow-up on: i.e. help us understand who we need to interview.
3. Surveys are useful when there is a large and diverse stakeholder community (e.g. shoppers, car drivers) that we need to interview.
4. Surveys are useful when face to face data collection is not feasible.
5. Surveys should not be used as a replacement for interviews.
6. Surveys should not be used for getting mission critical data unless they are properly designed and administered.

### **9.3.2 Focus Groups**

Focus groups are used extensively across industries. A focus group is a sample of 8-15 people who are guided through a session which is usually 1-3 hours and is run by a moderator who keeps the group on track. The moderator uses a script to guide the group activities and keeps the group focused on providing their feedback to a product or set of issues as presented by the moderator. The success of a focus group is highly dependent on the skill of the moderator.

Unlike surveys, focus groups are open ended so that new topics can emerge during the session. The results of the focus group are not quantitative and cannot be subjected to analysis -- they are qualitative.

Focus group results are often good predictors of survey results and are often used to identify the topic areas that should be covered by a survey.

Focus group composition is critical to the success of the group. The group should be a representative sample of the population. The reliability of the results improves when we can compare the results of multiple focus groups.

Focus groups are often recorded to produce “verbatim” which can be studied in detail later. In some cases focus groups are observed by interested parties either overtly or covertly. One has to be sensitive to privacy issues when recording or observing focus groups.

### **When to use focus groups?**

1. When you are probing or exploring requirements or reactions.
2. When you need to understand what a survey should ask.
3. When you want initial reaction to a prototype or first cut at an application.
4. When you want to find out what people like or don't like about the current system.

### **Hints for focus groups.**

1. The interaction between participants is often very revealing, more so than the interactions between moderator or participants.
2. Remember that focus groups are not for design or planning, they are for probing and discovery.
3. The skill of the moderator is often the key to success.

### **9.3.3 JAD – Joint Action Development**

JAD also stands Joint Application Design among others. Also called Joint Application Development or Design. Like a focus group, a JAD session is a moderated group activity but with the goal of finalizing some artifact so that everyone has signed off on the same requirements or design options from competing sets of requirements from multiple sets of stakeholders or multiple design choices.

Unlike a focus group, a JAD session is goal oriented in the sense that we are attempting to produce a deliverable through the overt collaboration of the participants.

JAD Sessions can last several days and can become quite heated so a strong moderator (acting as a referee) is needed to keep the session productive and not deteriorate into a brawl.

#### **The JAD sequence.**

1. The goals of the session are defined. e.g. "To finalize and assess how well the design of the web banking interface meets the user requirements and quality expectations."
2. The appropriate stakeholders identified. e.g.. Customers, bank staff, developers, marketing department, security staff, auditors.
3. Representatives for each stakeholder group are invited.
4. Each is asked to prepare a written preliminary analysis.
5. These are circulated in advance so everyone knows the positions and concerns of the other stakeholders.



6. The JAD session meets.
7. The stakeholders discuss the issues, agree and finalize the assessments. (Or at least in theory....)

### **When to use a JAD**

1. When there are various stakeholder groups with different requirements that are not consistent or complete.
2. When there needs to be a negotiation on what the requirements or design or quality levels should be.
3. When the process has ground to a halt because of communications or organizational issues.
4. When the stakeholders are isolated from each other and do not understand the concerns of other stakeholders

## Human Based Testing

Non-computer based testing, or human testing is the evaluation of a program or other artifact by a group of people, usually while the code is still being developed. We consider these methods to be part of the testing process rather than the development or coding processes because their purpose is to identify errors, not to correct them.

### ***9.4.1 Types of Inspections***

The following are the more commonly recognized types of inspections.

#### **SYSTEM REQUIREMENTS INSPECTION**

This inspects be the high-level requirements for software systems. This type of inspection may be applied to multiple levels of system requirements.

The purpose of the high-level requirements inspection is to:

1. Ensure proper allocation of functions to software, firmware, hardware, and operations.
2. Validate all external usage interfaces.
3. Verify that all the software system functions are identified and broken into configuration items.
4. Ensure all configuration items within the software system are identified.
5. Verify that the identified configuration items provide all functions required of them.
6. Ensure that all interfaces between configuration items within the software system are identified.
7. Verify correctness of the software system structure.
8. Ensure that all quantifiable requirements and requirement attributes have been specified.
9. Ensure that the requirements are verifiable.

#### **SOFTWARE REQUIREMENTS INSPECTION**

This inspects be the detailed requirements for specific software components and/or modules. The purpose of the software requirements inspection is to:

1. Verify a complete and accurate specification of each of the following:
  - a) Software functions
  - b) Input and output



- c) States and modes
  - d) Response time requirements
  - 3) Interfaces.
2. Ensure specifications are included for error detection and recovery, reliability, maintainability, performance, and accuracy.
  3. Ensure the traceability of requirements from higher level documents.
  4. Verify that the requirements provide a sufficient base for the software design.
  5. Verify that the requirements are measurable, consistent, and testable.

## **ARCHITECTURAL DESIGN INSPECTION**

This inspects the high-level software system design.

The purpose of the architectural design inspection shall be to:

1. Ensure the design meets approved requirements.
2. Validate all interfaces among modules within each component.
3. Review the list of modules and the general function(s) of each module.
4. Validate fault detection, identification, and recovery requirements.
5. Verify the component structure meets the requirements.
6. Validate the selection of reusable components.
7. Ensure traceability of the design to the approved requirements.
8. Validate input and output interfaces.

## **DETAILED DESIGN INSPECTION**

This inspects the software component and/or module design at the detailed level.

The purpose of the detailed design inspection is to:

1. Ensure that the design meets the approved requirements.
2. Validate all logic algorithms, data structures, and calls within each module.
3. Verify that the detailed design is complete for each module.
4. Ensure traceability of the design to the approved requirements.
5. Ensure that all required and/or applicable programming standards are followed.
6. Ensure that detailed design meets requirements and is traceable to the high-level software system design.

## SOURCE CODE INSPECTION

This inspects the module source code.

The purpose of the source code inspection is to:

1. Ensure that the code meets the approved requirements.
2. Verify the technical accuracy and completeness of the code with respect to the requirements.
3. Verify that the code implements the detailed design, and that all required/applicable standards are satisfied.
4. Ensure traceability of the code to the approved requirements.
5. Ensure that the code meets requirements and is traceable to the detailed design.

## TEST PLAN INSPECTION

This inspects a software test plan for the software capabilities required by the detailed level of requirements.

The purpose of the test plan inspection is to:

1. Detect defects and misconceptions in the definition of the test plan.
2. Ensure that all new and modified software functions operate correctly within the intended environment and according to approved requirements.
3. Ensure that all new and modified interfaces will be verified.
4. Identify and eliminate extraneous or obsolete test plans.
5. Ensure that each requirement will be tested.

## TEST PROCEDURES INSPECTION

This inspects test procedures for software capabilities required by the detailed level of requirements.

The purpose of the test procedures inspection is to:

1. Verify that the set of test procedures meets the objective of the test plan.
2. Verify that each test procedure provides:
  - a) A complete and accurate description of its purpose
  - b) A description of how it executes
  - c) All expected results



3. Ensure that each test procedure identifies which requirement(s) it is testing and correctly tests the listed requirement(s).
4. Ensure that each test procedure identifies the required hardware and software configurations.
5. Ensure that each test procedure will execute without errors.

### ***9.4.2 The Egoless Approach***

The key factor in a successful inspection is an egoless attitude on the part of the participants. To effectively use these techniques, team members must feel comfortable subjecting the programmer or person responsible for the artifact to a barrage of questions: What were the programmers intentions? What assumptions were made? Why was this logic used?

To keep things from getting personal, team members should direct their comments to the program and not the programmer. To avoid getting personal, team members must keep in mind that errors are not the fault of the programmer but are inherent in the task of programming.

### **Benefits of Human Testing Methods**

1. Bugs are caught early in the development cycle which implies the cost of fixing them is several orders of magnitude less than if they were identified in the testing phase. These methods can be applied before the code is written while computer based testing cannot be done until the code has reached the executable stage.
2. The actual errors in logic or coding can be identified and not just the symptoms or effects of these errors (i.e. the output of a successful test case).
3. It is easier to identify errors that come from faulty or incomplete understanding of the program requirements.
4. Empirical results show that these techniques tend to find batches of errors as opposed to the individual errors found by computer based testing.
5. Human based methods are efficient in uncovering code that does not conform to organizational standards. Adherence to standards leads to easier program maintenance.
6. Human based methods are very efficient in identifying both logic and coding errors. Studies suggest that up to 80% of all errors can be found by human testing methods.

### ***9.4.3 Code Walkthroughs***

Walkthroughs are sessions in which the participants act out the role of the computer by manually executing each line of code for a given set of test cases.

#### **The roles of the participants**

The team should consist of three to five members. The roles filled by the participants are:

##### ***Moderator***

The moderator is responsible for keeping things on track. The team should concentrate on finding errors and not on correcting them. He is also responsible for distributing materials and ensuring that the errors found were later corrected by the programmer.

##### ***Secretary***

The secretary is responsible for recording all errors found.

##### ***Tester***

This person is responsible for creating one or more test cases to be applied during the session and presenting the material to the team. This person is usually the program's author.

##### ***Observers***

Observers provide technical advice to the team as needed.

##### ***Participants***

Participants should include representatives from the following groups:

1. The program's author(s).
2. Experienced programmers involved in areas related to the module under scrutiny.
3. Novice programmers who will gain experience and add a fresh perspective to the process.
4. Maintenance – especially staff that will be responsible for maintaining the system.
5. Programmers not directly involved in the system (“outsiders”).
6. A programming language expert.
7. Domain experts, who have intimate knowledge of the business requirements.



***Management participation?***

Management should generally be excluded from participating in the walkthroughs. When the boss sits in during a peer review, the participants may feel that they are under review and not the program, which raises issues about conforming to authority.

**Rules for walkthroughs**

There are several key rules to a successful walkthrough:

1. Provide adequate time A session of 90 to 120 minutes seems to be the best length of time for a walkthrough. The time must be scheduled so that there are no interruptions. Sessions lasting longer than about two hours tend to be less productive.
2. Use multiple session when necessary Expect to cover between 100 to 150 lines of code per hour. Multiple sessions may be required to walk through a large program.
3. Prepare a set of test cases Even though this is a non-computer based test, test cases are needed to start the process. These should be presented by the tester. The content of a test case is not as important as in computer based testing. Instead, here the test case is a jumping off point. Keep the test cases simple since the session is limited in time and people cannot process data quite like a computer. Include flow charts and data dictionaries.
4. Provide a copy of the program being tested to each team member Distribute copies of the code to team members in advance so that everyone can be familiar with the program before the session.
5. Provide other support materials Other support materials may be required to efficiently conduct a walkthrough. These may include;
6. A list of questions prepared by each team member after reading the program or unit under review.

**Walkthrough steps**

The following procedure should be followed when conducting a walkthrough.

1. Using the test data provided by the tester, begin evaluating the code line by line and monitor all variables on paper or a blackboard. Also remember to monitor all output from the program as well. As you walk through the tests, remember not to rely on them alone to uncover errors. More errors are uncovered by questioning the programmer than by the test case itself.
2. Have the secretary note the errors as they are found.
3. When the session is over, compile a list of errors and give them to the programmer for correction and to other team members to ensure that nothing was missed.

4. At the end of the session, have the moderator evaluate the progress of the team. He should then schedule a walkthrough session for the next module or a follow-up session if all the modules have been tested. If a substantial number of errors have been found, the moderator may wish to reschedule a walkthrough for the same module.
5. Conduct a follow-up session to quickly review the errors found and the corrective actions taken.

### **The checklist**

After the program has been fully human tested, the list of errors should be studied and used to create a checklist of commonly occurring errors. This checklist is then used in future walkthroughs and code inspections to focus attention on problematic areas.

### **9.4.4 Code Inspections**

Code inspections, like walkthroughs, are peer reviews. However, where a walkthrough is a live simulation of the execution of the program, a code inspection examines the structural, logical and stylistic aspects of the program code.

Because a code inspection does not attempt to emulate program execution, it does not make use of test cases. Instead the code is critiqued using a checklist; which is a list of common coding errors and, where appropriate, the organization's documented code standards.

### **Roles**

Code inspections are usually conducted with a team of four members.

#### ***Moderator***

The moderator's responsibilities are:

1. to ensure that all team members have the appropriate materials such as flow charts, data dictionaries, program listings and other materials far enough in advance that everyone can be familiar with the program under inspection;
2. to ensure that all participants have a copy of the organization's standards; and
3. to schedule the session and record errors.

#### ***Programmer***

The programmer reads the code, line by line to the other team members.



**Designer**

The designer must ensure that the suggestions made during the inspection do not negatively impact on the rest of the system. As well, the designer is there to note where other parts of the system need to be redesigned due to the findings of the inspection.

**Test specialist**

The test specialist provides assistance to the team based on his experience with common programming errors.

**Duration**

Sufficient uninterrupted time is critical to the success of the inspection. Usually the time required is 90 to 120 minutes.

**Steps**

1. The programmer reads the code line by line and comments on the logic used and the reasons for its choice. Other team members raise questions and evaluate each line to see if errors are present. Because the programmer is the most familiar with the code, he is usually the one who spots the most errors. The act of just reading the code aloud often assists the programmer in uncovering significant errors.
2. Team members evaluate the program based on a known checklist. The checklist will contain all the known and common errors that can be expected to be found.
3. Errors found, but not on the checklist should be recorded by the moderator to further refine the current checklist. During a typical session, things may get a little off track. Some lines of questioning may not be that productive, but the moderator is expected to keep things rolling and avoid tangents.

In most code inspections, the job of team members is to find errors, not fix them.

A alternate approach to code inspections changes the basic function from error detection to error detection and correction. While this adds to the length of time a code inspection takes, it is often true that the identification of a bug suggests an immediate solution.



## Logic and Model Testing

### *Module Ten*

*We do not see the world as it is but rather as we are.*

Anian Nin

*I have yet to see any problem, however complicated, which, when you looked\* at it in the right way, did not become still more complicated.*

Poul Anderson

*Everything we hear is an opinion, not a fact. Everything we see is a perspective, not the truth.*

Marcus Aurelius



## 10. 1 Introduction to Models

We will use the terms model and modeling quite a bit in this module without any sort of rigorous definition; instead I will assume that everyone understands what these terms mean, and that our general common understanding of these terms will be adequate for a while.

It has been argued that the primary activity that separates humans and animals is our ability to create abstract models. However, it now appears that model making is an essential part of all cognition, whether human or not. I

And so we find models in every aspect of software development: object models, data models, functional models, state models, logic models and a whole lot more. Whenever people engage in this sort of formal modeling activity, there are certain questions that always seem to come up, and should in the mind of every software tester.

1. How do I know when my model is right? What does it mean for a model to be right?
2. How do I know what level of detail to put into my model? What is the right level of granularity? What can I leave out?
3. My model is different than someone else's model of the same thing, how can they both be right? Can one be more right than the other?
4. How do I know when my model is complete enough to stop modeling?

The answer to all these questions, and the answer that most people don't want to hear is that it is a subjective judgment on the part of the modeler. However, we also saw that notion of subjectivity somewhere else – when we dealt with the concepts of quality. In a sense, model testing is an interesting fusion of structural, functional and quality testing.

There are also some facts that make this sort of testing easier for us because of the following two factors.

1. Formal modeling, like we are doing in this course, is just a special case of informal modeling, which is something every one does all the time. It's not some new esoteric or abstruse technical area like multivariate calculus or quantum mechanics. You already are a modeler.
2. As a consequence of the the previous fact, you have an intuition good enough to answer all of the questions above when you model, it is just a matter of hooking up that intuition you have from using informal models so that it works with these formal models.

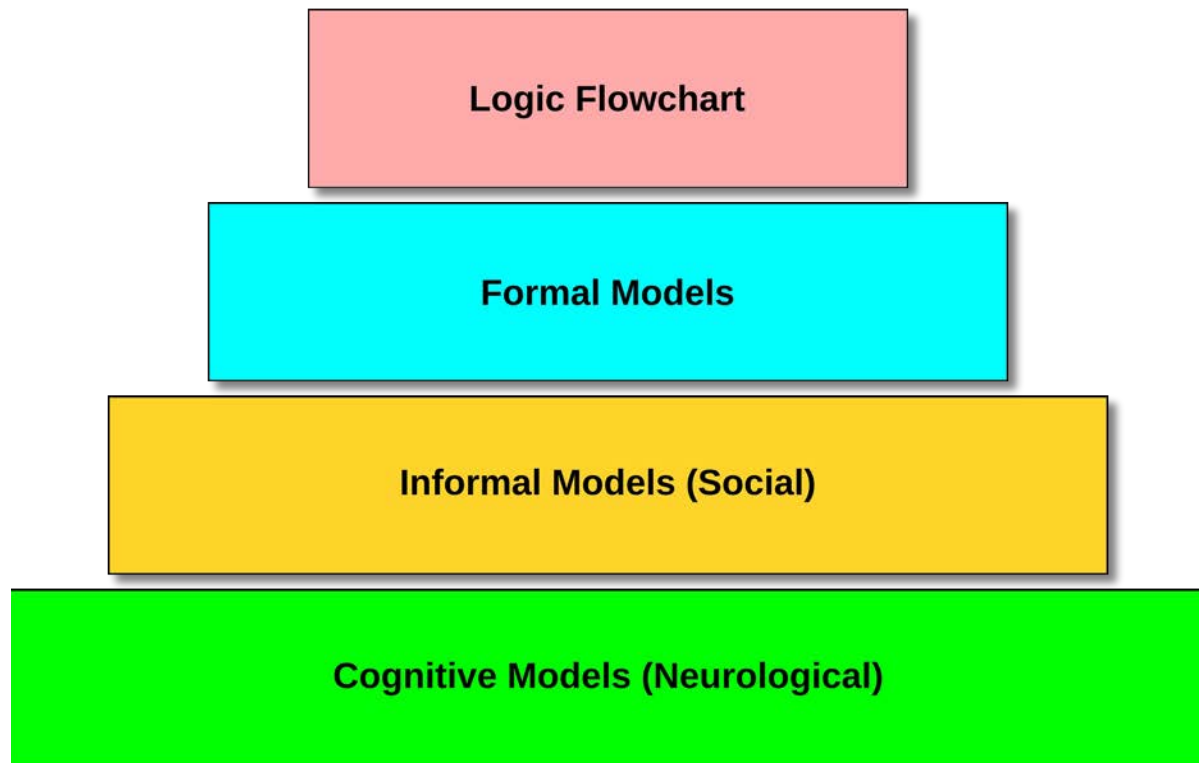
A well designed model communicates information efficiently and effectively; and those two properties are often more the result of good modeling as opposed to the fact we are using UML. Diagrams are not models, rather diagrams are used to describe models. In this module, we will examine the basic properties of models and what makes some models good and some bad, as well as how to work with a set of models in a unified way.

In this module we will look at:

1. The properties of effective models.
2. Understanding model perspectives through views.
3. Organizing models into collections using architectures.
4. How to organize, plan and execute model testing.

### ***10.1.1 The Modeling Hierarchy***

This module is about modeling, which means that the first thing we need to do is identify what models are. Modeling is a fundamental human activity – we all create models all the time to help us process information about the world around us, to be able to make decisions and react to our environment.



These sorts of everyday models are intuitive and what we call informal – which is to say there are no explicit rules that govern how we make or use them, just a more or less set of common “ways” we all follow to some degree, which is in itself is quite remarkable since it seems that no matter where or when we live, these mental models we create and use everyday are similar the models that everyone else uses.

There are two basic reasons why our informal models of the world resemble the models others around us have. The first reason is common biology – our brains are structured to process information in a standard way which we all share. The second reason is that we learn a set of informal modeling rules through the use of language – sort of a prototypical model – which is common across all cultures.

UML and various forms of logic representation are special types of formal model and therefore follow the best practices of formal models. Similarly, formal models are a special type of informal models, so understanding what makes good informal models lets us understand how to evaluate more effective formal models. Informal models are similar in every human culture because the best informal models present and encode information in ways that are natural for how the brain encodes, stores and interprets information.

## 10.2 Biological Bases of Models– Neuro–cognition.

Our brains are designed to be efficient in real time as opposed to being exhaustive and thorough processors of data. To be fast thinkers, we take cognitive shortcuts by creating models. We extract salient features from our environment and generate prototypes of what we think is out there and then let our brains fill in the details. This allows us to react effectively based on a minimum of information and take fast action in a possibly dangerous situation.

This sort of approach has a high degree survival value. If you were trying to survive on the Savannah 100,000 years ago, it would be useful to be able to identify a new animal that you had never met before as a potential predator before you became its dinner. Creating a mental model is an effective strategy since we do not have to analyze the whole situation in order to take action.

For example, our predator prototype might be “If it has sharp teeth, big claws, makes growling noises and runs at me, then it's a predator.” If a new animal is seen that has big teeth, long claws and runs at you, the model can be used to determine “Three out of four is close enough, I'm getting out of here.”

Consider trying to listen to a conversation at a party; if you concentrate on the conversation, your brain abstracts out all of the other conversations so that you don't notice them and can focus on the person talking to you.

### 10.2.1 Autism: When Mental Models Break

One of the more remarkable insights we have about how our biologically based mental models work are reports of what happens when they don't work effectively, such as in the case of Carly Fleischmann (<http://carlysvoice.com/home/aboutcarly/>). Carly is an autistic girl who lives in Toronto and was diagnosed as having the mental capability of a six year old. Or at least so everyone thought until she was eleven and sat at her dad's computer and typed “hurt teeth help.” This breakthrough moment revealed to her therapists and family that Carly was a normal intelligent girl who, as she put it, was trapped in a body that she could not control. Since then, Carly has become a leading figure in educating the public about living with autism through her blog and public appearances.

In Carly's case, her autism did not allow her to ignore irrelevant information. When Carly was asked in a 20-20 ABC News story why autistic people cover their ears, flap their hands, hum and rock; she replied (this is transcribed from the video interview)

*It's a way for us to drown out all sensory input that overloads us all at once. We create output to block out input. Doctors have said that we cannot process information That's not really true. Our brains are wired differently. We take in many sounds and conversations at once. I take over a thousand pictures of a person's face when I look at them. That's why we have a hard time looking at people. I've learned how to filter through some of the mess.*

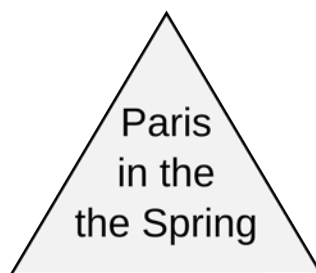


### ***10.2.2 Evidence from Selective Perception.***

Selective perception is the result of our brains filling in the gaps in our perception. We perceive just enough to figure out what we are observing and then very quickly fill in the gaps from what we already know. Helen Meisenhelder in *Selective perception: turning green into blue* describes it this way:

For most of us, perception is reality. Yet, our perceptions do not always mirror reality, they are heavily influenced by our expectations, and prior experiences. We observe stimulus with a preconceived notion of how it should appear and find it difficult replacing that notion with the facts. Instead of reality, we "selectively perceive what we expect and hope to see". This biased phenomenon is defined as selective perception.

Selective perception makes it next to impossible for us to proofread our own writing because we know what we wrote and that is what we see every time we proofread the text. Interesting fact: to avoid selective perception problems, professional proofreaders proofread text backwards because if they see a sentence, they will just perceive enough of the words to fill in the rest. Like in this old optical illusion.



The brain edits out, or rather never processes, the second "the" in order to satisfy the expectation that we are reading a semantically and grammatically correct sentence.

### ***10.2.3 The McGurk Effect.***

One of the most dramatic demonstrations of how the brain edits perception was discovered by Henry McGurk by accident and occurs when the brain perceives different visual and auditory input from a speaker. The brain rewrites the auditory input so that it becomes consistent with the visual input.

For example, if a listener is watching a video where the person is mouthing the word “vase” but the audio has been dubbed so that they are hearing “base”, the following happens.

1. When the listener has his eyes open, he hears the word “vase”
2. When the listener has her eyes closed, she hears the word “base.”

There are several excellent demonstrations of this on YouTube, including this one “Try The McGurk Effect! - Horizon: Is Seeing Believing? - BBC Two” at (<http://www.youtube.com/watch?v=G-IN8vWm3m0>).

Even when you know the effect, you can't stop it from happening because it is taking place at a neurological level. For example, the following are all experimental discoveries about the McGurk effect.

1. The effect works on perceivers with all language backgrounds.
2. The effect works on young infants.
3. The effect works when the visual and auditory components are from speakers of different genders.
4. The effect works with highly reduced face images.
5. The effect works when observers are unaware that they are looking at a face .
6. The effect works when observers touch—rather than look—at the face .

### ***10.2.4 Common Cross Species Neurological Architecture.***

As a final note on this fascinating side topic, ethologists (scientists specializing in animal behavior) have been finding very convincing evidence that it is not just humans who have the ability to create these mental models. Various studies seem to indicate this ability is shared by the great apes and dolphins but also by species of birds, mammals like dogs and the octopus. This suggests that mental models may be the result of common ways or organizing brain structures and processing.



## 10.3 Informal Models – Social Communications

When we communicate with each other we construct our messages in ways that are consistent with the way our brains process information. Messages that package information in ways that are consistent with our mental models have a sense of efficiency and effectiveness to them.

These effective communications methods become part of the language and culture of a community, and show remarkable consistency across different languages and cultures. The rules that we use in these informal models are the same ones that we use in formal models, which means if we ignore these rules creating a UML diagram, we wind up with a model that fails to be effective.

### ***10.3.1 Social Basis of Models: The Speech Contract***

The philosopher H.P. Grice noted in the late 1970's that human conversations were governed by a social contract of great strength. In fact, this speech contract is so powerful that a deliberate violation of the contract by one party in a communication can cause a violent reaction from the other. In every culture in the world, breaking the speech contract is considered either inadvertent, which is taken as a sign of mental illness or stupidity, or deliberate on the part of the speaker, which is always taken as an insult. Intentionally breaking the contract is always a sign of contempt for the other person.

Social contracts are tricky things to identify and describe. Generally, a society controls the behaviours of its members through a system of norms and values. A norm is a "way we do things," although it is often more associated with the opposite "what we just don't do." A value is a reason or rationalization for a set of norms.

We learn the norms of a society through the socialization process – our parents and teachers show us how to be good little polite boys and girls with the goal of becoming productive members of our society by following its norms. The problem with norms is that in many cases we can't say exactly what they are because they are never explicitly stated. You learn norms by being sanctioned or violating them and by emulating the behaviour of others who serve as role models.

For example, if you were asked to describe polite conversation, you would be hard pressed to define or describe it, but you generally would have no trouble knowing when the rules of polite conversation were being violated.

When these norms constrain or structure our interaction with others, they form a type of social contract. This social contract often exerts a compulsion on us to act a specific way but yet, because the norms underlying it are not overt, we are unaware of this influence.

For the purposes of this course, the part of Grice's analysis we are interested in are the rules of the speech contract which might be considered the rules by which we package information for efficient communication.

*Our talk exchanges do not normally consist of a succession of disconnected remarks, and would not be rational if they did. They are characteristically, to some degree at least, cooperative efforts; and each participant recognizes in them, to some extent, a common purpose or set of purposes, or at least a mutually accepted direction. This purpose or direction may be fixed from the start (e.g., by an initial proposal of a question for discussion), or it may evolve during the exchange; it may be fairly definite, or it may be so indefinite as to leave very considerable latitude to the participants (as in a casual conversation). But at each stage, SOME possible conversational moves would be excluded as conversationally unsuitable. We might then formulate a rough general principle which participants will be expected to observe, namely: Make your conversational contribution such as is required, at the stage at which it occurs, by the accepted purpose or direction of the talk exchange in which you are engaged. One might label this the COOPERATIVE PRINCIPLE.*

The cooperative principle is about making communication efficient, which is of course what good models should do – be efficient in how they work with information. Grice elaborates on what this notion of cooperation means by positing a number of maxims that we tend to follow in our conversation.

### **QUANTITY**

Relates to the quantity of information to be provided:

1. Make your contribution as informative as is required (for the current purposes of the exchange).
2. Do not make your contribution more informative than is required.

### **QUALITY**

Try to make your contribution one that is true'

1. Do not say what you believe to be false.
2. Do not say that for which you lack adequate evidence.

### **RELATION**

Be relevant.

### **MANNER**

Be perspicuous'

1. Avoid obscurity of expression.
2. Avoid ambiguity.
3. Be brief (avoid unnecessary prolixity).
4. Be orderly.



Any detailed discussion of this topic is really beyond the focus of this course, however it is useful to look at speech contract briefly to see that these maxims actually become important when we start to look at what makes “good” models.

### **Formal versus Informal Models**

This course is about formal models. So far the biological example and the speech contract have been examples of what we call informal models. The primary differences between these and formal models are:

1. Informal models are part of our cognition are used without us being aware of them. Formal models are defined independent of how we think and are constructed and interpreted based on a set of stated rules.
2. Informal models are general in nature and apply to many different kinds of data while formal models are defined to only apply to a narrow and well defined type of data or experience.
3. Informal models tend to be qualitative and give us estimates while formal models are usually quantitative and produce more precise results.
4. Informal models tend to have higher rates of error and are not as easily validated or verified as formal models.

However, the important point that needs to be made is that formal models are derived in some sense from how we do informal models – and this means that everyone has a “modeling intuition.” For this course, what we want you to do is to start to fine tune that intuition during the learning of the formal models so that you can start to develop your sense of how to create and analyze formal models at a high level of competence.

## 10.4 Formal Models

From this point on in the course, we will only be discussing formal models so any reference to “model” without a qualifier should be understood to mean a formal model.

There are a number of reasons to use formal models but the main reason is to simplify. Remember that when we looked at the biologically based metal models, one of the reasons we created those models was to avoid being overwhelmed by details and instead be able to immediately identify and deal with the most important facts. As the UML User's Guide says:

We build models so that we can better understand the system we are developing. We build models of complex systems because we cannot comprehend such a system in its entirety. A model is a simplification of reality.

For example, suppose we are hired by the city of New York to provide information to individuals and groups about the city, whether they are tourists, businesses or whoever else.



If I wanted to get from Central Park to Penn Station, the aerial photo is a lot harder to work with than the map on the right. The simplification that the model (map) provides removes the clutter so that I can focus on the essential features that are necessary to plan out my route.

Simplification is the primary motivation for using formal models – we need something we can work with when the raw data of reality is too overwhelming to be used effectively.

### ***10.4.1 Analysis versus Design Models***

Paradoxically enough, the first topic we are going to look at in this section is a pseudo-property of models, one that is often used to claim that there are two fundamentally different types of models:

1. ***Analysis Models***: These are claimed to be models of what already exists or in some terminologies, models of the existing system as is.
2. ***Design Models***: These are claimed to be models of what doesn't yet exist or in some terminologies, model of the system to be.

While this distinction is legitimate, being a design or analysis model is NOT a property of the model but rather describes how it is being used at a particular point in time.

In this course, I will avoid the terms analysis and design models and instead refer to using a model as a map or as a blueprint.

A model is a map when I use it to understand something that already exists, in the same way that I use a road map to figure out where various roads in a city lead to so that I can get where I want to go.

A model is a blueprint when I use it as a guide to create something that doesn't exist. A recipe for curried eggplant is a blueprint because it describes something that doesn't exist at that moment (to avoid a philosophical discussion we will just take that to mean that MY curried eggplant doesn't exist at this moment and I very much want it to, as opposed to curried eggplant in some generic or existential state).

To illustrate this, suppose that I want to build a GPS tracker. I search the Internet and find a circuit diagram that lays out exactly how to build a generic GPS tracker. A circuit diagram is a formal model but since the GPS unit doesn't exist yet, the diagram is a blueprint. After I build the GPS tracker the circuit diagram now becomes a map showing me what is in the existing GPS. Now suppose that a few months later, I decide to modify my GPS tracker by making it solar powered since I'm tired of buying batteries for it. Of course I have totally forgotten how I built the GPS so I consult the map of the device (the same old circuit diagram) to figure out how to make the modifications to my GPS.

A model may be used as a map or blueprint depending on why it is being used – being a map or a blueprint is not a property of the model itself.

## 10.4.2 Characteristics of Good Models

There have been a number of attempts in the IT community to define what makes a model good. Most of the attempts agree on some of the basic characteristics, although they state them in different ways, most of these lists are in many ways unintentional re-statements of the maxims of the speech contract. The characteristics of good models are the following:

### **Models are Accurate.**

This is the maxim of quality (say things that you know are true). In formal terms we can express this as follows:

1. When a model is being used as a map, every element in the model represents something that actually exists in the real world.
2. When a model is being used as a blueprint, every real world object that is critical or essential is depicted in the model – there is no model element that “should not be built”.

Consider this example:

### ***NATO bombing continues; U.S. blames bad maps for embassy strike***

*BELGRADE, Yugoslavia (CNN) -- NATO followed its apology for the bombing of the Chinese Embassy in Belgrade with more air strikes on Yugoslav targets overnight Saturday and Sunday morning .... Contributing to the problem, CNN was told, were the maps for the area. All available maps for the area showed the Chinese Embassy in its former location and not its current location, where it has been for about four years.*

There are a number of different ways to express this concept of accuracy. Grady Booch and the other developers of UML expressed it this way:

*Models have to be models of something – models are representations of some subject out in the real world*

Robert Binder expressed it this way:

*The best models are connected to reality.*

### **Models have a Frame of Reference**

This is an expression of maxim of relevance. It means that good models do not show elements or details that do not contribute to the purpose of the model but only contain relevant information. Models simplify reality by emphasizing the important features and deemphasizing (or abstracting out) the unimportant with respect to the purpose of the model.



However, it is the modeler who makes the decision about what is important. This means that all models are inherently subjective because they represent someone's decision about what should be depicted.

For example, in the four figures below, we see our real world image of New York and then several maps (models).



Each map above uses a different frame of reference. The top right shows us the subway lines and MTA stations, the bottom left map shows us the borough and other boundaries but does not show any streets. The bottom right shows us how the land in Manhattan is used, but not boundaries or subways.

Each of these map/models has been created with an intent to depict some view or aspect of New York according to a frame of reference, which in these cases are transportation, politics and land usage respectively. These frames of reference are obviously derived from the reason we created the model or, in other words, its purpose. If I want to find out where to catch the subway, the land use map is the wrong choice of model because it contains information which is irrelevant to the purpose of my using the model.

and missing information I need. The information it contains may be accurate, it's just not useful.

### Models are the Right Size

The idea of being the right size is a restatement of the maxim of quantity from the speech contract (make your contribution as informative as is required for the current purposes of the exchange but do not make your contribution more informative than is required.) We can restate this in the following two rules:

1. Don't tell me stuff I already know.
2. Don't leave out things I need to know but don't know.

This is not the same as a frame of reference which says that we leave out information that is not related to the purpose of the model. This property says that we leave out information that the person using the model already knows and do not leave out things that we expect they don't know but need to know.

This means that two models may be of the same subject and have the same frame of reference but differ in the amount of detail based on the intended audience. The amount of detail is often chosen by anticipating the level of knowledge that the users of the model already possess about the subject being modeled.

Recall our definition of a paradigm from a previous module. Up until the establishment of a paradigm, anyone writing about something in a field had to develop all their ideas from “first principles” or from scratch, since they had no idea what anyone reading their work actually knew so we tended to get massive works that took nothing for granted. However once a paradigm is established, an author can assume that every one using the paradigm has the same basic knowledge. That means new results can be communicated without having to restate what everyone knows – massive treatises become short research papers.

For example, there is a paradigm for Particle Physics and anyone reading a paper in the subject is generally lost unless they know the paradigm, since authors assume that those reading the paper have a certain level of knowledge about the subject. For example, here is the abstract of the paper Calculation of the Cherenkov light yield from low energetic secondary particles accompanying high-energy muons in ice and water with Geant4 simulations by Leif Rädeland and Christopher Wiebusch

*In this work we investigate and parameterize the amount and angular distribution of Cherenkov photons, which are generated by low-energy secondary particles (typically less than 500 MeV), which accompany a muon track in water or ice. These secondary particles originate from small energy loss processes. We investigate the contributions of the different energy loss processes as a function of the muon energy and the maximum transferred energy. For the calculation of the angular distribution we have developed a generic transformation method, which allows us to derive the angular distribution of Cherenkov photons for an arbitrary distribution of track directions and their velocities.*



Ummm yeah, ok.

## Models Have a Well Defined Notation, Structure and Technique

This is the characteristic that most clearly differentiates formal models from informal models. Formal models have the following:

1. Notation: A symbolic way of representing the model. This can be diagrams, equations or some other way of writing the model down.
2. Structure: A model often consists of a set of model elements and specific ways of arranging them so that some arrangements are legal and others are not.
3. Technique: There may be specific ways that have to be employed to construct the models, usually described in some model construction process.

There is usually a set of rules that defines what the legal model elements are and the legal structures that the model allows. This set of rules is generally called a meta-model since it is a model that describes how to make models of a particular type.

Techniques are often important because even though models simplify reality, the models themselves may be very complex and be difficult to construct. For example, the following is a model of the quantum harmonic oscillator for one particle in D (whatever that means):

This is an example of a model that has a mathematical notation and set of techniques although the structure is described by the underlying physics model.

$$i\hbar \frac{\partial}{\partial t} \psi(x, t) = -\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} \psi(x, t) + V(x) \psi(x, t)$$

$$V(x) = \frac{1}{2} m \omega^2 x^2$$

On the other hand, a circuit diagram is a graphical notation, as is a state diagram or a flowchart. In this course we will be distinguishing very clearly between the model itself and the notation used to describe the model. A misuse of the model's notation means that we run the risk of an incorrect interpretation of the actual model.

## Models are Incomplete

Since every model abstracts out information, especially with respect to its focus of frame of reference, the model cannot contain all of the information about the subject being modeled. This means that single model from a single perspective cannot be sufficient for most of our purposes.

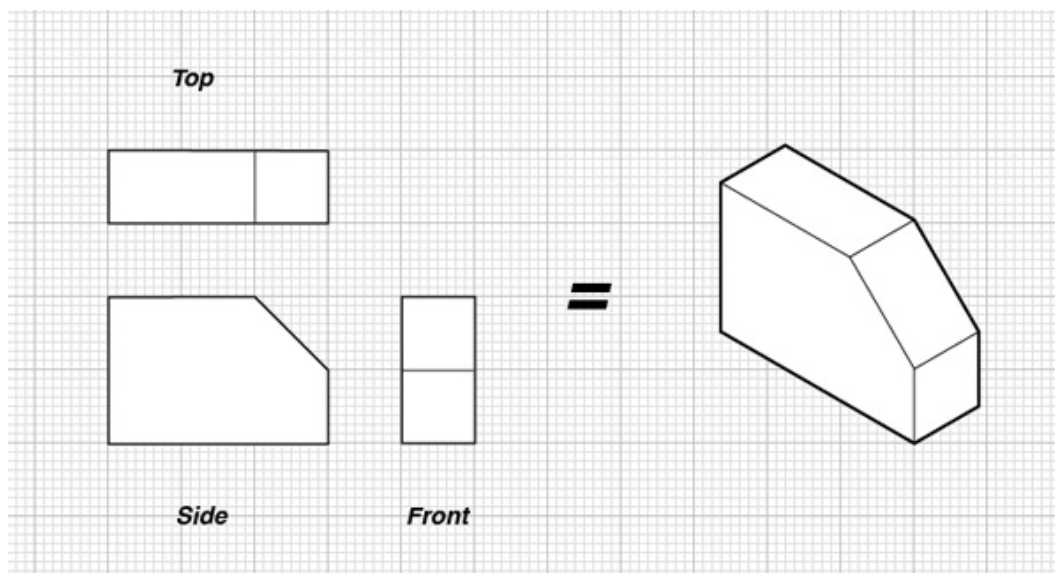
In fact, modeling is most effective when we describe the subject from multiple points of view – the aggregation of the models then provide a more complete description of the subject. An analogous situation might be an investigator trying to figure out what happened during a hold up. If she only asks one person what they say, she will get only that person's perspective and what that person noticed. On the other hand, the more witnesses she interviews, the more different details will emerge providing a more complete picture of the subject.

This leads to a very important principle of modeling systems:

***No single model is adequate to describe a system. Systems are best described a minimal complete collection of orthogonal models.***

What does this mean? First, we need a collection of models since each model leaves something out or omits some sort of detail. An complete set of models is a collection of models that, while each individual model leaves details out, the collection does not. Obviously for usefulness, we don't want this collection to become so large that it is unworkable, so we look for the smallest collections of models that are complete. For now, we will consider orthogonal to mean that the models are all done from different perspectives, the overlap between models in what they show is minimal.

For example, a solid object can be described by three orthogonal views in engineering: the top, side and front views.



## 10.5 Model Views and Architectures

One of the more confusing terms used in IT is that of systems architecture or IT architecture or software architecture – and the confusion is well motivated since there are many competing techniques and methods in use all calling themselves architecture, and are all quite different from each other. To compound the confusion is the existence of “business architecture” techniques which resemble many of the software architecture methods.

While we will not be getting into the whole subject of architectures, it does have a bearing on modeling since our choice of architecture influences our choice of models. Remember that we can best describe something in terms of a small set of orthogonal models, each of which (intuitively) is constructed from a specific view. We use the term orthogonal here to describe a set of models that each describe the same subject from a different point of view.

First we need to define what an architecture is, which is a bit problematic since there is no generally agreed upon definition. However, we can get a flavor of what an architecture is by looking at a couple of industry definitions

1. The fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution. (IEEE)
2. A representation of a system in which there is a mapping of functionality onto hardware and software components, a mapping of the software architecture onto the hardware architecture, and human interaction with these components. (SEI)
3. An architecture is the most important, pervasive, top-level, strategic inventions, decisions, and their associated rationales about the overall structure (i.e., essential elements and their relationships) and associated characteristics and behavior. (OPEN Process Framework).

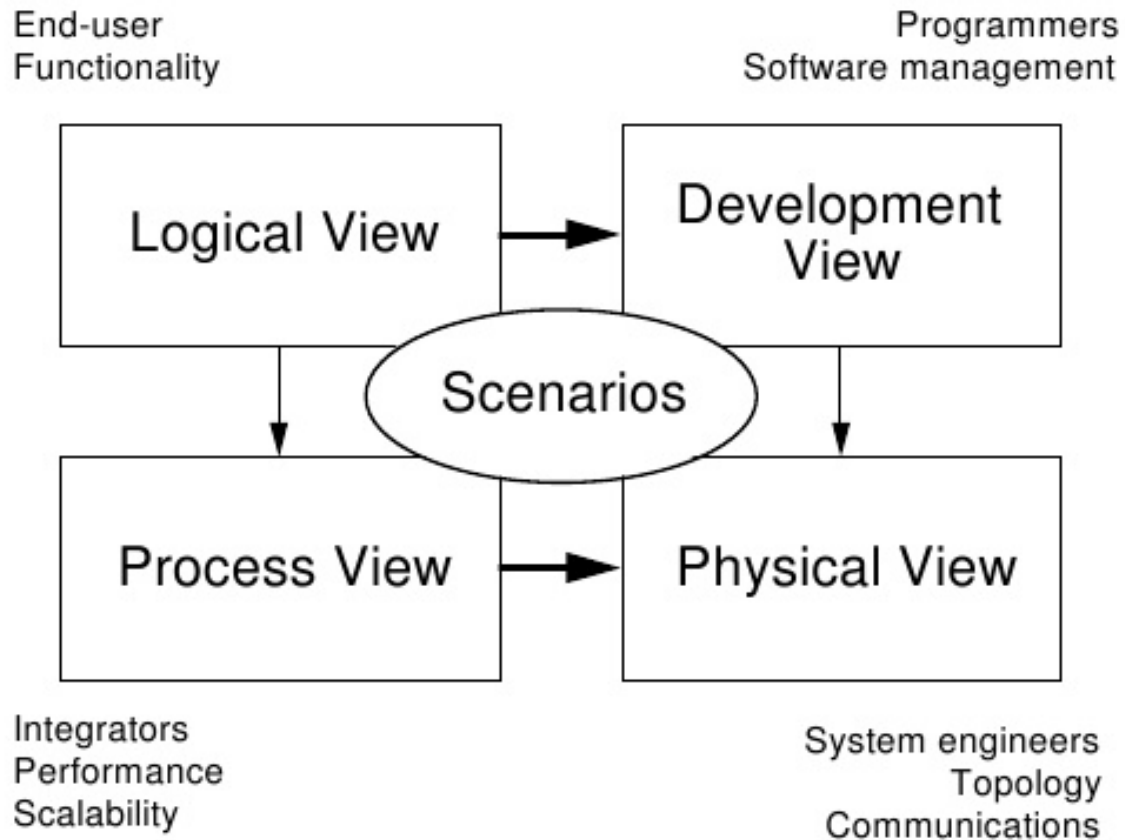
A core goal of any architectural system is to provide an overall complete description of the system and an identification of the various views that are used in the construction of models. One of the reasons for defining an architecture is to ensure that no aspect of the system has been overlooked in the modeling because a specific view was not utilized.

If an architecture can specify a set of views for modeling that allows us to produce a set of models that fully describes the system, then we have a complete set of models.

### ***10.5.1 The 4+1 Architecture***

The 4+1 architecture is an example of defining a set of views for modeling. This architecture was originally developed to be used in the Rational Unified Process (RUP), but has achieved wide acceptance outside RUP as well. As we will see later on, some of

the UML diagrams, like the component and deployment diagram, are included specifically in the UML standard to enable specific views to be included in the modeling process.



The views of the architecture are described in the RUP documentation in the following way:

### The Logical View

1. Addresses the functional requirements of the system.
2. What the system should do for its users.
3. It is an abstraction of the design model and identifies major design classes, packages and subsystems.

## The Implementation View

1. Describes the organization of static software modules, source code, data files, scripts, components and executables for example and describes them in the development environment.
2. Identifies the issues of configuration management and development issues like change management.

## The Process View

1. Addresses the concurrent aspects of the system at run time – tasks, threads and interactions.
2. Concurrency, system start up and shutdown, fault tolerance, response time, throughput and so forth.

## The Physical View

1. Deployment is mapping software to processor nodes showing how the executables are mapped onto the underlying hardware topology.

## The Scenarios

1. Contains key requirements that initially drive the discovery of architecture and later used to validate the architecture.

The scenarios can be thought of way of walking through the system as it performs a specific task and identifying the impact that this task has on the various models and how this activity is seen through the various views.

For example, suppose that we have a scenario where a use is logging on to a banking website. The logical view will describe the experience from the user point of view but as we walk through the scenario we can also look at how the other views see that scenario.

From the logical view, the user sees a log in screen, enters their credentials and then is taken to their home screen with a list of accounts and menus of options.

However from a physical view we have to ask where are the user credentials stored? How are they accessed to execute the log in? Where is the client's account information located and how do we connect to it?

From a process view we may find ourselves asking how to support maybe ten thousand simultaneous users all trying to log in at the same time. What sort of latency is allowable? What sort of security is available on the user passwords? How do we keep all the data from each user isolated from other users?

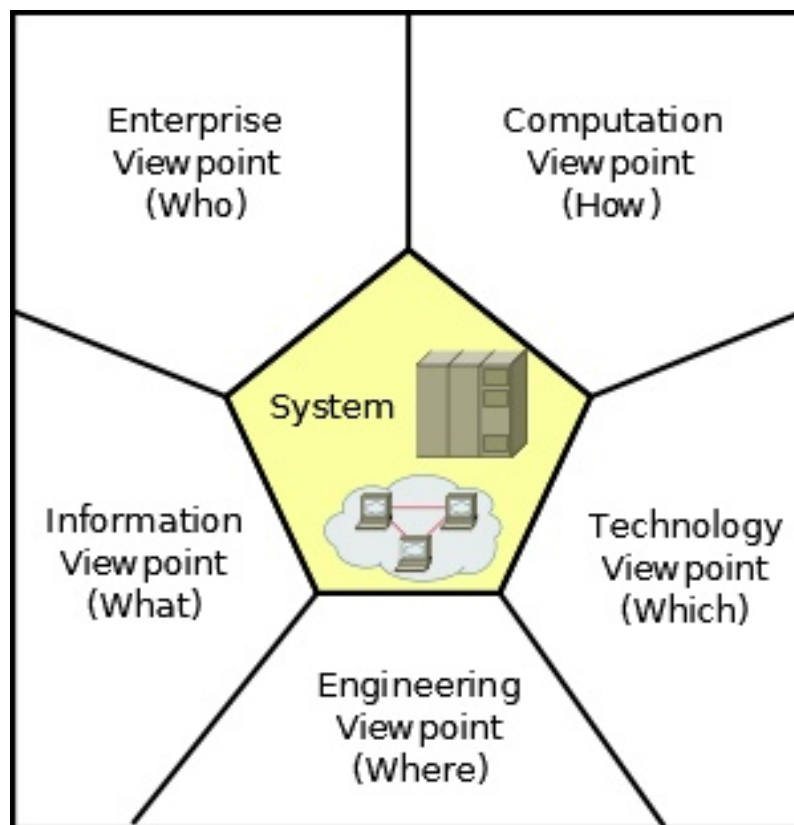
From an development view we might ask where are the encryption modules located? Where is the login logic located and how do I ensure that the scenario is accessing the right versions?

### 10.5.2 RM-ODP

To give another example of an architecture, consider the RM-ODP which is described as :

*The Reference Model of Open Distributed Processing (RM-ODP, ITU-T Rec. X.901-X.904 | ISO/IEC 10746) is a joint effort by ISO/IEC and ITU-T, which provides a coordinating framework for the standardization of open distributed processing (ODP) which supports distribution, inter-working, platform and technology independence, and portability, together with an enterprise architecture framework for the specification of ODP systems<sup>1</sup>.*

This architecture was developed to respond to the problem of working with modern systems where the specifications for the system are so complex that it exceeds the ability of any one person to understand all aspects of the system.



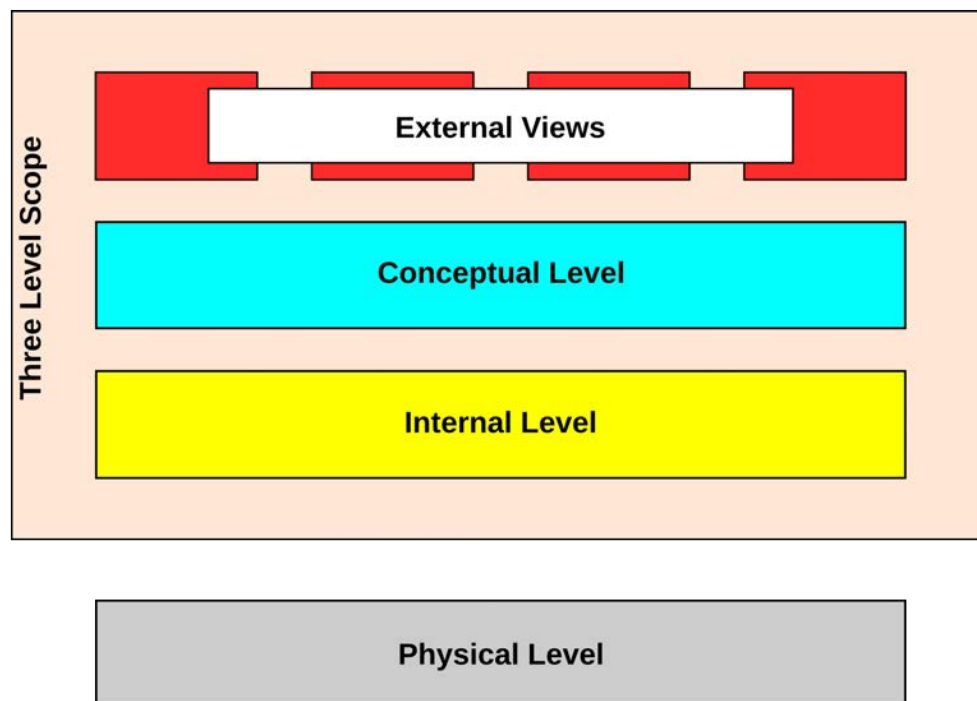
The RM-ODP defines five viewpoints, each of which represents the concerns of a group of stakeholders who have a common interests or concerns about the system. Each viewpoint has its own modeling conventions or techniques (called a viewpoint language) that is optimal for that viewpoint's audience.

*The five viewpoints defined by RM-ODP have been chosen to be both simple and complete, covering all the domains of architectural design. These five viewpoints are:*

1. *The enterprise viewpoint, which is concerned with the purpose, scope and policies governing the activities of the specified system within the organization of which it is a part;*
2. *The information viewpoint, which is concerned with the kinds of information handled by the system and constraints on the use and interpretation of that information;*
3. *The computational viewpoint, which is concerned with the functional decomposition of the system into a set of objects that interact at interfaces - enabling system distribution;*
4. *The engineering viewpoint, which is concerned with the infrastructure required to support system distribution;*
5. *The technology viewpoint, which is concerned with the choice of technology to support system distribution.*

### **10.5.3 The ANSI-SPARC Three Level Architecture**

The ANSI-SPARC Three Level Architecture was proposed in 1975 in an early effort to create a standard system database architecture. Although never a formal standard, it has been the foundation for design of many database products even though it could not be fully implemented until 1998 when the technology finally became available to enable database vendors to provide updateable views.



## The External Level

This is the level closest to the users and is concerned with how the data is viewed by the individual users. This implies that each user can have their own unique “model” or how they think the data is organized, which is often referred to as an “external schema”.

This is what is often called the “user logical” level. This is the view that is closest to the users and is concerned with how they conceptualize the data. As we will see in a later module, this is the level at which the business works with the data, usually in terms of objects or “things” that we are storing data about.

## The Conceptual Level

The conceptual level can also be thought of as the community view or model of the data. This level contains the entire information content of the database, but still at the level of abstraction that is above any implementation details. The conceptual level is defined by a “conceptual schema” but, unlike the external schema, there is only one conceptual schema. The view of the data does not correspond to any one stakeholder's view, instead the stakeholders' views are generated from this underlying logical model.

A main difference between the external level and this level is that we discover information at the external level while we define data at the logical level.

## The Internal Level

The internal level is where we are concerned with the implementation of the data. There maybe more than one “internal schema” which, in addition to describing how the data is stored, contains information about indexes and the physical ordering of the data. Depending on the data requirements, a single logical model may be segmented with each part implemented by a different internal schema.

This is still a level above the actual physical layer because we assume that we have available in infinite linear storage space. However, there may be many different possible internal models corresponding to the way we choose to implement our database of which the relational model is only one alternative.

## The Physical Layer

This is not part of the ANSI/SPARC architecture. The physical layer is concerned with a particular implementation of the internal schema on a particular system architecture. This is usually handled by the commercial database products.

Interestingly enough, this varies closely mirrors Peter Chen's descriptions in his original paper on entity relationship modeling.

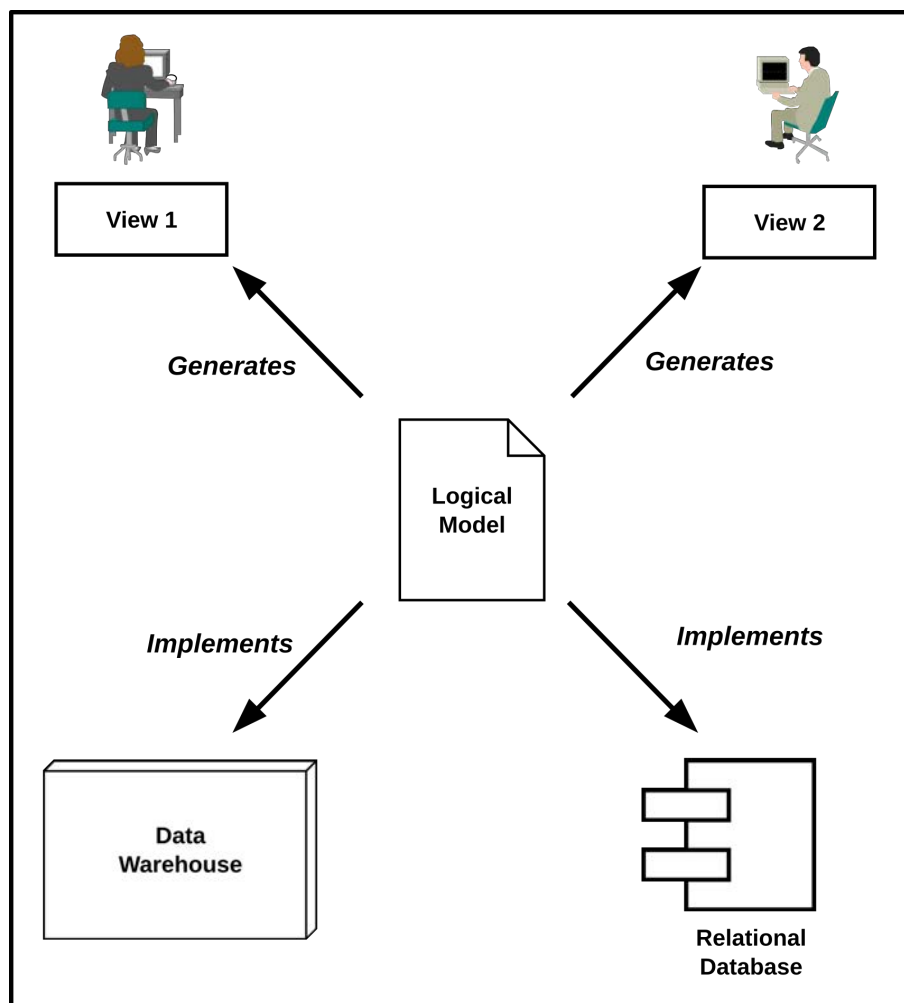
*In the study of a data model, we should identify the levels of logical views of data with which the model is concerned, We can identify four levels of views of data:*



1. Information concerning entities and relationships which exist in our minds.
2. Information structure – organization of information in which entities and relationships are represented by data.
3. Access-path-independent data structure – the data structures which are not involved with search schemes, indexing schemes, etc.
4. Access-path-dependent data structure.

The diagram below illustrates the three level architecture in practice. By exploring the views that the users have of the data – specifically their individual data models – we are able to define a logical data model that allows us to generate the specific external views. The logical model is no ones view of the data so we are free to design this model to whatever criteria we deem important.

Since the logical model is not implementation specific, it can be used to generate relational models or dimensional models for data warehouses or any other kind of implementation model.



## 10.6 Testing Models

There are several ways or modes that application models can be used in the testing process. In the following, a representation is a model of the component to be tested and an implementation is an executable or running build of that component. Taken from Robert Binder:

1. **Model Validation:** This is an attempt to establish that the modeled representation of a component is sufficient with respect to its behavioural requirements. In other words, is the model descriptive enough?
2. **Model Verification:** Representations can often be checked for correctness without executing an implementation of the representation. Verification techniques can be informal, like code inspections or reviews, or they can be more formal, like using mathematical proofs.
3. **Consistency Checking:** Representations are usually designed with respect to a technique or meta-model. We can verify that a representation is consistent with the well-formedness constraints of the meta-model. (Think of validating an XML document against a schema).
4. **Responsibility Testing:** Responsibility based testing determines whether or not the observable behaviour conforms to a test model derived from a representation of the component.
5. **Implementation Testing:** Implementation based testing determines whether or not the observable behaviour conforms to a test model derived from an implementation of the component.
6. **Product Validation:** This evaluates conformance of the observed behaviour to the stated requirements.

### 10.6.1 Test versus Application Models

Notice that in the six testing modes we listed above, we are assuming that the model we are dealing with is that of the application being tested. In fact, we also mention the term test model as well. To avoid confusion we should clarify the distinction between the two classes of models we use.

**Application Models:** These are the models of the system or application being tested. These models are the ones developed by the system designers and developers. These models are application specific.

**Test Models:** These are models we use to design and execute the tests. We will look at several test models in this course. These models guide us in choosing test strategies and apply to a variety of applications.



## Test Ready Application Models

Not all application models are what we call test-ready. A test-ready model contains enough information to automatically generate test cases. A model that is not test-ready can still be used to generate test cases, but the process must usually be done manually, because the tester usually has to make assumptions and subjective assessments to fill in the missing gaps.

Binder identifies six features that a model must have to be testable or test-ready:

1. It must be a complete and accurate reflection of the kind of implementations to be tested – the model must represent all features to be exercised.
2. It abstracts (i.e. does not include) detail that would make the cost of testing prohibitive.
3. It preserves detail that is essential for revealing faults and demonstrating conformance.
4. It represents all events of the state model so that we can generate these events – typically as messages sent to the unit under test.
5. It represents all actions of the state model so that we can determine whether a required action has been produced.
6. It represents state so that we have an executable means to determine what state has (or has not) been achieved.

The claim is that these are features which an experienced tester adds to a not test-ready model as part of his or her own internal preparations for testing.

## Cartoon Models

Binder refers to OOAD models which are not test ready as cartoons

*Most OOAD methodologies provide a loose graphical syntax and symbol set. This is accompanied by minimal guidance for impressionistic rendering of behavior and structure that happen to come to the designer's attention. These are cartoons: they do not demand complete information, consistent usage, or useful integration among all facets of a model. Cartoons are useful for sketching, refining, and documenting solutions, but they are not test-ready: they lack the content and consistency necessary to produce executable test cases. {Bind00:116}*

Cartoon models are not test ready because they generally tend to be incomplete, inconsistent and ambiguous. However, they can still be used, as we have mentioned, by a tester to manually generate test cases.

## 10.7 Logical Model Testing

A fundamental activity that occurs all the way through the development process is the modeling and analysis of logic. In previous modules we have used the term "process" to apply to software development processes, but in this module and the ones that follow we will be using the term "process" in a different sense – to represent the general idea of a logical process. When we use the term "process" without qualification, we will mean a logical process, otherwise we will qualify it in some way as in "development process."

We are also going to be looking at state modeling since it is intimately associated with process modeling since every logical process can be written as a set of states and transitions, and vice versa.

### Logic and Processes

Even though we don't necessarily consider logic, such as in business rules, and processes, as in workflows, to be the same, when we look at them from a modeling perspective, we can see that they are really just two different views of the same underlying type of subject.

1. Logic and business rules are often implemented as an ordered sequence of decisions, which we could consider a process for executing the logic.
2. Processes have different ways of executing depending on logical decisions made at specific points.

No matter if we start from the point of view of modeling logic, or the point of view of modeling a process, we seem to wind up coming back to models that resemble each in form, content and style. Depending on our modeling objectives, we can emphasize one of these views over the other. For example, in structured analysis and design, a flowchart is used to emphasize the logic and business rules while a data flow diagram emphasizes the movement of data through a process. A work flow in a structured model tends to be optimally modeled when both these perspectives are used.

### ***10.7.1 The Model Representation***

We often use either an activity diagram or flowchart to model process logic throughout the development cycle.

1. During the requirements phase the diagram is used to model business processes and logic.
2. During the requirements phase the diagram is used to model user interactions, scenarios and use cases.



3. During the analysis phase the diagram is used to model the system processes and algorithms that implement the system functionality, and to help map functionality onto system objects.
4. During the design phase the diagram is used to model the program implementation logic used by the developer.

## What is a Process?

For the purposes of this course, we will use the following operational definition of a process:

*A process is made up of a series of actions that can be executed in a number of possible sequences. This series of actions must have the following properties to be considered a process:*

1. **Well defined values:** All variable data and information used in the process is defined as inputs to the process. Any values used in the process that are not provided as inputs are hard-coded and invariant and do not change between execution instances.
2. **Multiple alternatives:** At specific points in the sequence, conditions may be specified that allow for alternative paths through the logic. There is no requirement of all of the specified actions to be performed during the execution of the process.
3. **Invariance:** The same inputs always produce the same outputs.
4. **Deterministic:** The same inputs always follow the same path through the sequence of actions.
5. **Unambiguous:** Every decision point unambiguously specifies an alternative path for every possible input.
6. **Complete:** A path through the sequence is defined for all possible values of all the inputs to the process.
7. **Decomposable:** Each action that makes up the sequence can either be completely described atomically or can be described itself as a process.
8. **Concurrency:** The sequence of actions can split into sub-sequences so that each sub-sequence can be followed independently of and concurrently with the others provided each sub-sequence itself is also a process.

While seeming complex, this definition just formalizes what you already know about processes intuitively. There are a lot of things that look like processes but fail to be so under this definition. For example, business procedures that require a subjective decision are not invariant since different people following the same procedure may come to different outcomes depending what each decides.

Similarly, we often see ambiguity in procedures ("Don't sell fireworks to people who aren't mature enough") which often leads to subjective interpretations. However the most common reason why what appear to be processes but are not is because of in-

completeness. ("We don't know what to do in this case, our policy doesn't cover it.")

The invariant condition just means that an activity is performed in a consistent manner. It should not be done differently on Thursday than it was on Wednesday unless it has different alternatives specified for Wednesday and Thursday.

The deterministic and unambiguous conditions means that the sequence of the activities is always clear, we never get to a point in the process where we do not know what activity to do next. The decomposability condition says that we should never reach an activity and not know how to do that activity. The last condition could be considered more optional than the others – it just means that we can subdivide our process into independent sub-processes that can be all done at the same time.

### ***Descriptive Processes***

A process is descriptive when it describes how something is done. A descriptive process does not limit what must be done, but is more like a guideline.

Normally the process being described is documented in some manner so that when the process changes, the related documentation must be changed so that it continues to describe the process.

Example: A recipe is a descriptive process. It describes how to cook something, like Chicken Tandoori, but the cook can make changes in the recipe as they see fit. However, if changes are made in the process of cooking Chicken Tandoori, then those changes imply the written recipe (documentation) needs to be updated.

The main problem that many people run into when working with descriptive processes is confusing the documentation with the process. Changes in the process are resisted because the documentation is seen as being sacrosanct. As we will see in the next slide, the documentation is seen as proscribing the process.

To summarize descriptive processes think "Follow the instructions, more or less, if you want to get the job done."

We generally encounter two kinds of descriptive processes.

1. **Workflows:** These are purely descriptive and describe how activities are sequenced and organized. The emphasis is on the sequence of steps and the logic of the alternatives
2. **Algorithms:** An algorithm is also a sequence of steps but the emphasis is on describing how each activity is performed. The emphasis is on the content of the steps rather than on the sequence of steps.

This is not an either/or distinction, a process may be both a workflow and an algorithm to varying degrees. The difference is what we are emphasizing in the process.



### ***Proscriptive Processes***

A process is proscriptive when it describes how something must be done. A proscriptive process sets limits on what can be done, and can be thought of as the rules that define and delineate the actions that can and must be taken.

Normally the process being proscribed is regulated in some formal manner so that to change the process, the related regulations must be changed so that process can be allowed to change.

Example: If you are applying for a loan at a bank and do not follow their process, you will not be allowed to get a loan. However, even bank employees must follow the process or there will be sanctions applied by the bank, or perhaps even criminal charges.

Proscriptive procedures define what must be done and are usually intended to protect the interests of those involved in the process.

In a nutshell, proscriptive processes are rule oriented. They are there to describe how to follow the rules or policies that are in place, usually to ensure that something is done correctly or to protect the interests of the stakeholders.

For example, a significant percentage of the laws that make up our legal system deal with procedural law, which defines how something must be done to be legally valid. For example, how to arrest someone and advise them of their rights, how to execute a search warrant, etc. Procedural law ensures that the rights and interests of the various stakeholders who could be affected are preserved and also provides a way of deciding if something was done correctly.

To summarize proscriptive processes think "Doing it by the book"

We will also distinguish between two kinds of proscriptive processes.

1. ***Procedures***: A procedure is a process that has been mandated by an organization as "How we do this process here." A procedure can be thought of as a workflow that must be followed to ensure that all of the organization's policies are relevant to a process are applied. A policy can be thought of as a rule.
2. ***Protocol***: A protocol is a procedure that transcends organizational boundaries. Procedures generally are set by an organization at some level; eg. corporation, division, state or federal government while a protocol is a set of trans-organizational procedures that deal with a specific subject area.

Procedure example: When hiring musicians for an orchestra, the policy of the orchestra is that the only basis for hiring must be the quality of playing and not gender or age or any other factor.

To apply this policy, the procedure for auditioning a musician is to have them play from behind a screen with no information about them supplied to the adjudicators. Failure to follow these procedure, including even being able to hear the clicking of high heels, is a violation and the audition is declared void.

Protocol example: The field of biology has established "aseptic" protocols which are used in medicine, veterinary science, dentistry and other biological related fields.

The field of aviation has established pilot protocols such as the preflight inspection.

Failure to follow protocols usually result in sanctions but more likely will could result in some kind of systems failure or disaster.

Protocols differ from procedures in that they codify algorithmic information related to a subject area as opposed to a procedure that enforces organizational policies.

As we look around, we can see that we follow many procedures in the course of our daily lives, from following the traffic signs and regulations to how buy lunch to how we do our banking.

Procedures are not bad things in and of themselves, and we tend to rely on people following procedures to ensure that we all can all live together safely. As a result, we sanction those that don't follow procedures (In the US, we sanction those that do not follow the procedure of driving on the right hand side, while in the UK, driving procedure demands we follow the policy of driving on the right hand side).

The primary difference between a procedure and a protocol is one of scope. Protocols usually mandate certain practices which a community or profession has identified as a "best practice" or a "necessary practice." For example, the protocols on the treatment of epidemics, or how an archaeologist should document a new discover or how a crime scene investigator should collect evidence.

An organization may have to establish procedures to ensure that protocols are followed. The general rule of thumb is that no one enforces protocols, but if you don't follow them and something bad happens, you are held responsible whereas if you had followed the protocols, you would not be considered responsible, or at least not as responsible.

Generally procedures are arbitrary, like driving on the right as opposed to the left, while protocols are based in some best practice or body of evidence, such as having your doctor wash his hands before he operates.

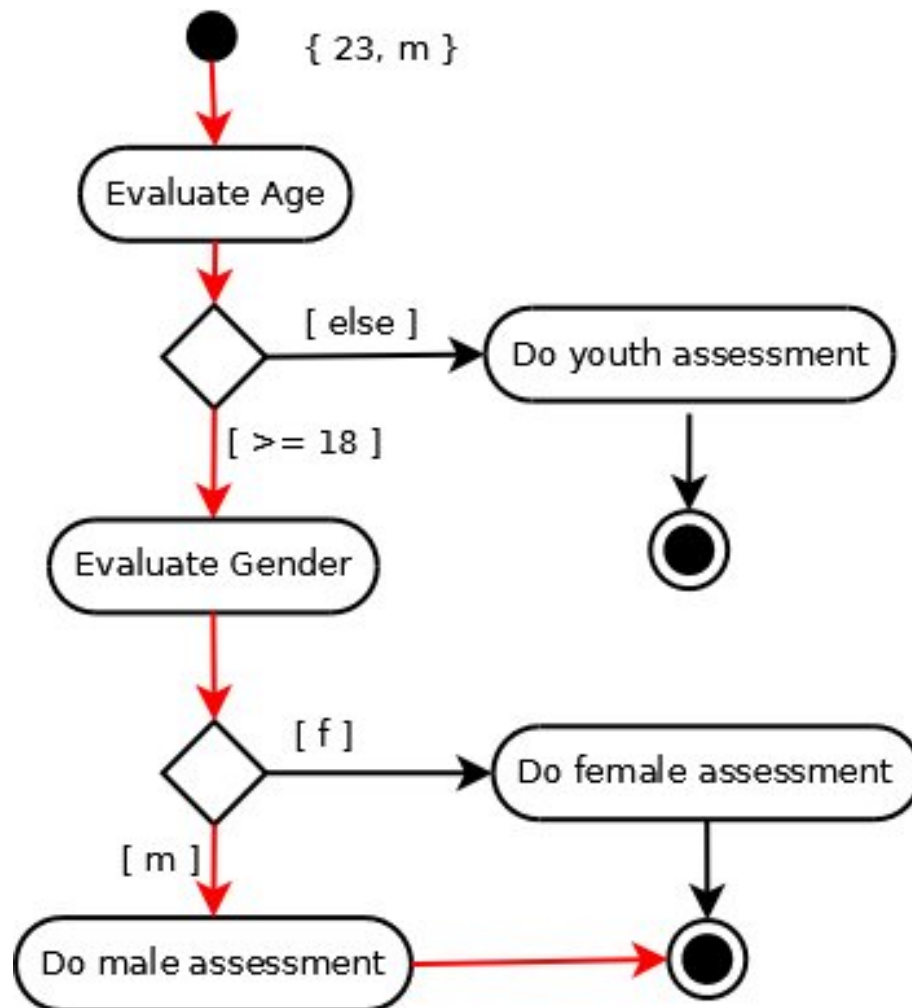
### ***10.7.2 Verification and Validation***

Recall in a previous model we introduced the IEEE best practices in terms of correctness, completeness, etc. This sort of analysis is especially important when we are dealing with loic diagrams since the possibility of errors in either the subject of our model or model is increased (remember the type I through type V errors)



One of the ideas that we will be looking at in this part is the idea of exercising a logic diagram. To exercise a diagram is to provide a set of test cases that causes a set of transitions in the diagram to fire

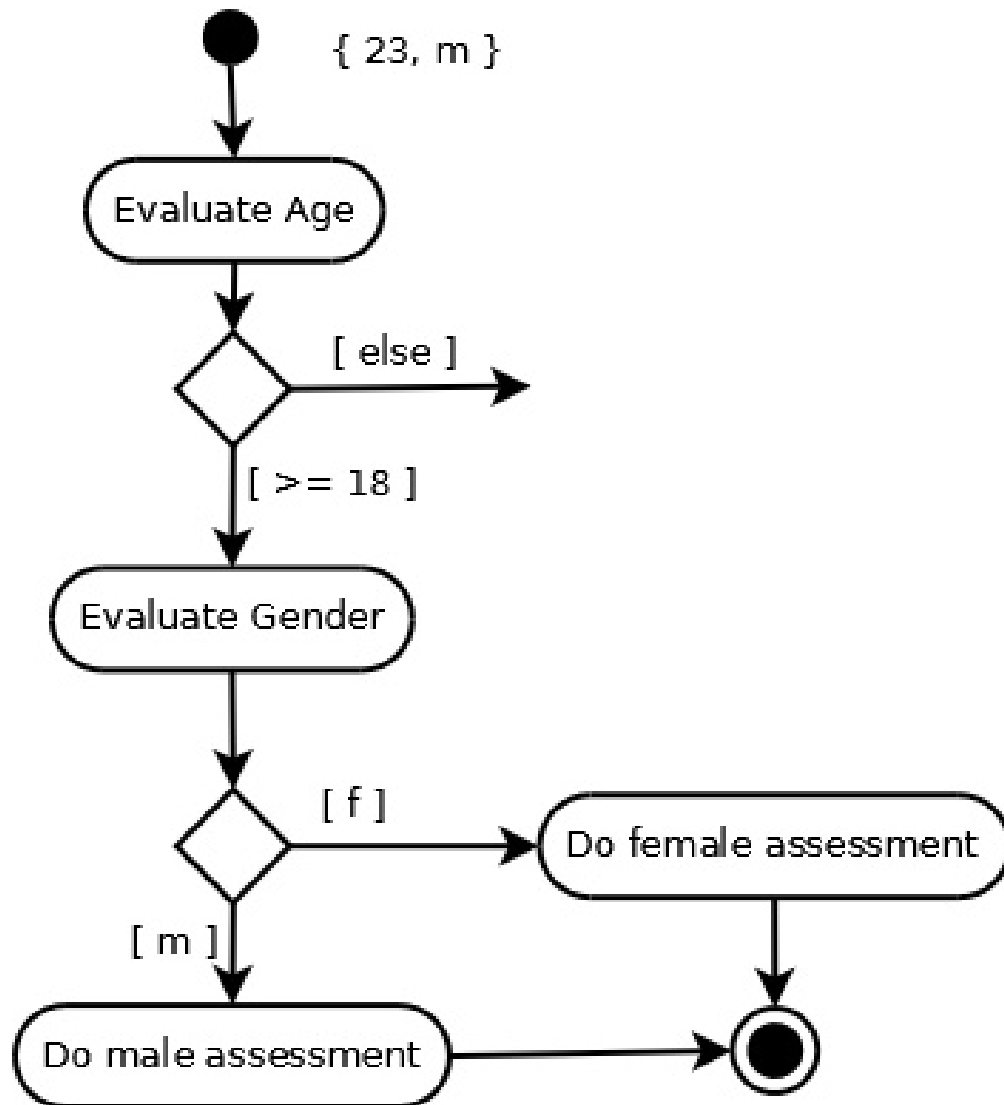
For example, in the diagram below we can see the transitions exercised by the test case {m,23}



A diagram is completely exercised (100% coverage) we have a set of test cases that exercise all of the transitions ({m, 23 } {f, 23 } {m 17

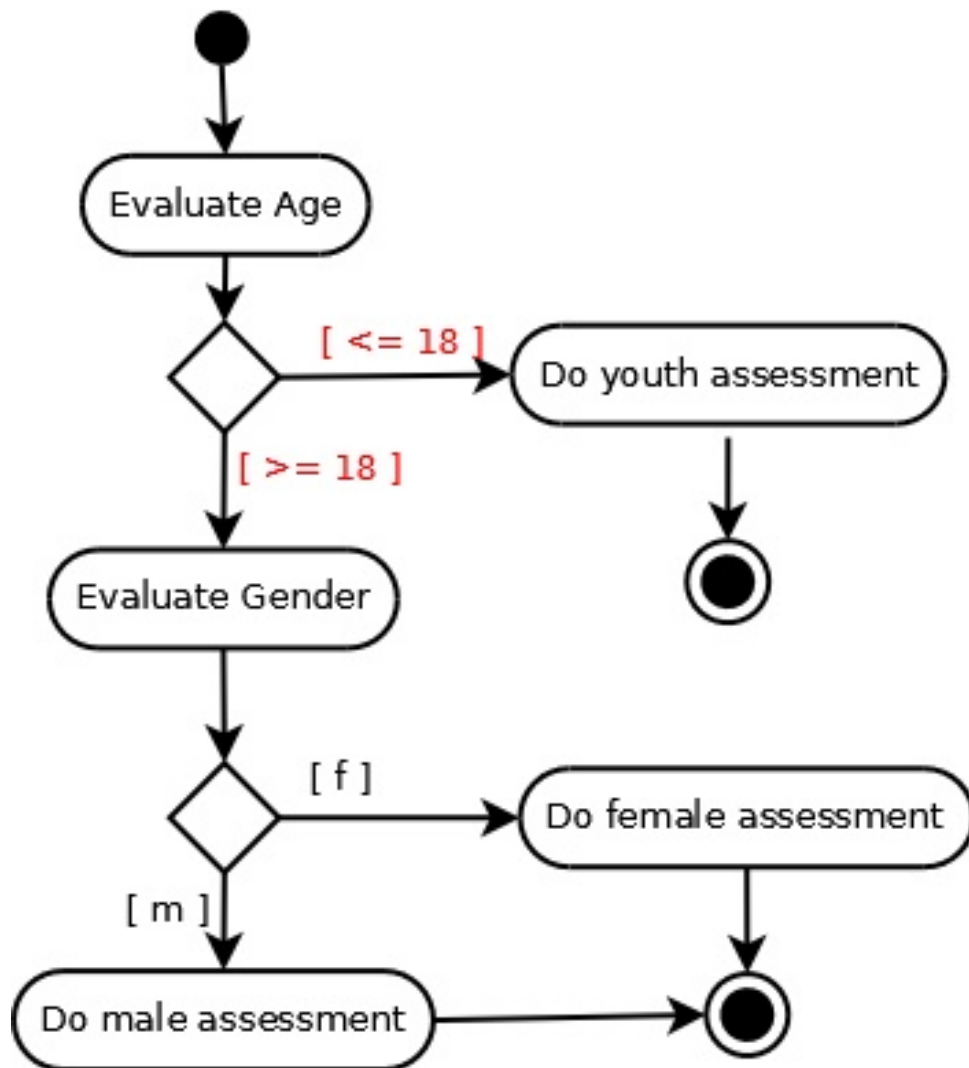
### Activity Diagram Incompleteness

A diagram is incomplete when one or more branches are missing.



## Inconsistent and Ambiguous Diagrams

Usually this is due to poorly formed decision conditions.

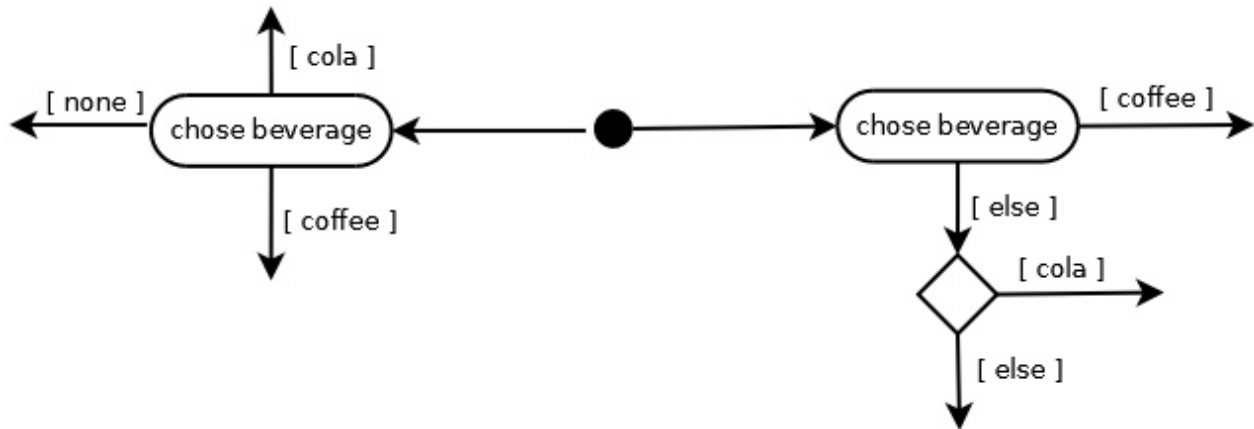


The above diagram shows the ambiguity that comes from poor boundary conditions for the guard. Notice we could have produced the same problem with the conditions `[ <18 ]` and `[ >18 ]`.

In these situations, there is often a specific test case that can be chosen (usually by following the functional testing technique of boundary value analysis) that show us the flaw in the diagram. For both the cases just mentioned, a test case that would show the error is `{m,18}` or `{f,10}`

## Incorrect Diagrams

Incorrect diagrams are those that do not describe either the existing process accurately or define a process where some aspect of the process cannot produce that required results.

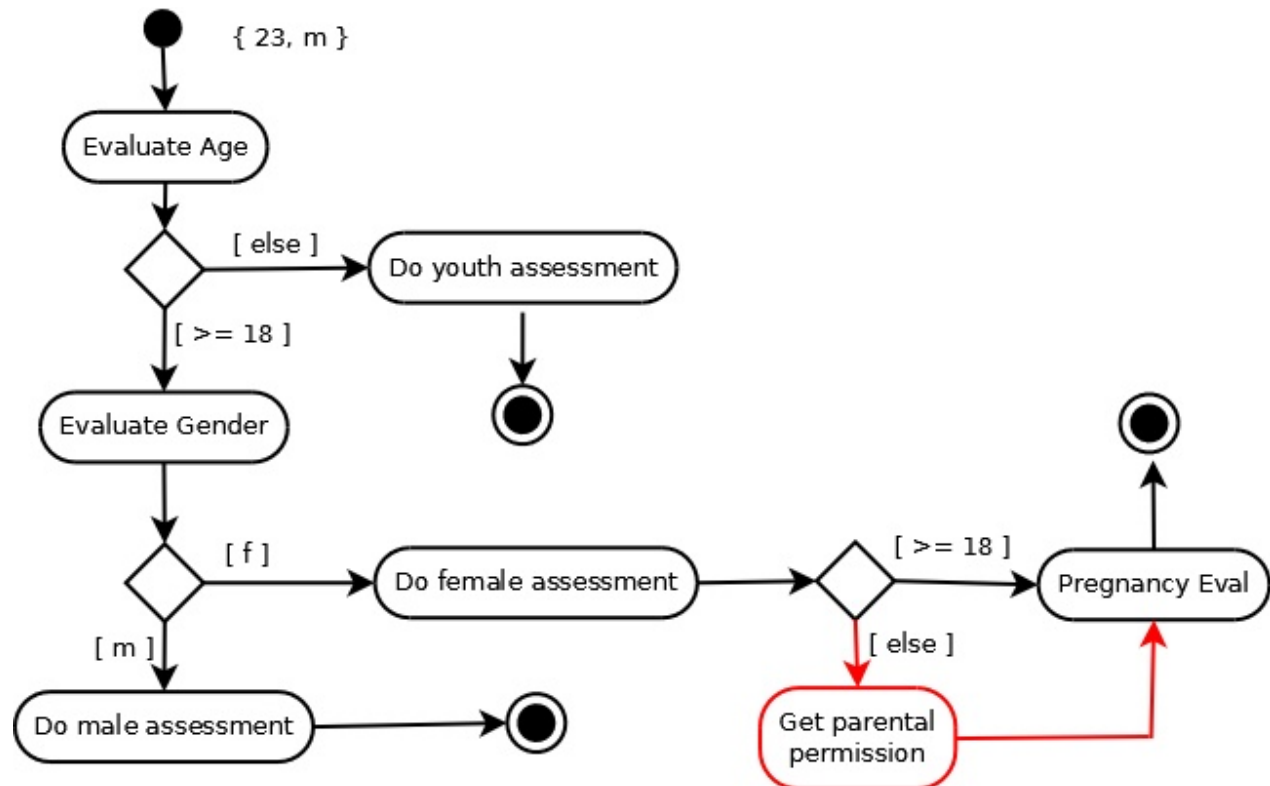


The branch on the right is familiar to us and codes the logic (look for beverage, if you find coffee, do that, otherwise if you find cola then do that, or else..) What you will see a lot in poorly executed UML is something like the left branch.

The left branch is incorrect because it does not capture the sequence that coffee is chosen before choosing cola. Instead when both coffee and cola are found, no decision is possible because of the missing logic. Note that cannot be fixed by correcting a guard condition but instead the underlying logic has to be corrected.

## Unreachable Logic

This occurs when no test case can be designed or chosen that will cause a specific transition to be exercised. Consider the following diagram and note the part of the logic that can never be executed.



## 10.8 State Machines

Combinatorial testing, which most of the traditional black and white box testing methods are, assumes that the same combination of inputs presented to a system will produce the same set of outputs. This is a reasonable assumption given the kinds of systems that were tested by the classical software testing methodologies.

A state machine is a system where the outputs are determined by both the specific inputs and the current state of the machine. Such systems that show this state specific behaviour are often called stateful systems while those that do not are called stateless.

State machines are said to exhibit sequential behaviour and are of considerable interest in the testing of various systems since components may have states defined and this be state machines.

There are four parts to a state machine:

1. **State:** A defined abstraction that described by some set of criteria by the observer defining the state machine (i.e. states are whatever the designer chooses them to be, choices that we hope are well motivated by the realities of the problem domain).
2. **Transition:** An association between or a sequence of two states.
3. **Event:** An input to the system or an interval of time or a time marker (e.g. Thursday at noon).
4. **Action:** The result of output of the system that follows the introduction of an event.

This view of state machines does not allow any fuzziness in the definitions of states, in other words, we cannot “evolve” slowly from one state to another, rather states a well defined boundary criteria. Generally this is not too onerous a restriction to work under.

When we look at the basic mechanism in the next section, it should be apparent that the highly interactive and event driven nature of most modern complex systems suggests that they are more accurately described when viewed as state machines.

### 10.8.1 The Basic Mechanism

The machine starts in a defined initial state. The machine waits for events to happen. When an event occurs, it is either accepted or not recognized in the current state of the machine. If the event is not recognized in the machine's current state, it is ignored as if it had not occurred at all.

If the event is accepted, the state machine is said to fire a transition and the output action, if one is defined, takes place. The transition causes the machine to enter a new state (which may be the same as the previous state).

This process is repeated over and over until a transition takes the machine into its final state.



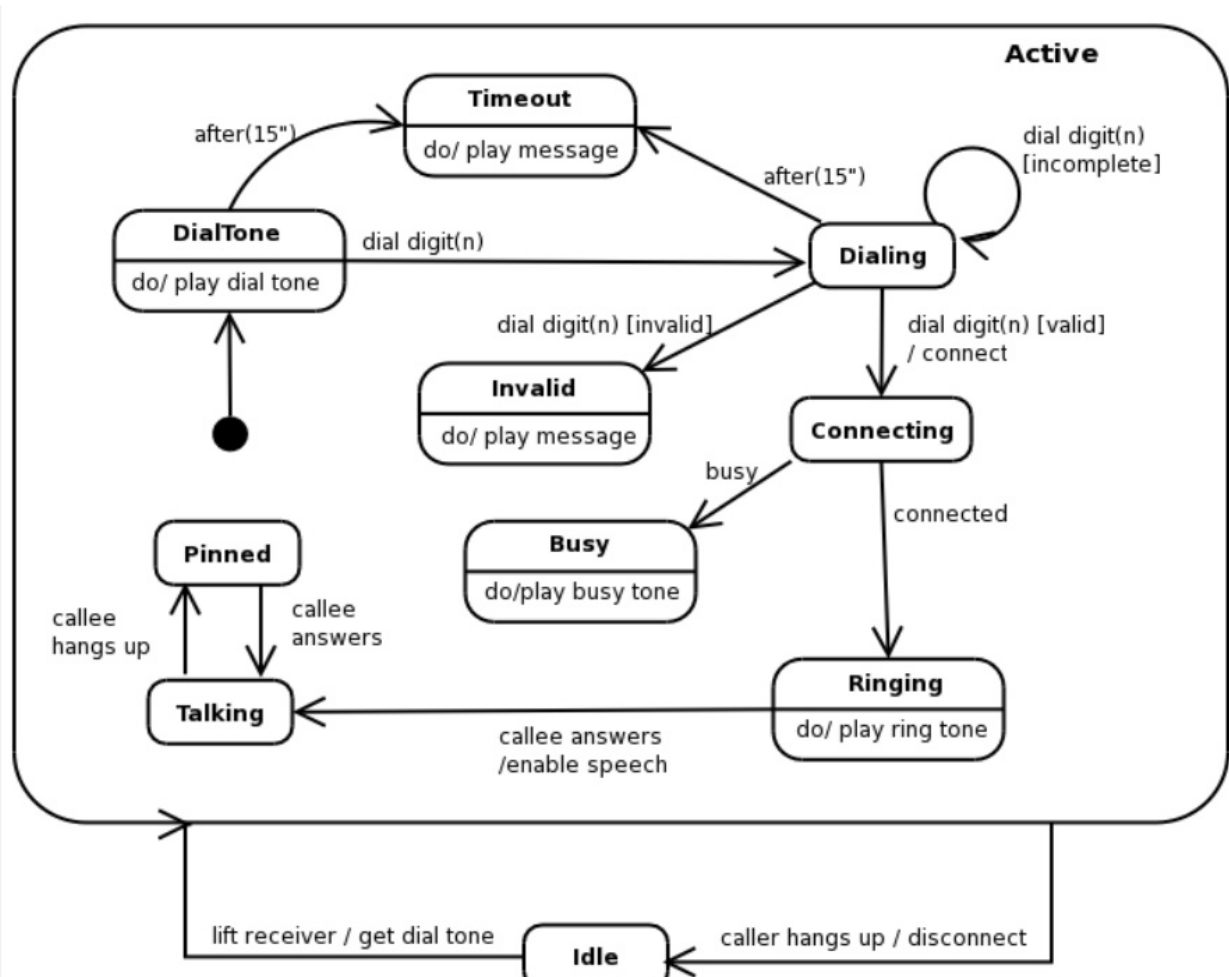
### 10.8.3 UML Statecharts

State machines, or finite state automata, are not innovations from the OO world; the structured paradigm has always worked with Mealy machines and Moore machines, and state transitions tables and so forth.

However, these older finite state techniques were not specifically directed towards complex systems and tend not to scale well when they are applied to analysis of complex systems. UML has borrowed from a number of sources to produce state charts, which are more specifically designed to do complex systems modeling.

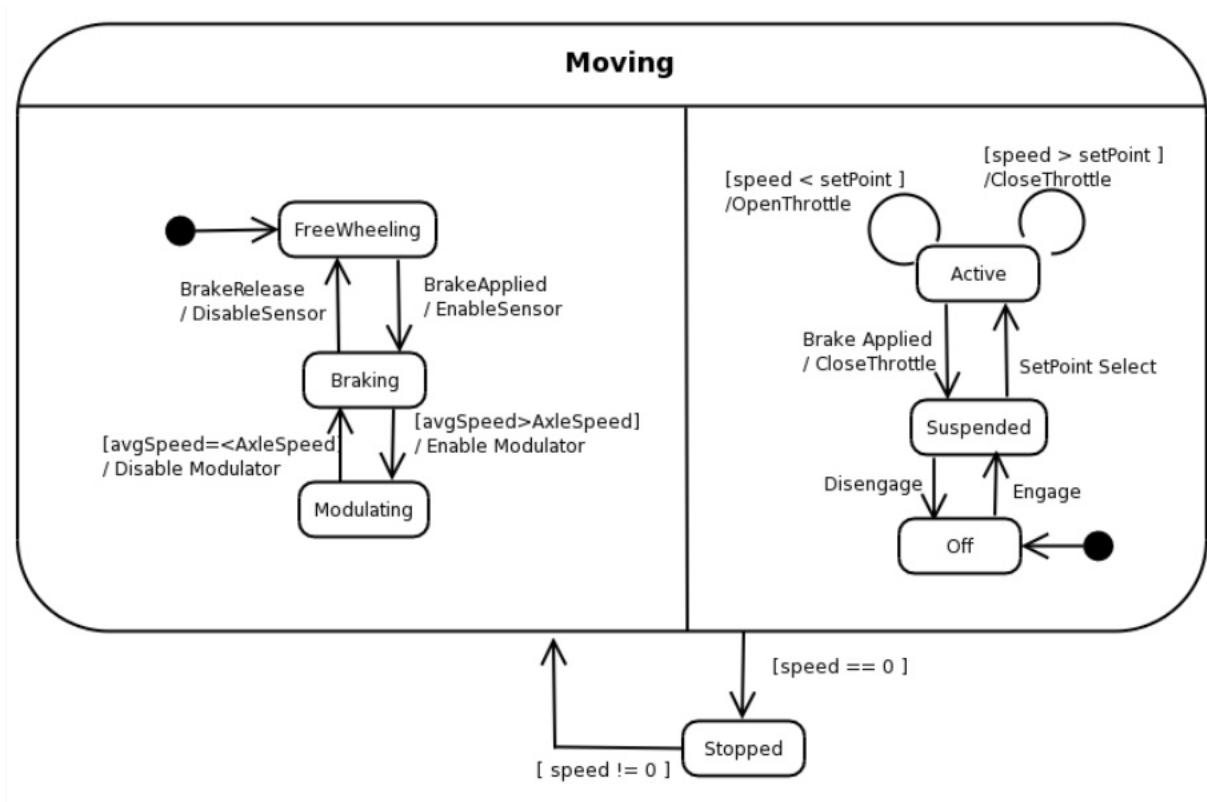
The advantage to state charts over these other techniques is that they tend to be easier to read and work with.

Consider this sample state chart taken from the UML 1.4 specification.



One of the points to note is that the state chart shows what we call nested behaviour which is something that is not normally depicted in a traditional state diagram.

Another feature that state charts have is the ability to show orthogonal sub-states. This is demonstrated in the state chart below.



## State Machine Faults

The challenge of testing state machines is that if a machine has  $n$  states, testing it is roughly equivalent to testing  $n$  traditional applications.

For each state, not only do we have to do normal combinatorial testing, but we have to make sure the state transitions are correct. Binder provides the example of a traffic light; while the red light comes on in response to the RedOn message, this behaviour is correct only when the light is in the Yellow state.

The most complete set of state machine faults I have found is Binder's. These are reproduced here.

**Structural Faults Checklist**

1. One state is designated as the initial state and has only outbound transitions.
2. At least one state is designated as a final state and has only inbound transitions.
3. There are no equivalent states.
4. Every state is reachable from the initial state.
5. The final state is reachable from all other states.
6. Every defined event and every defined action appears in at least one transition.
7. Except for the final and initial states, every state has at least one inbound and one outbound transitions.
8. The events accepted in a state are unique or differentiated by mutually exclusive guard conditions.
9. The state machine is fully specified. Every state/event pair has at least one transition that is either explicitly accepted and produces an action or is rejected, either implicitly or explicitly and handled via some exception mechanism.

**State Definitions**

1. State names are meaningful at there is no more that about 20 states – otherwise consider using a nested state chart.
2. There are no “do nothing” states, like “wait”
3. State names no not contain irrelevant information

**Transitions**

1. The entire set of truth values for all the guards on a transition are accounted for (i.e. we do have an unaccounted for truth value).
2. For all truth values, the set of guard on a transition are mutually exclusive.
3. The variables tested in the guards are available to the class under test.
4. A guard with three or more variables is modeled with a validated decision table.
5. The evaluation of the guard expression does not produce any side effects.

**Robust Behaviour**

1. There is an explicit specification for an error-handling or exception -handling mechanism for events that are implicitly rejected. In effect, the machine is completely specified.

2. The state of the implementation will not be incorrectly changed, corrupted or otherwise undefined as a result of illegal messages. The illegal event mechanism either preserves the last good state, resets the class to a valid state or performs a safe self-destruction.
3. For each action, there are no side effects or exceptions that would corrupt or change the resultant state(s) associated with this action.
4. Explicit exception, error logging, and recovery mechanisms are specified for events that are missing.
5. A timeout interval is specified for each state, explicit exception, error logging, and recovery mechanisms are specified for each timeout.
6. The major and minor failure modes of the target environment for the system under test are catalogued. An explicit exception mechanism is specified for each failure mode.
7. A resultant state, fail-safe terminal state, or fail soft terminal state is specified for each failure mode.
8. If the system is restartable from any other state than the initial state, the restart mechanism is modeled as part of the state machine and defines a path to a viable normal state



## The Testing Maturity Model

### *Module Eleven*

*It's hard to read through a book on the principles of magic without glancing at the cover periodically to make sure it isn't a book on software design.*

Bruce Tognazzini

*If you can't test it, don't build it. If you don't test it, rip it out*

Boris Beizer

*It is not a question of how well each process works, the question is how well they all work together.*

Lloyd Dobens and Clare Crawford-Mason



## 11.1 Development Processes

Before we look at specific processes we are going to take a look at a topic that is very important to this course, the idea of process maturity, but before we do that, we have to review some ideas about development processes.

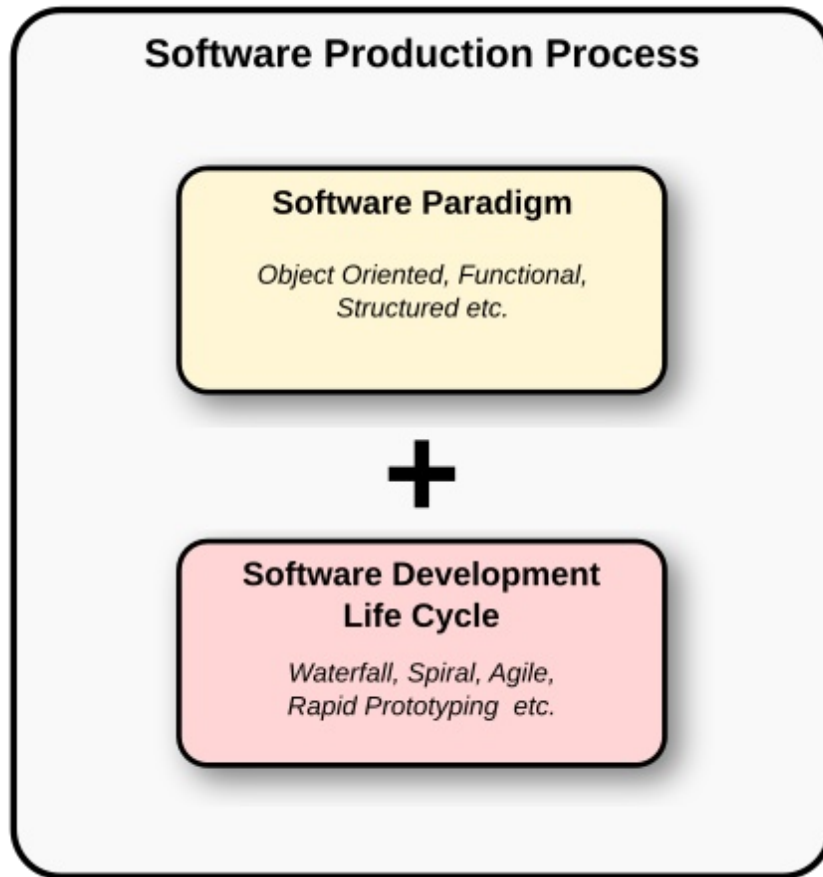
### *11.1.1 Paradigms and Processes*

For many decades, programmers and others involved in software development resisted the idea that software development shared common features with other kinds of development process; instead, software development was seen as a unique or special sort of creative activity. We mentioned this in module one where we described the old belief that nothing anyone else did or had ever done could be compared to programming because software was unlike anything else anyone had ever made or produced. Software development was seen as more like composing a symphony or writing poetry as opposed to more mundane processes like building a house or making a meal.

In the early 1970s, I asked a friend of mine who worked as a programmer at IBM to teach me how to program. His response was “You can't teach programming any more than you can teach someone to have perfect pitch. You either 'get' programming instinctively, or you don't.”

By the early 2000s, the work of the Software Engineering Institute and other industry groups had demolished this idea that software development was exempt from standard engineering principles. However, once the realization dawned on the software development community that development processes are critical to software production, the pendulum swung too far in the opposite direction and the industry became infatuated with the erroneous idea that if we just would get the right production process (Agile, RUP or whatever), then the software would literally write itself because the process was all that mattered.

In this section, we want to clearly delineate the the difference between the life cycle process (how we organize our activities to get the work done) and the paradigm (the actual activities themselves) used in concert with the life cycle process. The same paradigm (object oriented for example) can be used with multiple life cycle processes, and multiple paradigms can be used along with a single life cycle process (Agile for example). Experience shows that confusing these two, for example thinking an Agile technique is part of the object-oriented paradigm, makes it much more difficult to learn, understand or master either.



### 11.1.2 Paradigms

Software developers use specific programming models accompanied by wide range of supporting disciplines and tools created to support their programming model. Examples of these supporting disciplines might be: analysis and design techniques, specific conceptual modeling techniques and system architectures. This collection of methods, concepts and tools is what we are calling a paradigm in this course, borrowing from Thomas Kuhn's famous book *The Structure of Scientific Revolutions* where he defined a paradigm as [Kuhn70]:

*The entire constellation of beliefs, values, techniques and so on shared by the members of a given community. It also denotes one sort of element in that constellation, the concrete puzzle-solutions which, employed as models or examples, can replace explicit rules as a basis for the solution of the remaining puzzles of normal science.*

This is actually summarized quite nicely by Wikipedia when it describes what a paradigm describes and delineates:

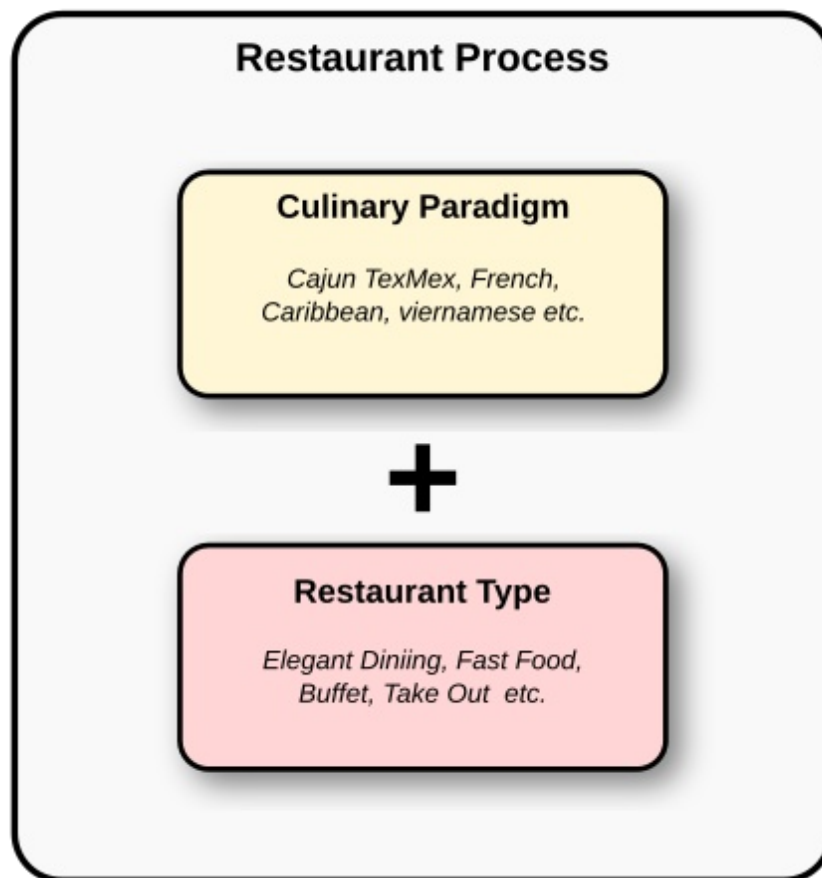
1. What is to be observed and scrutinized.
2. The kind of questions that are supposed to be asked and probed for answers in relation to this subject.

3. How these questions are to be structured.
4. What predictions made by the primary theory within the discipline.
5. How the results of scientific investigations should be interpreted.
6. How an experiment is to be conducted, and what equipment is available to conduct the experiment.

Paradigms, in a more general sense, are everywhere because humans tend to approach producing wide range of things in similar ways; the idea of life cycle and paradigm in software has analogues in many other contexts, cooking for example. The various software paradigms we use in development are analogous to the different kinds of cuisines that people use to create meals.

1. **Cooking Paradigms:** French cooking, Indian cooking, Tex Mex, Barbecue, Dessert making..etc
2. **Software Paradigms:** Structured Programming, Object Oriented, Functional Programming, Symbolic Programming, Logical Programming.

What sort of things might be found in each of these paradigms?



In the cooking paradigms we find specific cooking techniques (stir fry versus roasting), different ingredients and spices (curry versus ketchup), different tools (pastry brushes versus chopping blocks), different ideas on what sort of foods should be used (haggis versus began), different ways of deciding if something is “well prepared”, different ways of creating and assembling dishes for presentation and so on.

In software paradigms we see different assumptions about what sort of problems we are solving, different ways to organize code into applications, different ideas about what “well written code” means, different ways of designing programs or decomposing problems and of course different ideas about what makes good code.

At this point we can see where the original ideas about not being able to quantify or standardize code development had some basis – programmers of the time operated like cooks who just went into the kitchen to “create.” Many people cook intuitively – they just create on the fly and usually, if they are good, they wind up with something amazing; but they can't recreate it because it was a singular inspired act. In many respects, that is how many programmers often develop code.

### ***11.1.3 The Iron Triangle***

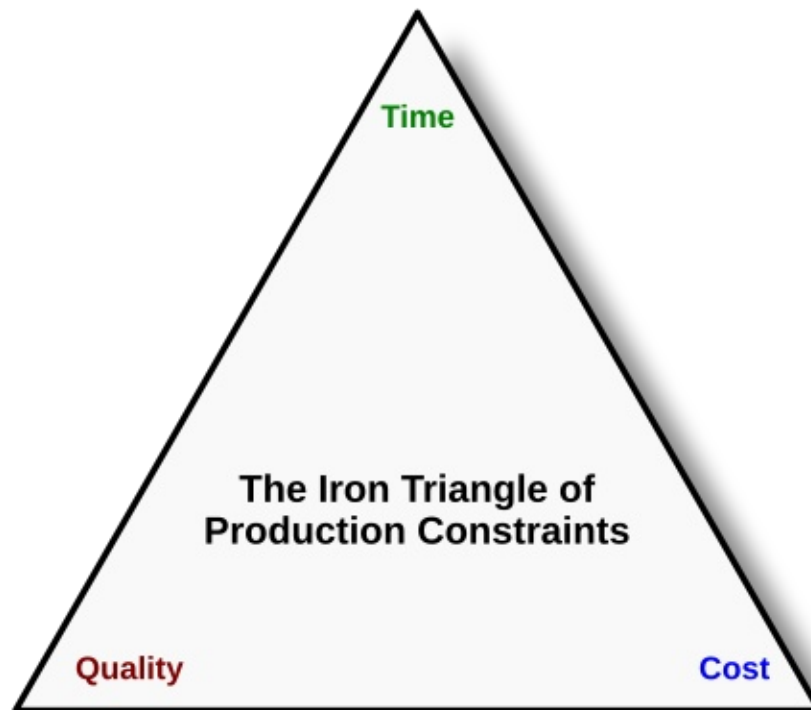
Just because you are a fantastic cook doesn't mean you can run a restaurant. Running a restaurant means cooking under real-world constraints; specifically:

1. **Specification:** You can no longer just cook what you want, you have to cook to customer specification – you have to cook what the customer orders. If they want roast chicken, you can't decide to cook the grilled salmon instead.
2. **Time:** You have to deliver the meal to the customer within a reasonable time – customers won't come back if they find themselves waiting longer than they normally expect to wait in for service in a restaurant.
3. **Budget:** You have to make sure that it doesn't cost more to deliver the meal than you are charging the customer for it.

These three constraints are collectively known in engineering as the iron triangle – it is the set of constraints that rules every single engineering project or other activity where we build or make something. An old saying in engineering is “You can have it good, fast and cheap, but you only get to pick two,” which describes the reality of working under these constraints. If you want it fast and cheap for example, don't expect it to be good.

When you run a restaurant, the iron triangle doesn't apply for just one meal but for every meal you create, day after day and customer after customer. At the minimum, this means that the kitchen has to be organized enough and there has to be an established process for the kitchen staff to follow consistently when preparing meals so that the restaurant can meet the iron triangle constraints every single time, whether they are preparing one meal or a hundred at the same time. Or at least, if the restaurant wants to stay in business.





Running a restaurant is not the same as cooking. Many people who are amazing cooks open restaurants and fail because they don't know how to run a restaurant. On the other hand, fast food restaurants have become successful because they have mastered a restaurant life cycle process specialized for fast food, but no one would argue that the minimum wage hamburger assembler in the back of a fast food restaurant is a brilliant cook or a chef.

There are cooking paradigms but there are also some basic restaurant life cycle processes – I've just picked several for illustrative purposes that seemed to be obvious. The important thing to notice is that the life cycle process doesn't depend on the paradigm or vice versa – we choose them independently but we motivate our choices on the basis of what combination of paradigm and process works for us.

#### ***11.1.4 Software Engineering***

We can apply the same ideas about cooking and restaurants to programming. I've worked with many brilliant programmers who, when left to their own devices, would produce amazing code that was often absolutely brilliant, but yet could not hold down a programming job. The reason is that they had mastered their paradigm – the art of writing code, but could not transition to being a software engineer because that meant having to write code under the constraints of the iron triangle.

This brings us to the most important point of this section:

***Anytime we build or create something subject to the iron triangle of constraints, we will use some form of the same production process as anyone else.***

Just like all restaurants have to follow a similar life cycle process independent of their paradigm, all software engineering follows similar life cycle processes regardless of whether we are using an object oriented, functional or structured programming paradigm.

### **11.1.5 Universality of Development Processes**

We have just used cooking as an example of paradigms – French, Chinese, Cajun and other styles of cooking represented culinary paradigms each with their own techniques, types of utensils and ways of preparing food. We also used the idea that a running a restaurant requires a restaurant life cycle process to get food to customers on time, to spec and within budget.

By the universality of the process we mean that all restaurants use the same set of life cycle processes regardless of whether they are Indian, Pizza or Sushi restaurants. Of course there isn't just one restaurant life cycle process, there are several variations on the basic life cycle process: the elegant dining room process, the fast food process, the buffet process.

So now we can look at combinations of restaurant life cycle processes and cooking paradigms to give us the elegant Cajun restaurant, the Chinese buffet and the Indian fast food restaurant.

We can apply that same analysis to software development. We have several variations on the basic development life cycle process – waterfall, iterative and agile – that can be used with different paradigms – object-oriented, structured, functional, logical, etc – to produce different software development process. However, in the IT world we tend to refer to life cycle processes as System Development Life Cycles or SDLCs.

It is not just chance that restaurants and software development share a similar pattern of paradigm and life cycle; there is a common development process that is followed whenever anyone makes anything under the constraints of cost, time and specification, whether it is building a house, doing surgery or making a movie. For the purposes of this course we will call this the production or engineering process since has always been connected closely with the various engineering disciplines.

This can be extend this further by saying that:

***Anytime we build or create something subject to the iron triangle of constraints, we will use the same general form of production process as anyone else.***



Now at this level we can see that software development under constraint is not merely programming, but has a lot in common with building other sorts of things. To keep this distinction clear in this course we make the following distinction between programming and software engineering.

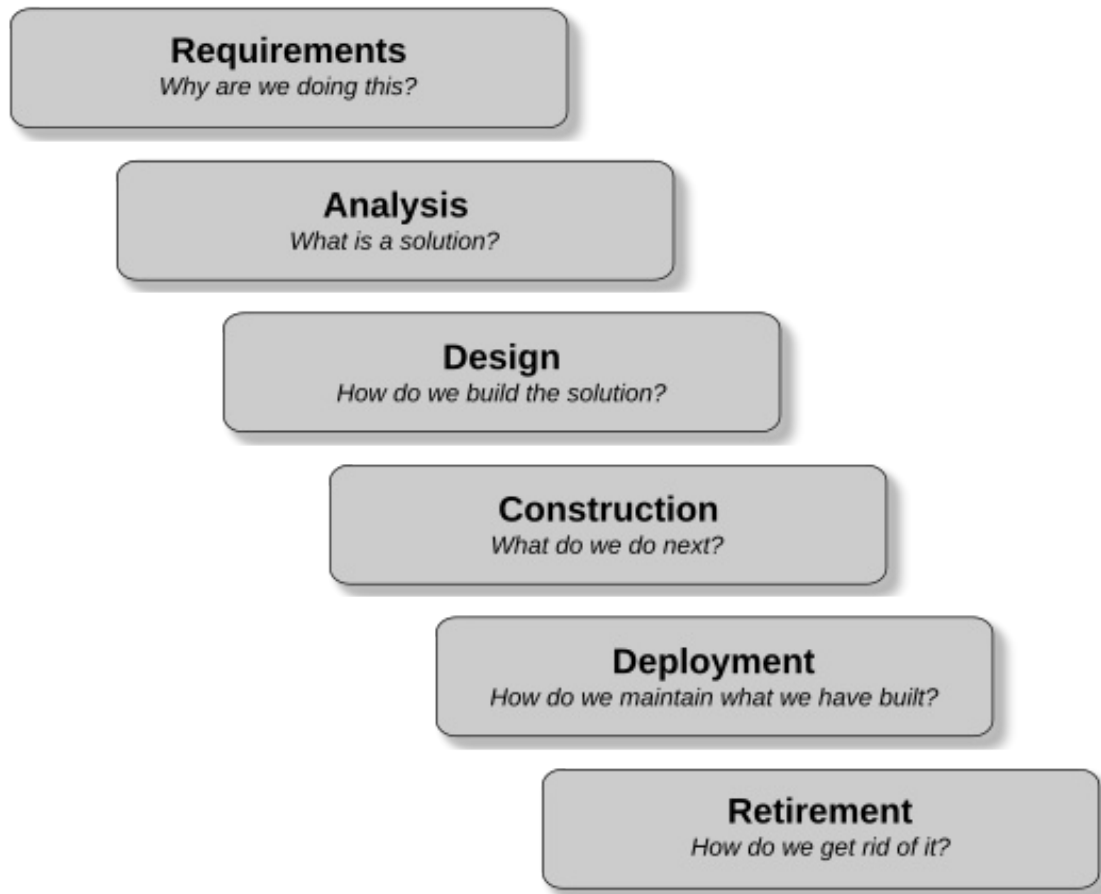
Programming is the art of writing software. From an “art of programming” perspective, good programming means writing elegant code – where the term “elegant” is used in the sense mathematicians use it to mean something that is simple, effective, and optimal – following best programming practices for that programming paradigm including basing the code on appropriate, effective and efficient algorithms and designs. Software engineering is the discipline of delivering working code within the constraints of deadlines, limited resources, meeting external standards and conforming to specifications of system performance and functionality.

### **Testing as a Paradigm**

*Many of the core ideas in testing and QA are part of a paradigm and are shared across a variety of disciplines like engineering and the social sciences. The sort of testing we do is not part of the process model but is an adaptation of a general testing and quality paradigm to software development.*

## 11.2 Production Processes

There exists a proliferation of software development processes and methodologies being promoted in the software development marketplace, each of which is usually claimed by its marketers to be the one that finally fixes the problems inherent in the other processes and will literally force your developers to write next to perfect code, even if they can't program. Or at least it seems that way when you read their sales materials.



More realistically, no matter what methodologies are actually in use, they all have to share a common core or set of logical steps anyone must go through on any project. To get the job done, each methodology has to go through the same logical steps that we are calling the production process, or more generally, a description of “How people build things.”

The production process should look somewhat familiar since parts of it are often referred to in the programming community as the waterfall system development life cycle (SDLC). The production process is the solving of a logical sequence of natural ques-

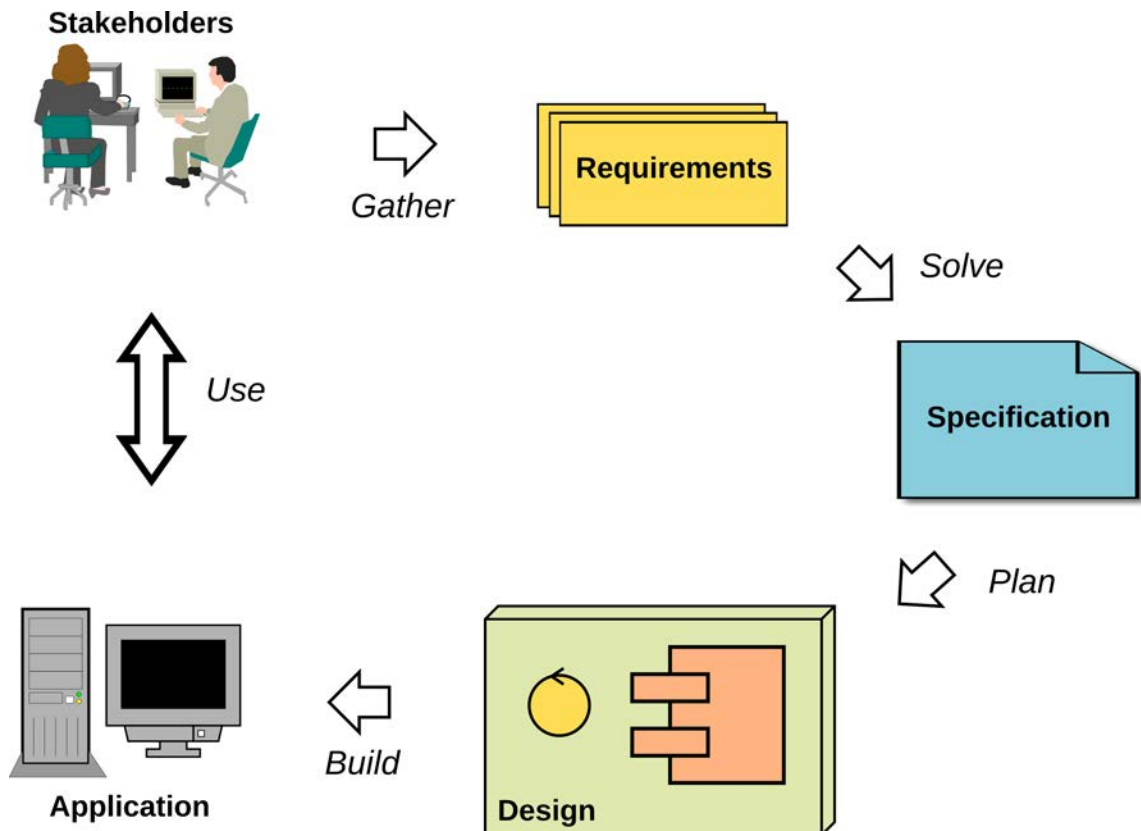


tions or problems that we encounter in any kind of construction process. These steps are:

1. **Requirements:** The basic question that has to be answered during requirements is “Why are we building this?” This is generally followed by two other kind questions. The first is “What do the users need what we are building to do for them?” At this point, we need to know the functional requirements (what the stakeholders expect what we are building to do for them) and the non-functional requirements (what the performance criteria are that what we are building has to meet). The second question is “What are the constraints – budget, time, architecture, etc – that have to be considered when building the solution?”
2. **Analysis:** In the analysis phase we formulate solutions to the problem posed by the requirements. We usually describe the solution in a specification that describes what is to be built. The specification serves as a design target, which means that whatever design we propose has to produce a product that meets the description of the solution in the specification. We may come up with multiple ways to solve the problem but which differ in terms of cost, quality and other factors, but eventually we need to settle on a single solution.
3. **Design:** While analysis describes a solution, a specification does not tell us how that solution should be built. A design describes how the solution is to be built given a specific set of resources, capabilities and constraints. In other words, for a single specification, we may have a number of different designs depending on what we have available to work with.
4. **Construction:** The construction phase is where the product described in the specification is built according to a given design. Notice that having a design alone is not sufficient to built something, you also have to employ the appropriate construction techniques.
5. **Deployment:** Something is deployed when it goes into use. The questions that we have to ask for the deployment phase are often about operational issues such as: “How are changes or repairs to be made?” or “How are users and maintenance people to be trained?” or “How do we monitor the product for possible malfunctions?”
6. **Retirement:** All products and systems have to be taken out of service at some point, whether it's a nuclear reactor, a model of car or a software application. This phase is concerned with how we move the product out of the its operational environment and replace it, if necessary, with its successor.

### 11.2.1 The Generic Software Development Process

Having looked at the general production process, now we look at how it is the core of what we will call the generic software development process, which is just translating how we describe the stages of the engineering process into the language and context of software development.



This natural progression is captured by the flow from requirements through specifications and design to building. In order to be effective at each step, we need to have at least a partial solution to the previous problems.

The stages are:

1. **Requirements:** Remember the purpose of the requirements phase is to answer the question “Why are we doing this?” At this point we want to identify the overall business need that is driving the software development. We also want identify who has requirements (stakeholders) and who has constraints that set limits on what we can do, whether these are budgetary, resource availability or process or business rules. And of course we collect and manage the requirements in a robust enough way to move to the next stage.

2. **Analysis:** The analysis produces a specification that describes the solution to the problem and meets the requirements identified in the previous stage. The specification describes the system in terms of its features: what it should look like and what it should do. We may develop multiple potential solutions but usually by the design phase we have chosen one as the specification we will use.
3. **Design:** Given that we know what we have to do, how are we going to turn this into reality given what we have to work with? This is the role of design; to create a step by step plan as to how the specified solution will be built. The design will require choices on architectures, technology, and other proven techniques – remember the part about engineers reusing design.
4. **Construction:** This represents the bulk of the time and cost of the project; the writing, testing and deployment of the application based on the design.

Notice that the generic software development process does not carry on through deployment and retirement. This reflects the historical fact that in the past, most software projects considered producing the delivery of the working software to be the final goal of their production process. In recent years a different way of looking at software as a product has emerged, called Application Lifecycle Management (ALM), which is an attempt to apply general product management principles to software. Since this is increasingly of interest to Agile testers, an overview of ALM is provided later in this module.

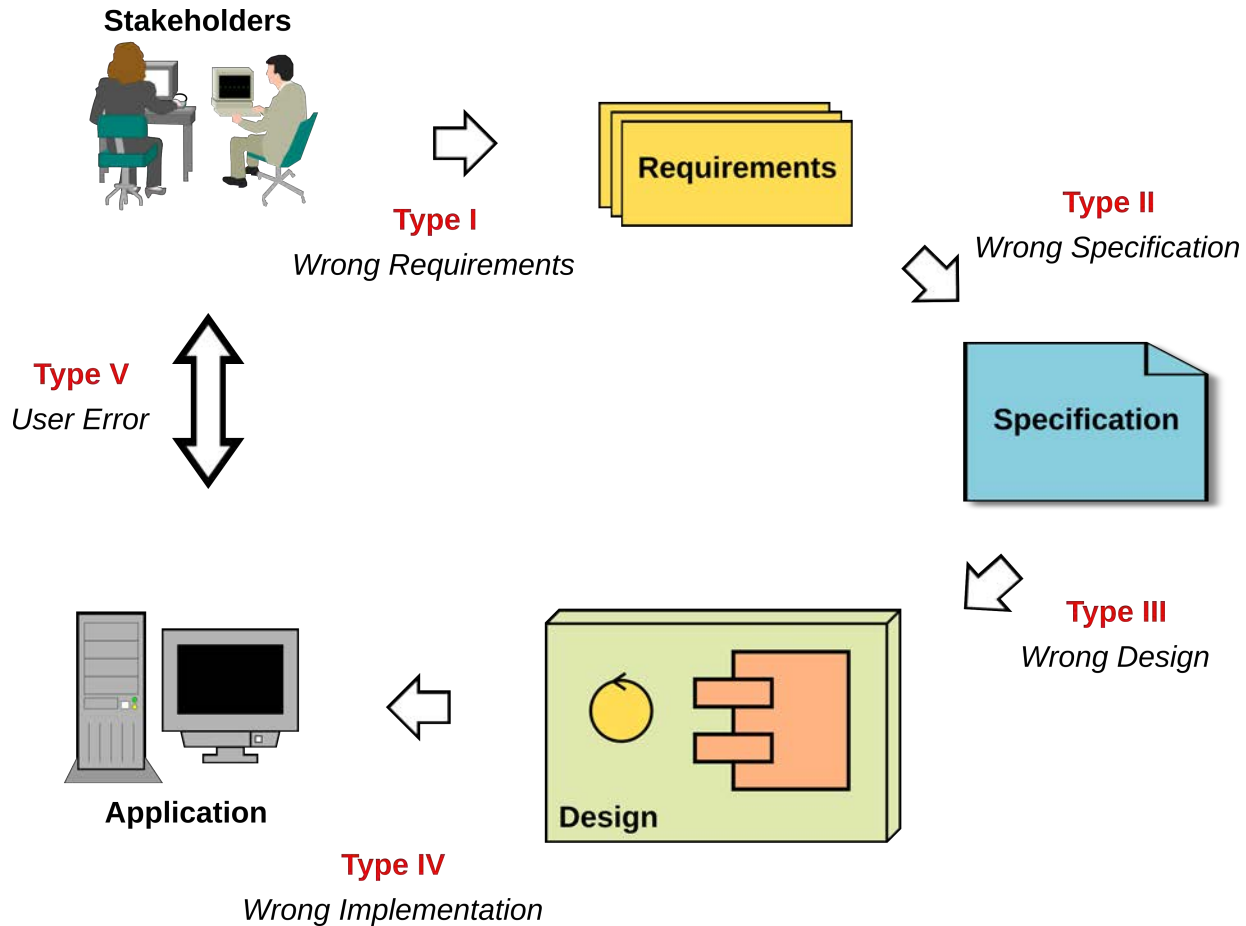
### 11.2.2 Where Things Go Wrong

Five types of systemic errors can commonly occur in building a system. Each of these error types represents a risk factor that has to be controlled for in the project development life cycle.

**Type I: Requirements are wrong.** This is the most difficult kind of error to correct since it cannot be detected within the project but can only be detected with the participation of the stakeholders who are outside the development effort. In the worst case scenario, identification of the error takes place after the finished product is deployed and being used by the stakeholders.

**Type II: Specifications do not satisfy the requirements.** Our system may work perfectly, but it does the wrong thing perfectly. Generally, a type II error occurs if there are requirements that are not addressed by the specification (omissions), features proposed that provides functionality that does not meet any requirement (surprise functionality), or a proposed feature that does not satisfy the requirement correctly (failures).

**Type III: System design does not meet the specifications.** Following the design leads to a system that does not operate as specified. For example, the specifications may require the use of encryption for security, but the encryption methodologies were omitted in the design by mistake. Or the architecture proposed cannot handle the projected throughput or exceeds the allowable response time.



**Type IV: The system build and artifacts do not conform to the design.** This generally means that someone made mistakes in building the system whether writing code or performing some other construction task that were specified in the design – omitting exception handlers for example.

**Type V: User Error.** This type of error is an unavoidable human error. The problem is that people do not always behave rationally and, while we may have designed and implemented an entirely rational system, it may be used in an irrational manner.

Type II - IV errors are called transcription errors, a term coined back in the 1970s by Glenford Myers, because they generally occur when someone has misunderstood (i.e. incorrectly transcribed) the outputs of the previous stage of the process. In other words, we get a wrong design because someone misunderstood the specifications and we get wrong specifications because someone misunderstood the requirements.

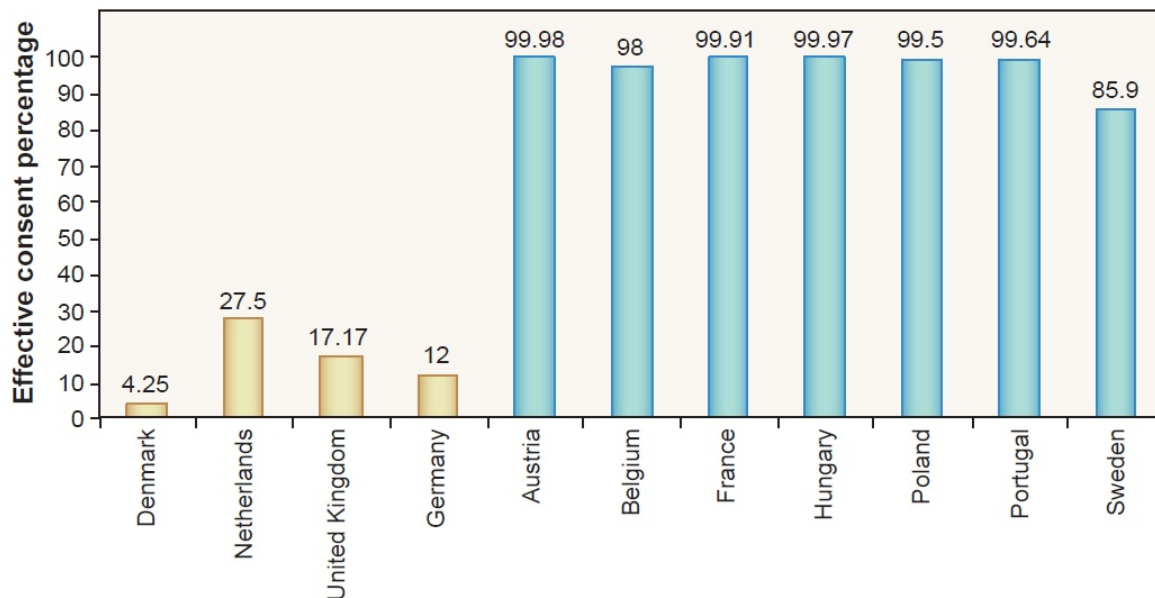
## Behavioural Economics



One of the confounding factors that we will have to deal with, especially for the type V errors mentioned before, is that people do not behave rationally. The science of economics makes a number of predictions about how markets and people will act based on a primary assumption that that people behave in rational ways to maximize their own utility in the choices they make. Very often, game theory is used to model choices because its models are based on that same assumption.

However, we discover that in real life these predictions often fail. One of the reasons for this failure is that while people often do behave rationally, people also behave in predictable but irrational ways when they make choices. The following is an example of what we mean.

In a paper entitled *The meaning of default options for potential organ donors* by Shai Davidai, Thomas Gilovich, and Lee D. Ross, the authors studied the underlying reasons for different rates of organ donation in different countries. The following is from their paper and describes the study:



**Effective consent rates, by country.** Explicit consent (opt-in, gold) and presumed consent (opt-out, blue).

*Rates of participation in organ donation programs are known to be powerfully influenced by the relevant default policy in effect (“opt-in” vs. “opt-out”). Three studies provide evidence that this difference in participation may occur in part because the requirement to opt-in or opt-out results in large differences in the meaning that individuals attach to participation.*

*Some countries (e.g., Germany and The Netherlands) have explicit consent, or “opt-in,” policies whereby citizens must indicate their willingness to have their organs harvested in the event of a fatal accident. Other countries (e.g., Austria and Belgium) have presumed consent, or “opt-out,” policies whereby citizens must indicate their unwillingness to participate. This difference across countries provides a very telling natural experiment. The difference in organ donation rates—typically exceeding 90% in opt-out countries and failing to reach even 15% in opt-in countries—astonishes most readers.*

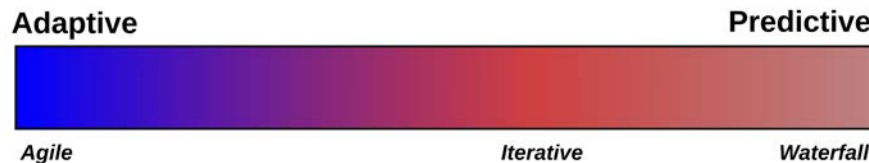
What this study shows is that the deciding factor on the donation rates is whether or not a person has to check a box to opt in or opt out. And later, based on what choice the donors made, they viewed the value of organ donation differently, or one would say they rationalized their behaviour after the fact.



## 11.3 Process Types

While all processes follow the generic steps, they are often implemented in different ways to respond to different development contexts. One of the main ways processes differ is along what we call the adaptive-predictive dimension.

### 11.3.1 Predictive versus Adaptive Processes



Highly adaptable (agile like) methodologies are designed to respond quickly to changing requirements and architectural constraints which makes planning more like guess work since the changes and unexpected discoveries between the development of a plan and the product delivery will probably invalidate some of the previous planning assumptions.

Highly predictive methodologies work by planning the future in detail. The project plan is developed for a specific known development target. Changes to that plan almost always cause rework. Predictive teams usually use a change management process so that only significant changes are considered for incorporation into the project.

The essential difference between the two approaches is that predictive methodologies know what is to be built beforehand, while adaptive methodologies do not, often because they are trying to discover a solution to an existing problem.

In terms of requirements, predictive methodologies can identify all the requirements in the early stages of a project and can assume those requirements will not change significantly during the project. Adaptive methodologies are used when only some of the requirements are known at the start of the project and it is expected that requirements will change while the project is underway, sometimes in unpredictable ways.

One of the decisions many organizations have to make is "How agile should we be?" There is a common misconception that Agile is the same as having no process. It is not; it is just a different kind of process. One of the key factors that motivates the different methodology types is how well we know the requirements at the start of the project.

### Predictive Processes

Many IT projects are highly predictive which allows for accurate planning in terms of time, budgets and resource allocation, although we tend to find these sorts of projects in more engineering oriented contexts such as embedded control systems, device and sensor automation and other similar areas.

In the Agile world, the waterfall process gets a bad rap as being fundamentally flawed or broken. This is not a justified criticism. The waterfall and its variants have been used successfully for thousands of years. While the waterfall has been very successful in the engineering world, it is not as effective in the software development world for highly user interactive projects because:

1. Requirements change much faster for software projects than for traditional engineering projects and most software organizations have no or little experience with change management.
2. Users are not often sure of their requirements until they actually start to use the software. As they use the new technology, they tend to come up with new requirements and often change their previous requirements.
3. Software technologies and their economics change rapidly, often during the development project. In particular, the introduction of new products and services into the marketplace can change user expectations about what applications should do for them and how they should function. For example, the popularity of Google made users expect that they should have some kind of search functionality in their other applications.

### ***Iterative Processes***

An iterative process is one in which a significant portion of the requirements are known but not all of them. The expectation is that during the project, requirements and other factors will be discovered which will have a significant impact on the final result.

To accommodate this flux in requirements, iterative methodologies tend to look like predictive methodologies but with “checkpoints” during the project where the final goal can be reevaluated and re-planned if necessary.

This is usually done by chopping the project up into mini projects. A specification is produced early on that is either

1. Incomplete: That means there are pieces missing from the specification since we don't know all the requirements.
2. Putative: While we can propose a specification, we expect that as we discover more requirements, the specification will change accordingly.

### **Adaptive Processes**

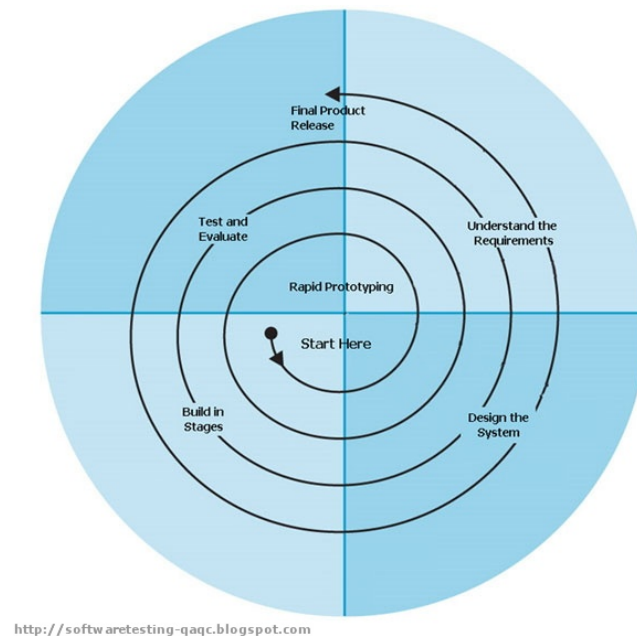
Highly adaptable processes are needed when the final solution cannot be accurately predicated. Adaptive processes are used when a problem needs to be solved, or when there are very few known requirements or no obvious solution at the start of the project. In this case the developer often has to take a few known requirements, craft a solution to those requirements and then present that partial solution to the user. By using this early prototype, the developer works with the user to find more requirements to incorporate



into a second prototype. This cyclic process continues until the user declares that the result is “good enough” so that the development process can be considered as over and the software delivered.

Adaptive processes are characterized by the requirements being in a state of constant flux and that multiple attempts at a solution for a single requirement may be necessary until we get something that works. Projects are usually complete when the problem is solved or when the business side essentially says “good enough.”

## Non-Agile Adaptive Processes



Just as point of interest, adaptive processes in software development have been around since the 1970s. Two of the more popular adaptive processes that gained prominence in the past was the Barry Boehm's Spiral methodology developed in 1986 which defined a series of iterations with the objective of producing prototypes which were then used to provide inputs into the subsequent iterations.

Another similar adaptive methodology is James Martin Rapid Application Development (RAD) which he developed in the 1980s while at IBM. The model was built around the idea that for some sorts of development, like working with user interfaces, the requirements are too fluid for a predictive approach. The RAD approach, like the Spiral methodology, centred around getting a prototype into the hands of the users to start generating feedback that would be used to continuously improve the product.

### 11.3.2 Agile Development Processes

In the late 1990's there was a rebellion against the high level of ceremony and formalism of RUP, which had become quite popular. Starting with Extreme Programming, several methodologies started to emerge that shared a common core of ideas:

1. Close collaboration between the development team and business experts.
2. Face-to-face communication as opposed to written documentation.
3. Frequent delivery of working prototypes to facilitate developer and customer communication.
4. Small, tight, self-organizing teams.
5. Sets of techniques to "craft" code and organize the team to anticipate the inevitable "churn" of constantly changing requirements.

These methodologies referred to themselves as "light-weight", "agile" and "low ceremony" and were intended for smaller projects with fast turn around times that did not need the formality or complexity of the RUP style methodologies.

Martin Fowler notes:

*Agile methodologies developed as a reaction to these [heavy-weight] methodologies. For many people the appeal of these agile methodologies is their reaction to the bureaucracy of the engineering methodologies. These new methods attempt a useful compromise between no process and too much process, providing just enough process to gain a reasonable pay-off. The most immediate difference is that they are less document-oriented, usually emphasizing a smaller amount of documentation for a given task. In many ways they are rather code-oriented: following a route that says that the key part of documentation is source code.*

*Agile methods are adaptive rather than predictive. Engineering [heavy-weight] methods tend to try to plan out a large part of the software process in great detail for a long span of time, this works well until things change. So their nature is to resist change. The agile methods, however, welcome change. They try to be processes that adapt and thrive on change, even to the point of changing themselves.*

*Agile methods are people-oriented rather than process-oriented. The goal of engineering methods is to define a process that will work well whoever happens to be using it. Agile methods assert that no process will ever make up the skill of the development team, so the role of a process is to support the development team in their work.*

On February 11-13, 2001, the famous Snowbird conference took place where seventeen representatives from a number of methods like Extreme Programming, SCRUM, DSDM, Adaptive Software Development, Crystal, Feature-Driven Development and Pragmatic Programming, and others that were attempts to satisfy the the need for an alternative to documentation driven, heavyweight software development processes. Out of that meeting came the "Agile Manifesto."

#### The Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:



|                              |      |                             |
|------------------------------|------|-----------------------------|
| Individuals and interactions | over | Processes and tools         |
| Working software             | over | Comprehensive documentation |
| Customer collaboration       | over | Contract negotiation        |
| Responding to change         | over | Following a plan            |

We follow these principles:

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity--the art of maximizing the amount of work not done--is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
11. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

### ***11.3.4 Extreme Programming***

Many of the concepts and techniques in Agile testing, such as TDD and using acceptance tests to drive overall development, originated in a Agile methodology called Extreme Programming or XP. The following brief overview of XP is derived from the description of the methodology by its creator, Kent Beck.

Extreme Programming (XP) is a set of values, principles and practices for rapidly developing high-quality software that provides the highest value for the customer in the fastest way possible. XP is extreme in the sense that it takes 12 well-known software development "best practices" to their logical extremes, and these in turn are the XP core. TDD or

Test Driven Development is one of those core practices as is the concept of refactoring.

XP is intended to be easily used for projects of up to a dozen programmers and twice that with some difficulty. While XP itself does not scale well, the idea to large scale XP development is that a project is organized overall along more traditional models, but is then split into multiple smaller XP projects.

The other point that we have to keep in mind is that we can have both traditional and Agile methodologies within a single organization or project. The two are not mutually exclusive.

For example, we may have a project to automate a customer management system for a bank. Banking development projects are usually high governance with layers of stakeholders putting in their requirements (state banking commissions, state and federal agencies, risk management departments, security departments, etc)

The overall project probably will be done by a waterfall approach (typical in banks since they are very risk averse and tend to require reams of documentation, sign-offs and all those other "fragile" constructs). However, suppose that within the project is a requirements to have an "Account Manager Workbench" – a computerized interface that the account managers will use to do their work, which was up to this point done manually.

The requirements people talk to the account managers who do not have any idea what this should look like (after all, they've never seen one before). Suddenly, we have an Agile sub-project! Using Agile methods, we can develop a workbench in concert with the account managers who will evolve their as they see the technology unfold.

### The XP Onion

XP is seen as a set practices within practices. Each layer of the onion feeds into and defines the activities in the layer beneath it.

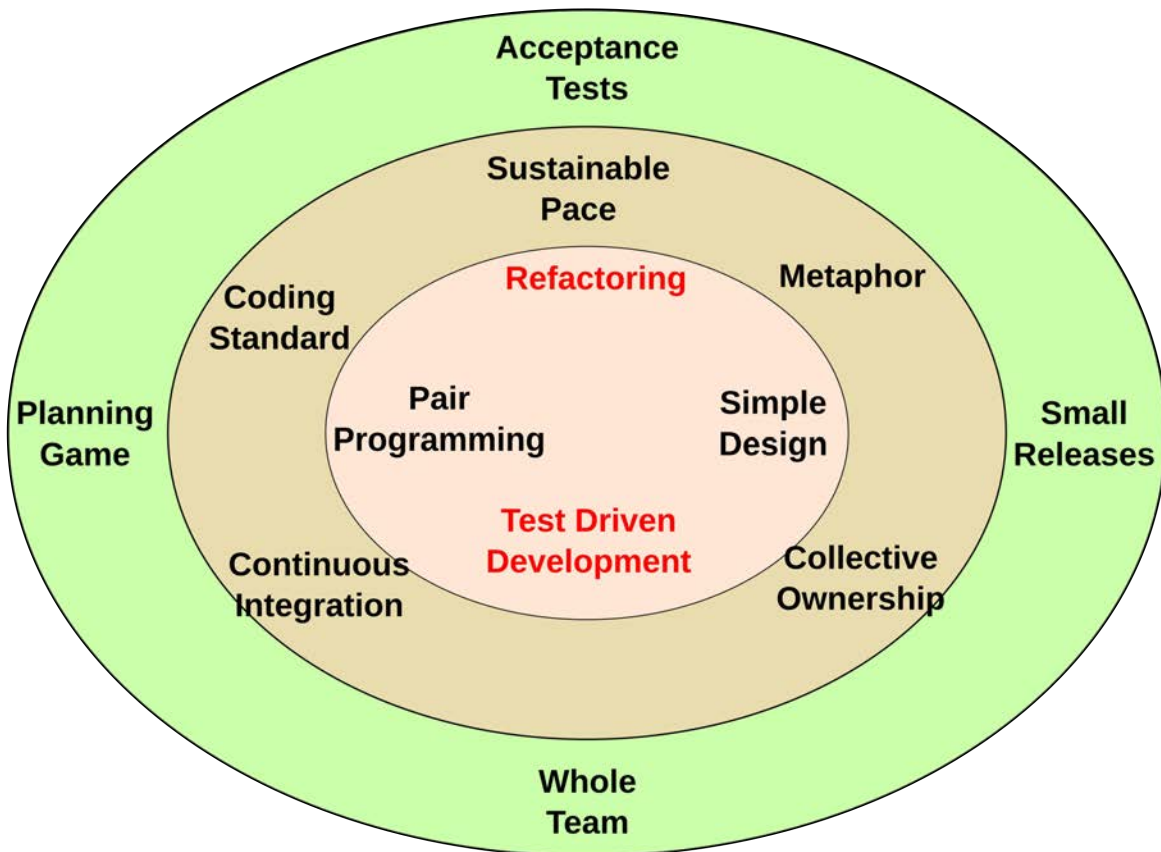
**The Planning Game:** Customers and developers cooperate to produce the maximum business value as rapidly as possible. Customers comes up with a list of desired features for the system. Each feature is written out as a User Story, which gives the feature a name. Developers estimate how much effort each story will take, and how much can be done an iteration. Customers prioritize the stories to be implemented, as well as when and how often to produce a production releases of the system.

**Small Releases:** Start with the smallest useful feature set. Release early and often, adding a few features each time.

**Metaphor:** Each project has an organizing metaphor, to provide an easy to remember naming convention.

**Simple Design:** Always use the simplest possible design that gets the job done. The requirements will change tomorrow, so only do what's needed to meet today's requirements.





**Continuous Testing (TDD) :** Tests are written for a feature to be added, then the code is written. When the suite runs, the job is done.

**Refactoring:** Refactor out any duplicate code generated in a coding session. You can do this with confidence that you didn't break anything because you have the tests.

**Pair Programming:** All production code is written by two programmers sitting at one machine. Essentially, all code is reviewed as it is written.

**Collective Code Ownership:** No one "owns" a module. Anyone is expected to be able to work on any part of the code base at any time.

**Continuous Integration:** All changes are integrated into the code base at least daily. The tests have to run 100% both before and after integration.

**40-Hour Work Week (Sustainable Pace):** Programmers go home on time, up to one week of overtime is allowed.

**On-site Customer Tests:** Development team has continuous access to a real live customer, that is, someone who will actually be using the system who does continuous test generation and acceptance.

**Coding Standards:** Everyone codes to the same standards so that there is no way to tell by inspection who on the team worked on a specific piece of code.

## XP Evolving Architecture

XP embraces change which means that an architecture and design "evolve" rather than being initially specified as something the application has to "fit into," in XP architecture has the sense of "the essential structure." (XP tends to like Zen-like statements)

Architecture and design are expressed in XP through:

1. Spike
2. Metaphor
3. First Iteration
4. Small Releases
5. Refactoring
6. Team Practices

In XP design evolves because it is driven by the requirements of the customer, which usually evolve and change as the system is developed. Refactoring is not just writing code, evolving design is not the same as a haphazard or unstructured design changes.

In fact, if the XP developers do not apply good design principles to the emerging design, it will become brittle and non-functional in a very short period of time. We will look at a couple of fundamental principles a bit later on.

One of the important uses of refactoring is to simplify the code. One of the benefits of refactoring for design is that the simplified code is usually better designed and more robust so that it is more readily changeable or resistant to changes in other parts of the system.

## Spikes

During early release planning game, the team develops "spikes" which is a quick throw-away potential solution. Based on the user stories and the spikes, an architectural approach will emerge that will suggest the system's structure.

Stories suggest constraints that guide architectural choices. One story might identity security and access constraints while another might suggest performance levels for multiple users. This may suggest a spike simulating multiple simultaneous users that might provide insight into replication and rollover designs.



The spikes allow the developer and customer to examine the costs and trade offs of the various proposed solutions. Spikes are used to experiment with implementation of functionality and how the solution would impact the existing system structure.

As an example of a spike, we might sit down with a customer that talks about his clients being able to place an order on line and have the results feed into the existing corporate order database. A spike might be produced that looks at developing a solution round a web server, and an application server that would allow back end connections to the existing database as well as to a new "on-line customer" database. Issues explored in the spike might revolve around the security and encryption of customer data, as well as firewalls and other system security features.

## **Metaphor**

From the initiation of the project, an effective metaphor is developed that helps guides the solution. The metaphor is like a high level conceptual design that describes the solution in the language of the problem domain. The metaphor describes the main conceptual objects and their relationships and provides a "check-point" or a high level view of a system that keeps the design oriented or on track. The metaphor also provides a common language for naming and talking about the artifacts of the system

The metaphor may change as more is discovered about the customer's domain. The metaphor is not tied to the architecture but suggests and influences the architecture. The system design supports the metaphor.

As an example of a metaphor, consider a project to develop web access for a bank's customers. In the metaphor we talk about customers, accounts and other "things" that have conceptual reality to the customer. However at the system architecture level, there is no real "account" but rather just a record in a data base, but this architecture allows us to support the metaphor. For example, we may now have a requirement to allow a customer to list all transactions on their account. By keeping true to the account metaphor, this extension of functionality is easier to implement than if we started at the database view.

## **First Iteration**

Michael Hill calls the first iteration a ZFR ("ziffer," Zero Feature Release). The first iteration does nothing but puts an initial architecture (often called the architecture spike) into place. The first iteration is guided by a selection of user stories that have architectural significance meaning they force the design of the whole system to be taken into consideration. This is the working skeleton since the first release has no functionality. The focus is making sure that the solution can actually be installed and is configurable.

The idea of a first iteration is well known to every programmer. If you are starting to work in a new programming environment, the very first program you are likely to write is the "Hello World" program. It does nothing, but it usually does force you to make sure

the environment is set up correctly, the compilation and link tools work, and all of the other configuration issues have been resolved.

The systems architecture defines a proposed form and structure of the system; and the design process involves fitting functionality and structural components being developed into that architecture. In the XP approach the first iteration defines the starting form of the system, and will be modified as the system evolves. The first iteration is used also to identify risk as we walk through the architecture.

### **Small Releases and Refactoring**

By creating a constant stream of small releases, we are able to monitor the impact of design changes and additions as we add the implementation of the user stories. Each release is a checkpoint: new functionality may break our existing design so we have to go back and make modifications to the design to accommodate the new requirements.

Remember that a spike is a throw-away solution to see how to solve particular problem – spikes help us choose low-risk ways to extend or modify the architecture and design. Refactoring allows us to change the design as we move forward. As we add new functionality, we may have to change the design of parts of the system without changing their functionality. As we will see, constant refactoring allows us to keep the design fluid and flexible while not giving up robustness.

Each pair of programmers is responsible for integrating their code into the code base at opportune moments, for example, when the unit tests all run at 100% or another chunk of the planned functionality is finished. Only one pair integrates at any given time.



## 11.4 Process Maturity

As the very first module suggested, we can only be as sophisticated in our testing as the developers are in their development process. The Software Engineering Institute (SEI) really kick started the process maturity movement in software with the publication of the Capability Maturity Model or CMM. We are not going to deal with the CMM except in a very superficial way as a lead in to the Testing Maturity Model (TMM) which we will take up in a following section.

Unfortunately, the concept of process maturity has been taken over to a degree by fad gurus who sell it as a magic formula or silver bullet that will fix whatever software ills be-devil your organization, which unfortunately has detracted from the incredible importance that process maturity has in making the engineering process work effectively – not just for software development but in any sort of development.

### Crisis? What crisis?

In the 1980s, the US General Accounting Office study on large government projects found that, for contracted software:

- 2 % was used as delivered
- 3 % was used after changes
- 19 % was used but extensively reworked or abandoned
- 29 % was paid for but not delivered
- 45 % was delivered but not successfully used

The US Department of Defence realized that this poor quality of the software components of its systems was impacting the development of US military systems. They understood the need to correct this situation and “to advance the practice of software engineering because quality software that is produced on schedule and within budget is a critical component of U.S. defence systems”

### ***11.4.1 The Software Engineering Institute***

From the SEI Website [www.sei.cmu.edu](http://www.sei.cmu.edu)

The Software Engineering Institute (SEI) is a US federally funded research and development center sponsored by the U.S. Department of Defense through the Office of the Under Secretary of Defense for Acquisition, Technology, and Logistics [OUSD (AT&L)] The SEI contract was competitively awarded to Carnegie Mellon University in December 1984.

The SEI is a preeminent software engineering R&D technology center. The SEI provides the technical leadership to advance the practice of software engineering so the DoD can acquire and sustain its software-intensive systems with predictable and improved cost, schedule, and quality.

The SEI mission includes four objectives:

1. accelerate the introduction and widespread use of high-payoff software engineering practices and technology by identifying, evaluating, and maturing promising or underused technology and practices
2. maintain a long-term competency in software engineering and technology transition
3. enable industry and government organizations to make measured improvements in their software engineering practices by working with them directly
4. Foster the adoption and sustained use of standards of excellence for software engineering practice

### The Two Part Solution.

The research carried out by the SEI pointed to the fact that organizations that produced quality software were consistently characterized by two factors:

1. They have an explicit and well defined software development process; and
2. They always followed the process. The more they stuck to the process, the better they were at producing software.

The SEI defines a software development process as:

*The means by which people, procedures, methods, equipment, and tools are integrated to produce a desired end result.*

### Defining Maturity

The second finding should not have been as surprising as it was to the software development community. Just having a process was not enough, it had to be part of the organization's culture so that it became "The way we do things around here." In other words, merely defining a process or using it intermittently, even if it is an Agile process, does not produce results – it is the commitment to using the process consistently that produces results. When an organization adopts a way of doing things as the norm (in the sense that deviations from this norm are sanctioned in some way) we say that the process has become "enculturated" or has become part of the corporate culture.

Process maturity can be thought of as always following the process, and when a new situation arises that the process does not handle, the the process is modified to that it can handle that situation the next time it is encountered. In a mature organization, the



process is not fixed or static or followed blindly, instead it is constantly evolving as the organization adapts to changing internal and external factors.

Process maturity is process agnostic. Any process, as long as it is appropriate for the organization and its needs, can be used to produce high quality software provided the organization follows it. This came as a bit of a blow to those organization that kept switching processes looking for the right process that would solve their problems; the SEI data demonstrated that it is not which process you adopt that is the critical success factor but rather it is using the process and sticking to it that makes the difference.

Of course, looking at other industries, we see process maturity as being the factor that often separates the excellent from the so-so organizations, whether it is in construction, medicine or government.

Consider that we learn from experience as individuals; when we encounter a new challenge, we figure out how to meet it and then the next time we come across that challenge, our experience tells us how to deal with it, we don't have to re-invent the wheel every time we encounter that situation.

In the same way, a mature process provides the same capability for an organization to learn from its experience; the challenges that arise that are outside what the process can handle are the stimuli for improving the process so that it can handle those challenges the next time the organization encounters them.

### ***11.4.2 Mature versus Immature Organizations***

The SEI describes the difference between mature and immature organization as follows.

#### **Immature Organizations**

1. The process is often improvised by the developers and management during the course of the project.
2. Defined processes are often not followed or enforced.
3. Project activities are reactionary and often focus on solving immediate crises.
4. Schedules and budgets are based on unrealistic estimates and routinely exceeded.
5. Bringing a project in line with deadlines or costs causes functionality and quality of the product to suffer.
6. Product quality is difficult to predict and there are no objective standards to assess product quality.
7. There are no standard ways for solving problems, everything is done on an “fire-fighting” basis and the same problems keep recurring with the same negative impacts.

8. The defined process is not realistic and the organization is characterized by the attitude "Following the process won't get the job done."
9. The defined process is often created in part by those who are not part of the development staff and tends to be more about reporting and management activities than work tasks.

### **Mature Organizations**

1. The software process is standardized across the organization and communicated to all existing staff and new employees.
2. All work activities are carried out according to the planned process.
3. The processes are usable and realistic because they describe how work actually gets done.
4. Defined processes are updated as necessary.
5. Improvements to the process are planned and evaluated through pilot tests or cost-benefit analyses.
6. Roles and responsibilities are defined clearly in both the organization and in each project.
7. Management continuously monitors the quality of the products and the processes using objective and quantifiable criteria.
8. There are defined processes for analyzing and solving problems with the product and process.
9. Budgets and schedules are realistic because they are based on historical performance data.
10. The infrastructure of the organization supports the process.
11. A disciplined process is consistently followed because all of the participants understand the value of doing so and enforce following the process at a peer level.

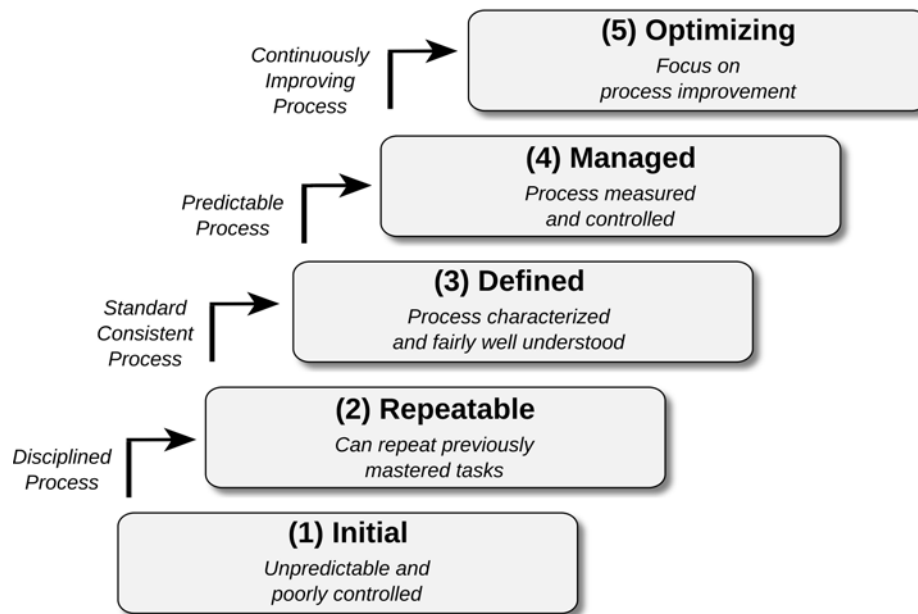
### ***11.4.3 The Capability Maturity Model***

The Capability Maturity Model (CMM) has been one of the most well known results of the SEI process maturity studies and has led to a slew of different maturity models. The maturity level concept has been applied to other areas: People CMM, Software Acquisition CMM, Software Engineering CMM, and Integrated Product Development CMM for example. However, in this course we are interested more in the essence of the maturity models and leave exploring the details of specific formal maturity models as a self directed activity.



## Maturity Levels

As an organization matures, it passes through five different maturity levels. There is more formal definition involving a set of process goals and other metrics, but this is really beyond the scope of this course. Instead we want get the fundamental essence of each level. The levels are evolutionary -- organizations cannot “skip levels” because the levels are not just based on the software development process, but describe the underlying organizational culture, its commitment to their process, and its overall approach to quality and continuous improvement.



The SEI describes the levels this way:

1. The Initial Level: Also referred to as the tribal level. The software process is characterized as ad hoc, and occasionally even chaotic. Few processes are defined, and success depends on individual effort and heroics. There is a reliance on “gurus” or key people who know the product and/or process as opposed to documentation or any form of corporate memory. There is little or no learning from past experience.
2. The Repeatable Level: Basic project management processes are established to track cost, schedule, and functionality. The necessary process discipline is in place to repeat earlier successes on projects with similar applications.
3. The Defined Level: The software process for both management and engineering activities is documented, standardized, and integrated into a standard software process for the organization. All projects use an approved, tailored version of the organization's standard software process for developing and maintaining software.

4. The Managed Level: Detailed measures of the software process and product quality are collected so that the process and products are quantitatively understood and controlled.
5. The Optimizing Level: Continuous process improvement is enabled by quantitative feedback from the process and from piloting innovative ideas and technologies.

Think of the stages of growth of a person. While a person is at the child stage, they are taken care of by a parent, are not expected to have a job nor make important decisions themselves. This stage of maturity is entirely appropriate for a six year old. Where it becomes dysfunctional is when a person is still at this level of maturity when they are 25 years old.

Similarly, all organizations start at level one as start-up. But as they grow, they have to mature or else they tend to not be able to meet the challenges of an established business. A lot of business fail because they experience a lot of growth, but the company is still be managed as a start up – usually because the founder insists on still running the company that way. The founder can make all the decisions when there are three employees at the beginning but not when there are 3,000 people on staff.

I find that it is easier to conceptualize the levels like this:

### **Level 1: No Process Level**

In a lot of the literature, this is often made to sounds like some dystopian and chaotic free for all. However when you take a look at a lot of organizations at this level we can separate out two basic types.

The first is the start up where the emphasis is on the personal skills and productivity of a few key people. It's often counterproductive to have a software development process when there is only two people; for example, the programmer and someone running the business side. In fact, it is probably counterproductive to have a formally defined process while the organization is still trying to figure out what it's doing.

The second is the organization that has grown past the start-up stage and is still hasn't made the transition to process based production. The organization is still being run as a start-up with reliance on key individuals ("We can't let Chani go on vacation because she is the only one who knows how our database is designed.") Eventually, these organizations either have to mature or they tend to fail.

### **Level 2: Repeatable Results**

What characterizes organizations at this level is that every project is on time, on budget and to spec. Projects are not late, over budget or defective in any way. The reason is that the organization has mastered all of the development processes needed to replicate success on every project from project management to vendor management to software testing.



This is also the level that is hardest for most companies to reach since it requires a major reworking of the corporate culture. As one of my clients pointed out “It took us 10 years to get to CMM level 5, but 8 of those years were spent getting to level 2.” This is a fairly typical – most organizations spend most of their time on their way to level 5 trying to get to level 2.

### **Level 3: Standardized Process**

This level is characterized by a standardized process that has various implementations, however what really defines a level 3 maturity is that the process is enculturated. What that means is that the process is seen as “the way we do things around here” and is enforced by the people in the organization, not by management. In a level three organization, if you are not following the process, then it is your peers that are likely to call you out on it.

The result of having a standardized process is that the organization is able to start collecting data on their process and can start identifying what works and what doesn't. And having a standardized process (not just defined but actually in day to day use by everyone) allows QA to actually be done effectively.

If you think of the organization like a basketball team, level one is a bunch of guys showing up at the court and just throwing the ball at the hoop now and then. To get to level two, each individual player has to master the skills of dribbling, passing, shooting, defending as well as learn the rules of the game. At level three, we can take those five level two players and integrate them into a team.

### **Level Four: Proactive Quality**

Once an organization has been a level 3 for a while, it can start to prevent defects from occurring by developing best practices and establishing a review culture. For example if we know from experience that certain design decisions result in certain kinds of errors, we establish a best practice that says something like “Don't do x because these bad results will happen, instead do y because that will work.” These best practices are based on the organization's past experiences and what they have learned from previous projects.

The reviews follow naturally from having these standards or best practices. When work is done, it is reviewed to ensure it has followed the best practices or standards of the organization. However, these best practices and standards themselves are also constantly evolving to keep the corporate knowledge base of best practices current.

## Level Five: Optimization

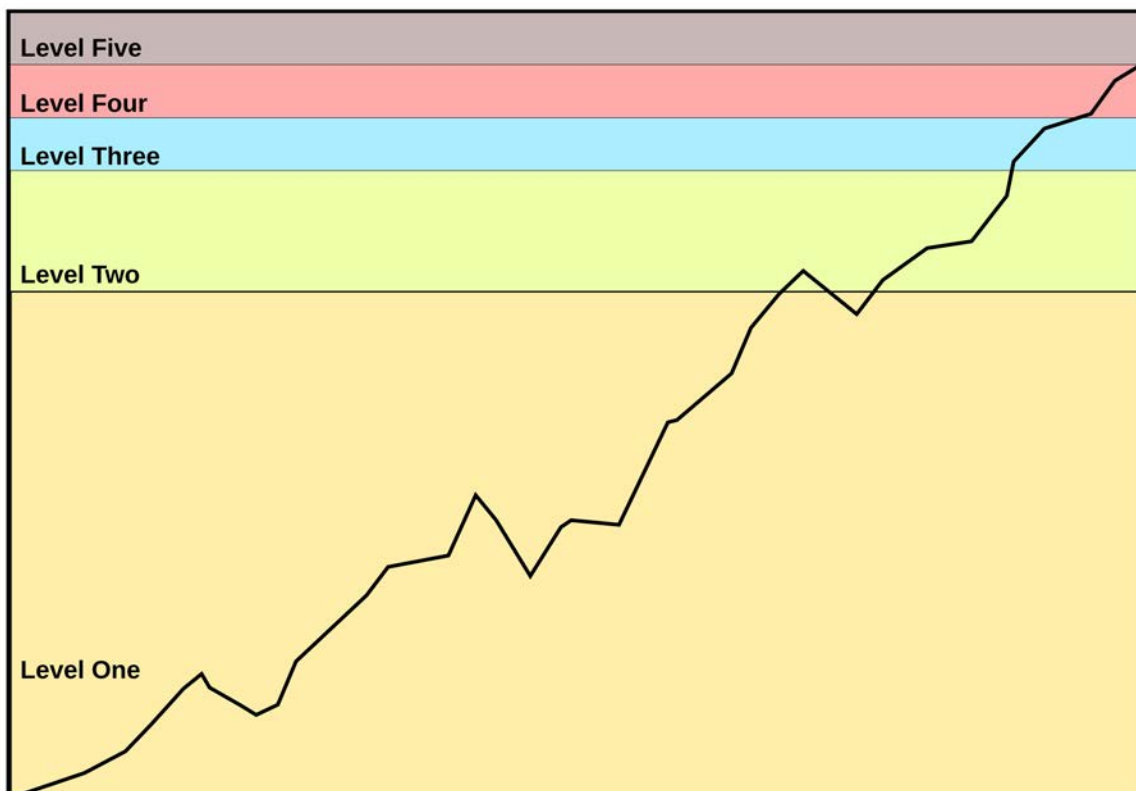
At level 4, work is done then reviewed – at level 5, the best practices and reviews are integrated into the work itself. Level 5 can be thought of as work being reviewed and tested as it is being done. An simple example of this is a spell checker. Level 4 spell checking is analogous to running a spell checker on your document while level five is like having the spell checker identify misspellings as you type and offer on the fly suggestions or automatically correct common typos.

### 11.4.4 Observations on CMM

While the material so far presented is from the academic point of view, one of the aspects of CMM I have observed is that it works because at some point the people in the organization embrace it and refuse to go back to the old way of doing things.

One aspect of the move towards CMM that is not apparent from the nice even presentation of levels is that it is a long and difficult process, and that getting to level two represents a major accomplishment for many organizations.

In fact, a typical path might look like diagram below where CMM level is on the vertical axis and time is on the horizontal axis.



## Getting to Level Two

This is often the hardest step for most organizations. To be a level two organization, they have to master all the aspects of software development processes and the supporting disciplines like software testing, configuration management, contract management and project management. At level two, organizations run projects very well and are consistently delivering on time, on budget and to spec.

At level two, we have a professional organization that has good process practices, the only problem is that they may not be consistent throughout the organization. Getting to level two is a major accomplishment. Why does it seem so hard to get to level 2? The fundamental reason is that there are a lot of basics to master.

## Getting to Level Three

Once the organization has mastered all of the skills necessary for good process management, it can start to define a corporate wide process. Usually this does not go well initially because there is a tendency for groups within the organization to revert to what works – usually their own individual processes because those high quality level two processes get the job done efficiently and effectively for their group.

However, as the new process is used, even though it hurts initially, the organization starts to learn from past mistakes and is able to improve the process until it starts to return clear benefits to the people in the organization. When this happens, the people start to use the process, embrace the process, and make the process the way things get done. The process becomes part of the corporate culture.

Once this happens, moving to levels four and five are merely a matter of time.

## Anecdotal Observations

I had the following conversation, or fairly close to this, with a twenty something software developer in a CMM level five organization in Palo Alto CA.

Me: What's it like working in a level five organization?

*Developer: It is onerous dude. Like, if I want to make a code change there are all these steps I have to go through. First I have to justify why I want to make the change, then I have to do an impact analysis on what the change will do, and if that gets approved, then I have to do a code proposal, and if that gets past a review then we do a code walkthrough, write the test cases and then, and only like a third of the time we get this far, I get to write the code.*

Me: And that is onerous?

*Developer: Dude, when I did my PhD at Stanford, I would just like wire myself up on Red Bull and let the code flow in a three day coding binge. But this is so onerous I quit.*

Me: You mean you just quit now?

*Developer: No, I quit last year and went to work for a start-up.*

Me: But you're here. Now.

*Developer: Yeah, here I work nine to five from Monday to Friday. At the start up I was pulling 80 hour weeks and spending my weekends debugging and handling all sorts of crisis crap.*

Me: So why is this better? I though you would like the excitement of a startup.

*Developer: Hey. I'm 25 and single in California. I don't want to spend my weekends writing code when there are so many beautiful women out there on a Saturday night looking for guys as cool as me. I'll take onerous for that trade off.*

The other employees commented that while they didn't share his specific reason for working in a level five company, they did agree in principle that being in a level five organization meant that they didn't fight fires, manage crises, work over time, get emergency calls or work on trying to fix bugs. As one person put it, "when I go home, I go home. As long as everyone follows the process, my job is under control, but woe be to the person who improvises and makes my life stressful as a result."

### Going the other way...

The five CMM levels can be thought of as describing normal organization. It has been suggested by some analysts that the CMM level scale can actually extend in the negative direction. This work was first proposed by Finkelstein as an immaturity model that was appropriate for dysfunctional organization or companies in crisis.

This work was extended by Capt. Tom Schorsch of the US Air Force in his CIMM (The Capability Im-Maturity Model) who suggest that a number of the companies identified as being at level one are in fact even worse since there was no measure of dysfunctional processes in the original assessment. He also points out that level one assumes the organization wants to get better, but he proposed the negative levels for organizations that just don't care about improving.

Schorsch suggest that there are four levels below level one which can be described as:

#### **0. Negligent: Indifference**

Failure to allow successful development process to succeed. All problems are perceived to be technical problems. Managerial and quality assurance activities are deemed to be overhead and superfluous to the task of software development process. Reliance on silver bullets.



**-1. *Obstructive: Counter Productive***

Counterproductive processes are imposed. Processes are rigidly defined and adherence to the form is stressed. Ritualistic ceremonies abound. Collective management precludes assigning responsibility. Status quo über alles.

**-2. *Contemptuous: Arrogance***

Disregard for good software engineering institutionalized. Complete schism between software development activities and software process improvement activities. Complete lack of a training program.

**-3. *Undermining: Sabotage***

Total neglect of own charter, conscious discrediting of peer organizations software process improvement efforts. Rewarding failure and poor performance.

## 11.5 Testing Maturity Model

In the previous module we looked at the CMM to lead us into this discussion of what is called the Testing Maturity Model (TMM) which was developed by Ilene Burnstein, Taratip Suwannasart, and C.R. Carlson, at the Illinois Institute of Technology.

Their reason for doing this was:

*Testing is a critical component of a mature software development process. It is one of the most challenging and costly process activities, and in its fullest definition provides strong support for the production of quality software. In spite of its vital role in software development, existing maturity models have not adequately addressed testing issues nor has the nature of a mature testing process been well defined.*

*We are developing a Testing Maturity Model to address deficiencies these areas. The TMM will complement the Software Engineering Institute's Capability Maturity Model (CMM) by specifically addressing issues important to test managers, test specialists, and software quality assurance staff. The TMM will contain a set of maturity levels through which an organization can progress toward testing process maturity, a set of recommended practices at each level of maturity, and an assessment model that will allow organizations to evaluate and improve their testing process. [Burnstein et al]*

The TMM picks up where the CMM leaves off. The CMM takes as its primary focus the activities and capabilities needed to create software and, as a result, tends to gloss over some the details of software testing. To compensate, Burnstein and her colleagues have developed a complementary testing maturity model to “fill in the gaps” when it comes to testing within the CMM framework.

The objectives of Burnstein for a TMM were to define:

1. A set of levels that defines a testing maturity hierarchy. Each level represents a stage in the evolution to a mature testing process. Movement to an upper level implies that lower level practices continue to be in place.
2. A set of maturity goals for each level (except Level 1), and the activities, tasks, and responsibilities needed to support them. Organizations will strive toward testing maturity by focusing on the goals defined for each level.
3. An assessment model that consists of three components: a set of maturity goal-related questions designed to assess test process maturity, a training program designed to select and instruct the evaluation team that is to conduct the maturity assessment, and an assessment method that allows an organization to assess itself based on responses to the questionnaire and interview data.

### 11.5.1 Defining Testing Process Maturity

The attributes of a mature testing process are described below, each supporting one or more of the process maturity criteria described in the CMM. A mature testing process has the following:



1. A set of defined testing policies. There is a set of well-defined and documented testing policies that is applied throughout the organization. The testing policies are supported by upper management, institutionalized, and integrated into the organizational culture.
2. A test planning process. There is a well-defined and documented test planning process used throughout the organization that allows for the specification of test objectives and goals, test resource allocation, test designs, test cases, test schedules, test costs, and test tasks. Test plans reflect the risks of failures; time and resources are allocated accordingly.
3. A test life cycle. The test process is broad-based and includes activities in addition to execution-based testing. There is a well-defined test life cycle with a set of phases and activities that is integrated into the software life cycle. The test life cycle encompasses all broad-based testing activities; for example, test planning, test plan reviews, test design, implementation of test-related software, and maintenance of test work products. It is applied to all projects.
4. A test group. There is an independent testing group. The position of tester is defined and supported by upper management. Instruction and training opportunities exist to educate and motivate the test staff.
5. A test process improvement group. There is a group devoted to test process improvement. They can be a part of a general process improvement group, a software quality assurance group, or a component of the test group. Since the test process is well defined and measured, the test improvement group can exert leadership to fine-tune the process, apply incremental improvement techniques, and evaluate their impact.
6. A set of test-related metrics. The organization has a measurement program. A set of test-related metrics is defined, data is collected and analyzed with automated support. The metrics are used to support the appropriate actions needed for test process improvement.
7. Tools and equipment. Appropriate tools are available to assist the testing group with testing tasks and to collect and analyze test-related data. The test process improvement group provides the leadership to evaluate potential tools and oversees the technology transfer issues associated with integrating the tools into the organizational environment.
8. Controlling and tracking. The test process is monitored and controlled by the test managers to track progress, take actions when problems occur, and evaluate performance and capability. Quantitative techniques are used to analyze the testing process and to determine test process capability and effectiveness.
9. Product quality control. Statistical methods are used for testing to meet quality standards. "Stop testing" criteria are quantitative. Product quality is monitored, defects are tracked, and causal analysis is applied for defect prevention.

### ***11.5.2 The TMM levels***

Just like in CMM, each level of the TMM represents the overall “capability” of the organization. The following are Burnstein's description of the TMM levels.

### **Level 1 – Initial:**

Testing is a chaotic process; it is ill-defined and not distinguished from debugging. Tests are developed in an ad hoc way after coding is done. Testing and debugging are interleaved to get the bugs out of the software. The objective of testing is to show that the software works. Software products are released without quality assurance. There is a lack of resources, tools, and properly trained staff. This type of organization would be on Level 1 of the Capability Maturity Model (CMM) developed by the Software Engineering Institute. There are no maturity goals at this level.

### **Level 2 – Phase Definition:**

Testing is separated from debugging and is defined as a phase that follows coding. It is a planned activity; however, test planning at Level 2 may occur after coding for reasons related to the immaturity of the test process. For example, at Level 2 there is the perception that all testing is execution-based and dependent on the code, and therefore it should be planned only when the code is complete.

The primary goal of testing at this level of maturity is to show that the software meets its specifications. Basic testing techniques and methods are in place. Many quality problems at this TMM level occur because test planning occurs late in the software lifecycle. In addition, defects propagate into the code from the requirements and design phases, as there are no review programs that address this important issue. Post-code, execution-based testing is still considered the primary testing activity.

### **Level 3 – Integration:**

Testing is no longer a phase that follows coding; it is integrated into the entire software life cycle. Organizations can build on the test planning skills they have acquired at Level 2. Unlike Level 2, planning for testing at TMM Level 3 begins at the requirements phase and continues throughout the life cycle. Test objectives are established with respect to the requirements based on user and client needs and are used for test case design and success criteria. There is a test organization, and testing is recognized as a professional activity. There is a technical training organization with a testing focus. Basic tools support key testing activities. Although organizations at this level begin to realize the important role of reviews in quality control, there is no formal review program, and reviews do not yet take place across the life cycle. A test measurement program has not yet been established to qualify process and product attributes.

### **Level 4 – Management and Measurement:**



Testing is a measured and quantified process. Reviews at all phases of the development process are now recognized as testing and quality control activities. Software products are tested for quality attributes such as reliability, usability, and maintainability. Test cases from all projects are collected and recorded in a test case database to test case reuse and regression testing. Defects are logged and given a severity level. Deficiencies in the test process are now often due to the lack of a defect prevention philosophy and the porosity of automated support for the collection, analysis, and dissemination of test-related metrics.

### **Level 5 – Optimization, Defect Prevention, and Quality Control:**

Because of the infrastructure provided by the attainment of maturity goals at Levels 1 through 4 of the TMM, the testing process is now said to be defined and managed its cost and effectiveness can be monitored. At Level 5, there are mechanisms that fine-tune and continuously improve testing. Defect prevention and quality control are practiced. The testing process is driven by statistical sampling, measurements of confidence levels, trustworthiness, and reliability. There is an established procedure to select and evaluate testing tools. Automated tools totally support the running and rerunning of test cases, providing support for test case design, maintenance of test-related items, defect collection and analysis, and the collection, analysis, and application of test-related metrics.

### ***11.5.3 Maturity Goals at the TMM Levels***

The operational framework of the TMM provides a sequence of hierarchical levels that contain the maturity goals, subgoals, activities and tasks, and responsibilities that define the testing capabilities of an organization at a particular level. They identify the areas where an organization must focus to improve its testing process.

This section describes the maturity goals for all levels except Level 1, which has no maturity goals.

### **Level 2 – Phase Definition**

At Level 2 of the TMM, an organization has defined a testing phase in the software life cycle that follows coding. It is planned and repeatable over all software projects. It is separated from debugging, which is an unplanned activity. The maturity goals at Level 2 follow.

1. Develop Testing and Debugging Goals

This means an organization must clearly distinguish between the processes of testing and debugging. The goals, tasks, activities, and tools for each must be identified. Responsibilities for each must be assigned. Management must develop plans and policies to accommodate and institutionalize both processes. The separation of these two processes is essential for testing maturity growth, since they are different in goals, methods, and psychology. Testing at this level is now a planned activity and therefore can be managed, whereas debugging cannot.

### 2. Initiate a Test Planning Process

Planning is essential for a process that is to be repeatable, defined, and managed. Test planning involves stating objectives, analyzing risks, outlining strategies, and developing test design specifications and test cases. In addition, the test plan must address the allocation of resources and the responsibilities for testing on the unit, integration, system, and acceptance levels.

### 3 Institutionalize Basic Testing Techniques and Methods

To improve test process capability, basic testing techniques and methods must be applied across the organization. How and when these techniques and methods are to be applied and any basic tool support for them should be clearly specified. Examples of basic techniques and methods are black-box and white-box testing strategies, use of a requirements validation matrix, and the division of execution-based testing into sub-phases such as unit, integration, system, and acceptance testing.

## Level 3 – Integration

This critical maturity level is essential for building quality into software products early in the software life cycle.

At this level, the testing phase is no longer just a phase that follows coding. Instead, it is expanded into a set of well-defined activities that are integrated into the software life cycle. All of the life cycle phases have testing activities associated with them. Integration support associates testing activities with life cycle phases, such as requirements and design.

At this level, management supports the formation and training of a software test group specialists responsible for testing. This group also acts as a liaison with the users or clients, ensuring their participation in the testing process.

Basic testing tools now support institutionalized test techniques and methods. Technical and managerial staff are beginning to realize the value of review activities as a tool for defect detection and quality assurance.



The maturity goals at Level 3 follow.

1. Establish a Software Test Organization

A software test organization is created to identify a group of people who are responsible for testing. Because testing in its fullest sense has a great influence on product quality, and because testing consists of complex activities usually performed under tight schedules and high pressure, management realizes it is necessary to have a well-trained and dedicated group of specialists in charge of this process. The test group is responsible for test planning, test execution and recording, test-related standards, test metrics, the test database, test reuse, test tracking, and evaluation.

2. Establish a Technical Training Program

A technical training program ensures a skilled staff is available to the testing group. Testers must be properly trained so they can perform their jobs efficiently and effectively. At Level 3 of the TMM, the staff is trained in test planning, testing methods, standards, techniques, and tools. At the higher levels of the TMM they learn how to define, collect, analyze, and apply test-related metrics. The training program also prepares the staff for the review process, instructing review leaders and instituting channels for user participation in the testing and review processes. Training includes in-house courses, self-study, mentoring programs, and support for attendance at academic institutions.

3. Integrate Testing into the Software Life cycle

Management and technical staff now recognize that test process maturity and software product quality require that testing activities be conducted in parallel with all life cycle phases. Test planning is now initiated early in the life cycle. A variation of the V-model is used by the testers and developers. User input to the testing process is solicited through established channels for several of the testing phases.

4. Control and Monitor the Testing Process

According to R. Thayer, management consists of five principal activities: planning, directing, staffing, controlling, and organizing. Level 2 of the TMM introduces planning capability to the testing process. In addition to staffing, directing, and organizing capabilities, Level 3 introduces several controlling and monitoring activities. These provide visibility and ensure the testing process proceeds according to plan.

When actual activities deviate from test plans, management can take effective action to correct the deviations and return to test plan goals. Test progress is determined by comparing the actual test work products, test effort, costs, and schedule to the test plan.

Support for controlling and monitoring comes from standards for test products, test milestones, test logs, test-related contingency plans, and test metrics that can be used to evaluate test progress and test effectiveness.

## **Level 4 – Management and Measurement**

The principal focus areas at this level are to broaden the definition of a "testing activity" and to accurately measure the testing process.

Controlling and monitoring functions can now be fully supported by an established test measurement program. Staffing activities are supported by a training program. The definition of a testing activity is expanded to include reviews, inspections and walk-throughs at all phases of the life cycle.

Software work products as well as test-related work products such as test plans, test designs, and test procedures are all reviewed. This expanded definition of testing covers activities typically categorized as verification and validation activities.

The primary goal of this broadened set of testing operations is to uncover defects occurring in all phases of the life cycle and to uncover them as early as possible. Review results and defect data are saved as a part of project history. Test cases and procedures are stored for reuse and regression testing.

The maturity goals at Level 4 follow.

1. Establish an Organization-Wide Review Program

At TMM Level 3, an organization integrates testing activities into the software lifecycle. At Level 4, this integration is augmented by the establishment of a review program. Reviews are conducted at all phases of the lifecycle to identify, catalog, and remove defects from software work products and to test work products early and effectively.

2. Establish a Test Measurement Program

A test measurement program is essential in the evaluation of the quality of the testing process, to assess the productivity of the testing personnel and the effectiveness of the testing process, and for test process improvement. Test measurements are vital in the monitoring and controlling of the testing process. A test measurement program must be carefully planned and managed. Test data to be collected must be identified; how they are to be used and by whom must be decided. Measurement data for every test lifecycle phase must be specified. Measurements include those related to test progress, test costs, data on errors and defects, and product measures such as software reliability.

3. Software Quality Evaluation

One purpose for software quality evaluation at this TMM level is to relate software quality issues to the adequacy of the testing process. Software quality evaluation involves defining measurable quality attributes and defining quality goals to evaluate software work products. Quality goals are tied to testing process adequacy because a mature testing process must lead to software that is at least correct, reliable, usable, maintainable, portable, and secure.

## Level 5 – Optimization, Defect Prevention, and Quality Control

Several test-related objectives are at the highest level of the TMM. Organizations at this level test to ensure the software satisfies its specification and is reliable and that the organization can establish a certain level of confidence in its reliability. There are tests to detect and prevent faults. Prevention applies to requirements, design, and implementa-



tion faults.

Because the testing process is now repeatable, defined, managed, and measured, it can be fine-tuned and continuously improved. Tool support is available to collect and analyze test-related data. Test planning and test execution also have tool support. Through achievement of the maturity goals at TMM Levels 1 through 4, the testing process is now planned, organized, staffed, controlled, and directed.

Management provides leadership and motivation and supports the infrastructure necessary for the continual improvement of product and process quality. Test-related measurements help suggest and evaluate test process improvement procedures, methods, tools, and activities. Changes can be monitored and managed.

The maturity goals at Level 5 follow.

1. Application of Process Data for Defect Prevention

Mature organizations are able to learn from their history. Following this philosophy, organizations at the highest level of the TMM record defects, analyze defect patterns, and identify root causes of errors. Techniques such as Pareto diagrams are used [6]. Action plans are developed, actions are taken to prevent defect recurrence, and there is a mechanism to track action progress. At TMM Level 5, defect prevention is applied across all projects and across the organization. A defect prevention team is responsible for defect prevention activities, interacting with developers to apply defect prevention activities throughout the life cycle.

2. Quality Control

At TMM Level 4, organizations focus on testing for a group of quality-related attributes such as correctness, security, portability, interoperability, usability, and maintainability. At TMM Level 5, organizations use statistical sampling, measurements of confidence levels, trustworthiness, and reliability goals to drive the testing process.

The testing group and the software quality assurance (SQA) group consist of quality leaders who work with software designers and implementors to incorporate techniques and tools that reduce defects and improve software quality.

Automated tools support the running and rerunning of test cases and defect collection and analysis. Usage modeling is used to perform statistical testing. The cost to achieve quality goals is measured relative to the cost of not testing for quantitative quality goals.

3. Test Process Optimization

At the highest level of the TMM, the testing process is subject to continuous improvement across projects and across the organization. The test process is quantified and can be fine-tuned so that capability growth is an ongoing process. An organizational infrastructure exists to support this continual growth. This infrastructure, which consists of policies, standards, training, facilities, tools, and organizational structures, has been put in place through the goal achievement processes that constitute the TMM hierarchy.

Optimizing the testing process involves

1. Identifying testing practices that need to be improved.
2. Implementing the improvements.
3. Tracking improvement progress.
4. Continuous evaluation of new test-related tools and technologies for adaptation.
5. Support for technology transfer.



## Agile Testing

### *Module Twelve*

*Pessimists, we're told, look at a glass containing 50% air and 50% water and see it as half empty. Optimists, in contrast, see it as half full. Engineers, of course, understand the glass is twice as big as it needs to be.*

Bob Lewis

*A fool with a tool is still a fool*

Martin Fowler

*If you automate a mess, you get an automated mess.*

Rod Michael



## 12.1 Defining Agile Testing and Quality

Over the years, I have been quite fortunate to work with some extremely good Agile teams in various industries who use Agile to deliver significant amounts of high quality software while still maintaining an enjoyable, laid back work environment where everyone goes home on time and leaves their work at the office until the next day.

Working with these teams demonstrated to me in a real and tangible way how powerful Agile could be when done right. In fact, most of what I have put into this course is based on seeing how these elite teams implemented Agile concepts to create a workable, productive and human-friendly process. In fact, I have not included a single theoretical point in this course unless I have seen proof that it works out in the real world.

There is one observation that I felt was very telling – the very best agile teams would leave the office half an hour or so before the end of the work day and convene in a pub or similar venue. For the next thirty minutes or so, they would decompress by hashing over the day and making general plans for the next day. When I asked about this, one team leader explained it was a way for his team to bring closure to the day so that they didn't mentally or emotionally take their work home with them. As she said "You can't be at the top of your game every day if you don't have an outside life to return to every night."

Unfortunately, these teams seem to be in the minority, at least in my experience. Many Agile teams I have worked with seem to be under the assumption that Agile is about throwing out everything that isn't specifically Agile and/or working in a disorganized and chaotic manner. For example, one a common bizarre idea I encounter is that software testing doesn't need to be part of an Agile process since the developers will just "test as they go." This rarely ends well in practice.

### 12.1.1 Agile Testing

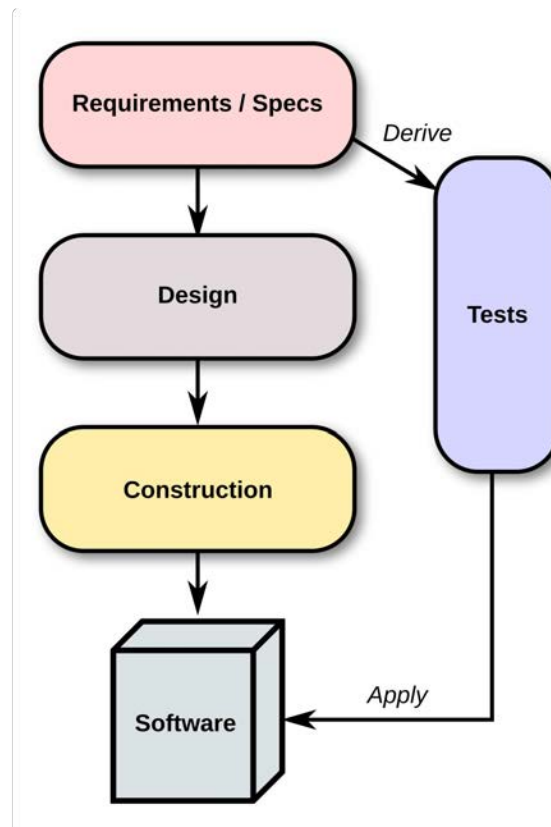
Recall from module one Boris Beizer's Phases in a Tester's Mental Life. When Beizer first published the concept, many people found stages 4 and 5 were confusing or just vague ("What is this state of mind stuff?"). Stage 1 was simple enough to understand – it was the stage in which testing was only done when something broke and we had to debug to find out what went wrong. Stages 2 and 3 were fairly standard, but stages 4 and 5 weren't really that well defined.

In retrospect, we can see that stages 4 and 5 describe what we are now calling Agile testing. I'm using the term "Agile testing" rather than "Agile testing and quality" since at level 4 and 5, the distinction between testing and quality starts to get quite fuzzy.

In the standard V-type diagram which appears in some form or another in most software engineering or testing books, the levels of testing ranked hierarchically based on the artifacts to be tested. The key point is that in traditional level 2 and 3 testing, we are fo-

cused on testing the output of the development process – we are testing “things we make.” Code is produced and compiled into components which we test to ensure it meets the requirements of the detailed design, then the system is built and we test it ensure it passes the system specification and so on.

There is often a disconnect between the testers and developers as they follow two independent work paths during the project: the developers move on from requirements to design and then construction, while the testers focus on how to test the completed software. Of course the accompanying diagrams here are not intended to depict the stages of any real development process, but are only for illustrative purposes.



At levels 2 and 3, we test software. At levels 4 and 5, we not only test the software but we apply quality and testing concepts to the software development process as well. We improve quality not only by testing a component but by identifying where errors might be occurring the the development process and then identifying corrective action to proactively prevent those errors from recurring in the future. At levels 4 and 5, our focus starts to become centred on how produce quality and ensure the development process produces artifacts at each stage of development that embody the required quality, which we then confirm through our testing activities.



At levels 2 and 3, the testers remain apart from the development team and evaluate the software based on tests devised from applying standard best testing practices for developing and executing tests which are derived from the requirements and specifications. But at levels 4 and 5, testers are integrated into the team as a built-in self correcting mechanism within the development process.

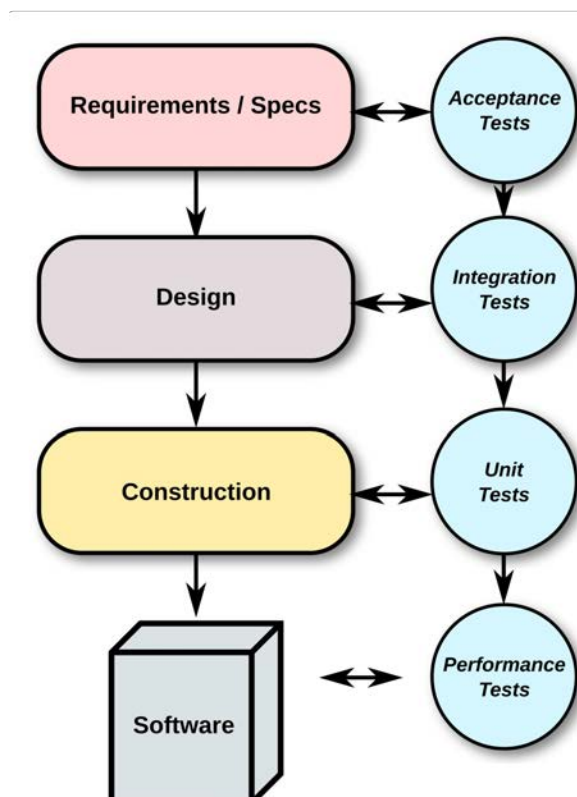
At levels 2 and 3, we use testing to show where development went off track; at levels 4 and 5, we use testing to prevent development from going off track.

### 12.1.2 Traditional Testing versus Agile Testing

There are two primary ways Agile testing is different from traditional software testing:

1. In traditional testing, we are concerned only with the quality of the software. In Agile testing we are also concerned with the quality of the Agile process itself.
2. In traditional testing, we are outside the development process and were only concerned with the resulting work product. In Agile testing, we are integrated into the process and work to make the process better by collaborating with our colleagues throughout the development cycle.

The diagram in the previous section showed the “separation of concerns” that characterized the traditional testing model which stands in contrast to the this diagram of the the Agile testing approach. In the diagram, there is interaction between testers and other team members at each stage of the process.



For example, at the requirements and specification stage, the testers are looking for deficiencies in the requirements that would later produce problems during development. For example, our testing is looking for contradictory requirements, missing cases or ambiguous descriptions of expected system behaviour. The tester collaborates with the business analyst and other team members to improve the overall quality of the requirements while they are being documented and collected. As the developers start their design, there are no “carry over” requirements errors that might lead to faults in the design.

The driving force of Agile testing is to be constantly looking for ways to prevent errors, including supporting processes like acceptance test driven development (ATDD) and test driven development (TDD) that use tests to ensure errors are caught and corrected as they occur, or that work is being done in such a way that the opportunity to make certain kinds of errors is removed.

It should be emphasized though that Agile testing does not replace standard testing – it is an evolution of the traditional role of the tester which elevates standard testing to a new level of performance and importance.

### Agile Testing Responsibilities

The activities of an Agile tester can be grouped into three general categories:

1. Project support activities that focus on the overall quality planning objectives of the Agile project.
2. Process improvement activities that focus on the Agile process itself.
3. Development support activities where testers collaborate on a regular basis with developers and business analysts.

We won't be dealing with point (2) above in any depth in this course, but we will be making reference to those activities wherever they come up during the natural flow of the material. The well known four quadrants of Agile testing are introduced in the latter part of this module which is our starting point for a detailed discussion in the following modules of point (3) above. However, while the quadrant classification of testing activities is useful and accurate, it does not define the whole scope of what an Agile tester does.

In the first part of this module point (1) is explored by looking at the kind of preliminary work at the start of a project that Agile testers do that helps ensure the eventual success of the project. While some of the project activities are done directly by testers, most of the work testers do at the start of the project is validating and verifying project activities that are the responsibility of other team members.

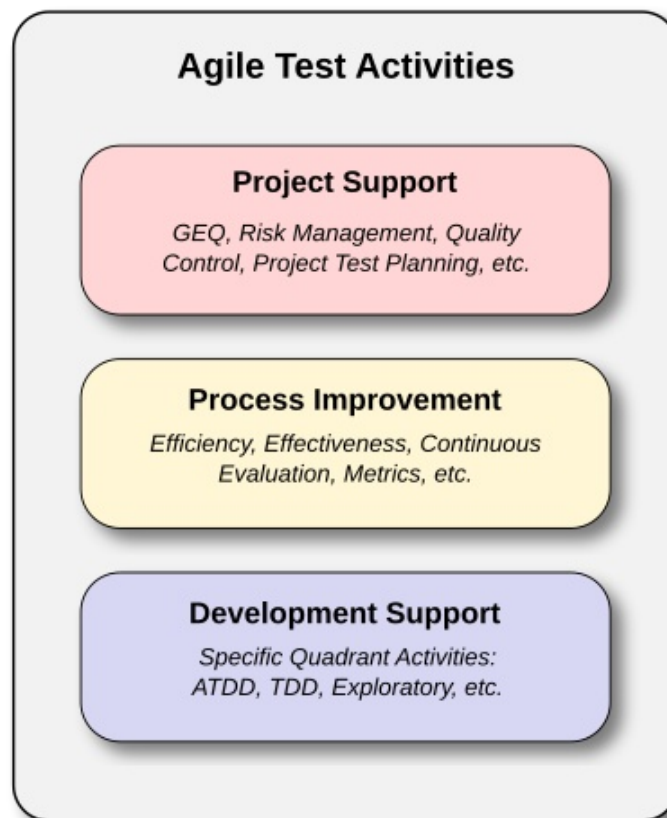
The Chaos report and other data presented in module one showed that one of the best ways to ensure efficiency and effectiveness in our project is to ensure the project is properly set up and avoids the early sort of errors that later come back to sink the development effort. For example, a couple of these errors are failing to identify the scope of



the project or failing to correctly identify the stakeholders for the project.

The quadrant model only describes point (3) above, but this is understandable since the work done in the third category tends to account for the majority of the work that Agile testers do during a project and winds up consuming most of the testers' time and effort. Many Agile testers find making the transition to Agile testing more natural by focusing on the quadrant activities first, then growing into the project support activities and then into the process improvement activities. This is a natural progression as both the project team becomes more mature in how they work and the testers become more mature in their testing activities.

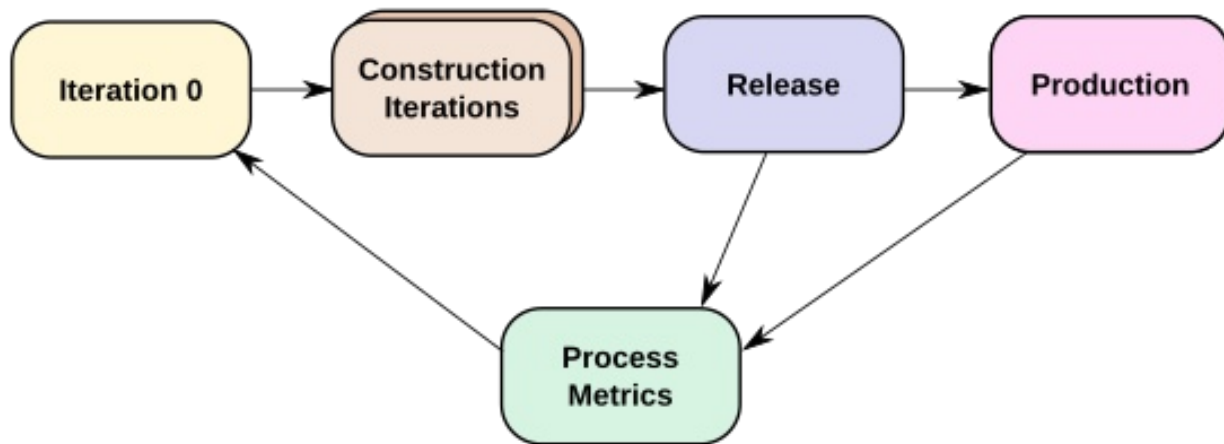
It should be emphasized that the project support activities and process improvement activities are not new kinds of activities but represent the integration of well established quality tasks and activities into the Agile process and an Agile project.



The material in this section draws from a number of sources and represents what appears to be the general body of informal material circulating about Agile testing. I have taken the liberty of adding to or rearranging some of the material where I think it makes it easier to teach. However, there are differences between various authors and practitioners on exactly what Agile testing is which means you may find other materials that differ from what I have included here. Just keep an open mind and take what is useful from the difference sources – exposing yourself to a diversity of ideas in Agile testing can only make you a better Agile tester.

## 12.2 Agile Testing Iterations

The diagram below is fairly standard except for the addition of “Process Metrics.” Almost every Agile process has a similar sort of structure where there is an initial set-up iteration followed by a series of construction iterations which terminate in a release. The type of testing we do is dependent on the iteration type. For example, the four quadrants describe what we do during the construction iterations while the testing we do in the initial iteration is geared more to project support and quality activities.



### 1.2.1 Iteration 0

Iteration 0 is the name often given to the initial planning phase done for any Agile project. This is analogous to the “zipher” iteration of the XP approach we looked at earlier. During this iteration, testers are collaborating with other team members to ensure the project starts up with minimal risk and has the best possible chance for success.

The activities that are normally done in the initial iteration are:

1. Business case development and feasibility studies.
2. Establishing the project scope and a clear statement of success criteria.
3. Identification of key requirements and use cases.
4. Identifying candidate architectures.
5. Development of risk models and performing a preliminary risk analysis.
6. Identification of key stakeholders.
7. Establishment of the “good enough quality” goals.
8. Selection of appropriate testing and development models.



## 9. Development of a project plan with costs and time lines.

These activities are generally project planning support sorts of activities. Most of the activities are collaborative in the sense we are not doing them ourselves but we are working with business analysts, developers and those managing the project who are tasked with the responsibility for these deliverables.

Referring to the idea of ALM from the another module, some of these activities involve both the ALM governance workflow as well as more traditional project specific activities.

### Business Case and Feasibility Studies

This falls partly under the governance workflow of ALM.

*Governance starts with business case development and feasibility studies. Once the development begins, governance is now implemented through project portfolio management where the role of the application in the overall business context and IT architecture is continuously evaluated, often with respect to some organizational wide metrics, standards and valuations. Once an application is deployed, it becomes part of the organization's portfolio of applications, and as such is an organizational asset like any other and is subjected to an ongoing assessment of its associated benefits and costs.*

The text in bold in the preceding quote is specifically of interest to us as testers for two reasons. First, it falls in our area of responsibility to ensure that the evaluation is done in a way that is valid, accurate, reliable and comprehensive; but we also need to measure the evaluation to ensure it "passes" when compared to the organizational standards. The evaluations are not done by the testers, evaluations are done by the appropriate team members with expertise in a specific area; what we are interested in is the quality of the evaluation process. A poorly executed evaluation of the business value of a project could be fatal to the project at some future point.

We won't cover the idea of a business case in this course since this is a fairly standard artifact in most project management methodologies. If you are unfamiliar with the idea of a business case, the following description of a business case from Wikipedia should give you a feel for what it is.

*A business case captures the reasoning for initiating a project or task. It is often presented in a well-structured written document, but may also come in the form of a short verbal agreement or presentation. The logic of the business case is that, whenever resources such as money or effort are consumed, they should be in support of a specific business need. An example could be that a software upgrade might improve system performance, but the "business case" is that better performance would improve customer satisfaction, require less task processing time, or reduce system maintenance costs. A compelling business case adequately captures both the quantifiable and non-quantifiable characteristics of a proposed project.*

While the business case tries to answer the question "Is it worth it for us to do this?" a feasibility study tries to answer the question "Can we do this with the resources available to us?" Again this is part of the responsibilities of governance in ALM and really doesn't affect the project directly but is more of a meta-project activity to decide if the project

should go forward and if so, under what sort of constraints or structures.

Derived from the business case is something often called a project charter which describes in a very succinct manner what the project is tasked to accomplish. This document or statement is also known by a wide range of names like vision statement or project commissioning document.

So why are we involved with business cases and feasibility studies? We don't actually write them but we do provide input on the following topics:

1. Do the feasibility study and business case make realistic assumptions about the level of quality and risk are consistent with our organizational quality management standards? Will this project require an exemption from or a change to those standards?
2. Do the business case and feasibility studies take into adequate account the risk analyses for both product risk and project risk?
3. Can we verify that the projected outcomes of the business case and feasibility study are consistent with historical and comparative data from similar projects?
4. Do the business case and feasibility study deliverables meet the quality standards established in the organization for those artifacts, or are they flawed in some manner.

This is not a complete list but it does give you a sense of what we are doing; ensuring that we are maintaining quality in this governance workflow. If errors are made here; for example, not taking into account a risk factor, then that overlooked factor might show up during the project and have a negative or even disastrous impact. One of my colleagues one described the testing activity here as the tester being the person who keeps saying "Oh, really? Convince me" to everyone else actually doing the work.

### **Project Scope and Success Criteria**

Out the business case should come a project charter which is the first step in defining the scope of the project. This is normally an activity that falls under the responsibility of the BA function. It can be thought of as defining the target or ultimate goal of the project by providing the answer to two critical questions:

1. Why do the stakeholders want a new system?
2. What do the stakeholders expect the new system to do for them?

Answering these questions is not as simple as it seems. For example, project can grind to a halt early on because the various groups of stakeholders cannot agree on why a system is being developed or what it is supposed to do. The project charter should start with a simple, brief statement of the organizational need and what the new system will do to meet the need. Part of what we do is ensure that we this can actually be devel-



oped from the business case because if we can't, then we have to call into question the quality of the business case.

Creating a charter can be a very difficult process because of the different perceptions among the stakeholders of what the real need is and what the new system "should" do. We often work with the business analysts and domain experts here to test the sort of results they are coming up with to ensure they manage to get a project charter that is workable and is signed off by all of the stakeholders.

One of the main testing activities at this point is to conduct reviews to ensure that the project charter meets all of our quality standards – for example we could apply the IEEE standard of correct, complete, consistent, valid and so on.

The project charter should not only describe what the project will do but also what the project will not do. Keeping track of the scope of the project can be difficult, and many projects suffer from "scope creep," which is the slow adding of more and more features to a project during development until the project collapses under its own complexity.

Once we have a project charter that delineates what the scope of the project is, we can create something often called an In/Out list as a tool for making decisions about what work needs to be done. In the example shown, the list is a table with three columns, the left column containing any topic, although usually a requirement or request with the next two columns identifying whether the request is in or out of scope. Being out of scope is often a reason for the stakeholder request to be rejected.

Any topic that is a scope issue is added to the list and then a consensus is reached as to whether it goes into the "in scope" or the "out of scope" column. If there is no consensus, then at least there is a point at which we have documented this as a scope issue and which may require us to take the matter back to governance to see if the business case needs to be reworked.

One of the process improvement aspects of having a well defined scope is that we are less likely to engage in over engineering because each feature being added has already been vetted. We also have a record of requests so that if a request that we have already deemed out of scope is raised later on, we can refer back as to why it was originally declared out of scope.

## Success Criteria

This is an often overlooked part of the project charter. It quantifies the answer to the question "What does the system need to do in order for the project to be considered successful?" It is easy to confuse this with acceptance testing or assume that this falls under acceptance testing. The success criteria differ from acceptance testing in several ways:

1. Success criteria are focused on the general issue of how we can measure if the delivered system has met the underlying business need identified in the business case. Very often we quantify the success criteria in terms of key performance indicators (kpi) from the business perspective. For example “The current case office can handle on average six cases a day, the new system will be considered successful if it allows the average officer to handle on average ten cases a day.”
2. Acceptance testing is usually focused on the product – looking at issues like whether users find the interface easy to use, or whether it supports a work flow to an acceptable level – while success criteria focus on the return on investment made in development.

Success criteria may also reference time and costs which may require delivery by specific deadlines, ensuring there are no cost overruns for example. Success criteria may also specify how much of the functionality must be delivered and how much is optional.

### Identification of Key Requirements and Use Cases

This task is about prioritization. Given that the system is intended to meet a specific need, this analysis asks what key features and use cases would be most likely to meet that need. These are the requirements that drive the choice of candidate architectures and other choices that are made later in designing and implementing the system.

If you recall this diagram from the over engineering section, we want to be able to ask the question what features have the highest value to the stakeholder and therefore provide a higher level of quality to them – recall the discussion on quality attributes.

This decision feeds into the work prioritization or sprint planning to allow the project to deliver the most important features early. This is one of those quality activities that also has a process improvement aspect to it because this makes product development more efficient. Normally we work with the business analysts and stakeholders to ensure correctness on the ranking.

### Identifying Candidate Architectures

This is usually done in collaboration with developers. Early on in the project, there will be a number of major design decisions to be made about what the overall architecture should be. In conjunction with the preceding activity, we often describe this process as being use case driven and architecture centric.

A software architecture is often described along these lines<sup>1</sup>:

*The fundamental organization of the system as a whole. Aspects of an architecture include static elements, dynamic elements, how those elements work together, and the overall architectural style that guides the organization of the system. Architecture also addresses issues such as performance, scalability, reuse, and economic and technological constraints.*



By identifying the key features and use cases, we have selected a high value set of functionality that delivers an acceptable level of quality to the user. We call this being use case driven.

Once those use case have been selected, then we look for an architecture that delivers that core or key functionality in an optimal manner. However, once we have chosen an architecture, then it may preclude us from adding more use cases, in which case we say that future development is now architecture centric.

However, if everyone is doing their job correctly, what we do wind up with is a system that delivers the highest level functionality and also can be developed efficiently in terms of construction.

The reason we work with the candidate architectures and key requirements early on is to ensure both high levels of quality in the delivered product and an optimized development process.

### **Development of risk models and performing a preliminary risk analysis**

We are concerned with two kinds of risk:

1. Product risk: This is the risk associated with the use of a product and is the more traditional form of risk that most people are familiar with.
2. Project risk: This is the risk of the project not being completed successfully, which is of concern to us from our perspective of Agile quality.

### **Identification of Key Stakeholders**

With the emphasis on the system users and use cases, non-users and their requirements are often overlooked. System requirements are not just user requirements because there exist parties, either people or organizations, who are not necessarily users, but have a vested interest in the how a system operates.

Their requirements can drastically alter the design of a system, and overlooking their requirements can result in a failed project or product. We call these invested parties stakeholders.

A stakeholder is anyone (person, organization or agency):

1. Who could be materially affected by the implementation of a new system or application or who has a vested interest in the project; AND
2. Who has the ability, legal, political, economic or otherwise, to force the developers to protect those interests to their satisfaction.

Stakeholders have input into the project because they have the power to affect the project. This definition of a stakeholder is quite pragmatic. There is no rule that says that you cannot take into account the requirements of non-stakeholders – we often do this for business or political reasons or to create goodwill with groups or individuals.

However, if you ignore some stakeholder requirements then you introduce a significant risk element to the project. We cannot totally identify the stakeholders until we have defined what the scope of the project is. We work with the business analysts to ensure that all of the stakeholders are identified and their requirements evaluated.

### **Establishment of the “Good Enough Quality” Goals**

This is fairly critical since we need to understand what the GEQ goals for the project are in order to set the testing parameters for the project – specifically how much testing to do and how detailed and thorough the testing has to be to meet the quality goals.

### **Selection of Testing and Development Models**

In a mature process, we have established certain development models and testing models, although we might think of them as templates, that we can use to bring past experience into the current project.

For example, a common project type in the banking industry is a data migration project where data is migrated from one data model to another. Based on decades of experience, most banks have a standard data migration project template that represents the institutional knowledge and best practices that have been distilled from previous projects.

At this stage of the project, we assist in ensuring that if there is a project model or template that should be used, it is used and any customization that needs to be done are in fact executed.

We also have the same concept with testing models. We may know from experience that if we are doing a data migration kind of project, there are certain key testing and quality tasks that need to be done, for example ensuring the data is not modified when being moved. Over time, we may develop a set of testing templates that are appropriate for certain kinds of projects.

In the Iteration 0 we ask the important question “Have we done this before and can we leverage our past experience in development and testing into this project?”

### **Project Plan with Costs and Time Lines**

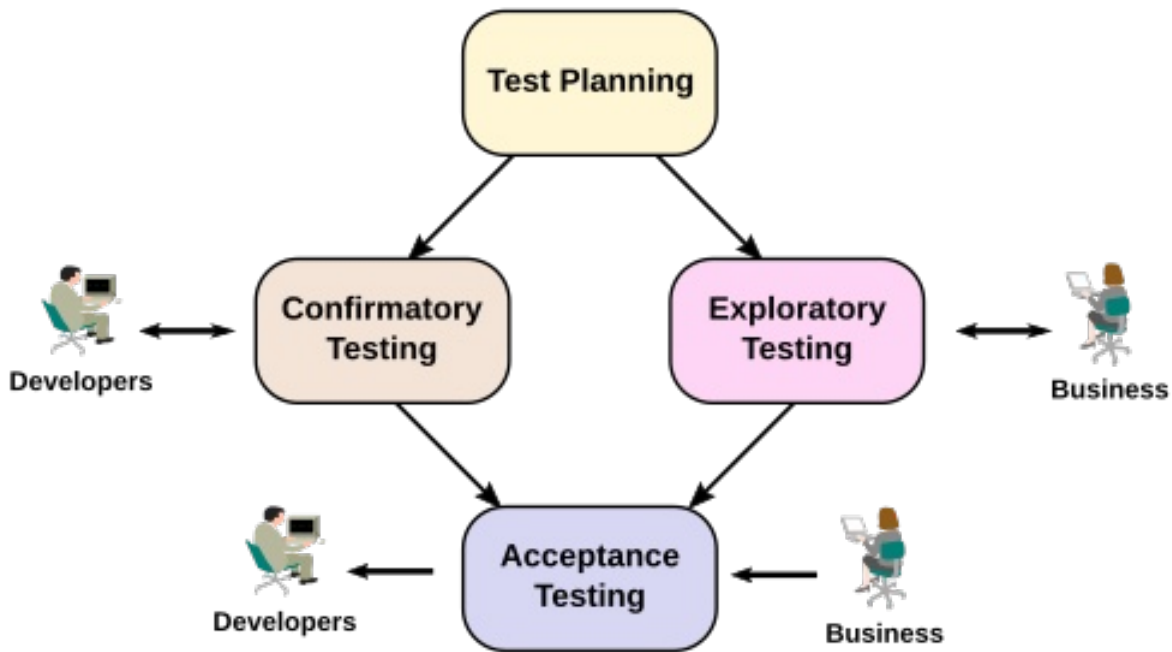
We won't be getting into this topic in this class except to note that our role here is to perform or manage reviews and verification of the project plan, time lines and costs. The important point to keep in mind is that we don't do work like develop a business case or



a project plan, but rather we ensure that proper reviews, assessments and evaluations are done by those who have the expertise to do so.

## 12.3 Construction Iterations

This gets us back into a little more familiar territory since we are used to developing test plans at the start of the project in the traditional testing model. The bulk of our testing activity takes place during the construction iterations as the team incrementally develops the project. The specific activities we usually do in this phase are described by the testing quadrants and will be explored in depth later.



There are four basic types of testing tasks done during a construction iteration:

1. **Iteration test planning.** Because requirements or other factors may have changed, the first task we need to accomplish before each iteration is to set out a test plan for that iteration. We normally cannot do this too far in advance since we do not know exactly what items will be up for development in that iteration.
2. **Confirmatory testing.** During an iteration, the developers will be using test cases to drive development. One of the most important tasks we do is to confirm that development is on the correct track by verifying and validating the test cases (which form part of the specification) to ensure they satisfy the requirements. In addition to working with the tests themselves, we also may find ourselves working with monitoring and testing the development processes as well.
3. **Exploratory testing.** During an iteration, issues may arise which may indicate problems with the requirements (missing cases) or some other aspect of development. Exploratory testing is the process of identifying those issues and working with team members to resolve any outstanding questions.

4. **Acceptance testing.** At the end of each iteration, the work done in that iteration is usually reviewed by the stakeholders and is often acceptance tested to ensure that we are still on track with what the stakeholders requirements are. Very often these acceptance tests fail as the result of new or changing requirements, or because a mistake was made in the original requirements analysis which was undetected, and this data is fed back into the iteration planning and design process. Of course changing requirements also feed back into our iteration test planning as well.

In the diagram above, the relationship of these four testing tasks is shown. The test planning feeds into the confirmatory testing and our collaborative work with the developers, and it feeds into the exploratory testing we do in collaboration with the business side. These two types of testing are covered in detail in the discussion on the quadrants. The acceptance testing at the end provides feedback from the business which then serves as input into the development process and the test planning for the next construction iteration.

### ***12.2.1 Release and Support***

Before a product is released, there are generally a set of finalizing tests that have to be done that are a mixture of traditional release testing and quality assessments.

1. **Systems testing.** Generally we cannot do a full systems test for non-functional performance until the functionality is fully tested. There are many examples of that one final feature added at the end of a project that had an impact on response time or some other critical parameter in a negative way. Even though we may be doing performance testing on individual components, we still need to do that final systems test before it gets into the hands of the customers.
2. **Documentation evaluation.** This review establishes that the required documentation exists for stakeholders, has been reviewed and vetted.
3. **Training evaluation.** This is another quality activity to ensure that the appropriate training for users and support staff is done.
4. **Product Acceptance test.** While we have been doing incremental acceptance testing, this pre-release acceptance test ensures that no significant features have been missed and that there are no obvious defects. From a quality perspective, we want to ensure that we correct those deficiencies before release.

As an example of the importance of a final acceptance test, I had a company as a client, who shall remain nameless, which discovered the importance of a product acceptance test the painful way. The company made electronic forms and had been very successful selling their product in Japan. After a while, they noticed a lot of requests from Korea for their product and decided to release a Korean version,. The file containing the various menu labels in Japanese was swapped out for a file containing the labels in Korean, but the file was incorrectly edited so about half of the labels remained Japanese.

Shortly after releasing the product, the company had to deal with a flurry of angry customers who demanded to know why half of their product's interface was in Japanese and not Korean. After that, no foreign language version shipped without a full product acceptance test by a native speaker of that language.

The other release tests take place during deployment testing and tend to be similar to what we would be doing in the more traditional model. Just a couple of examples.

1. **Configuration and environment testing.** This is where we ensure the product is working properly in the user's environment.
2. **Beta testing.** Sometimes problems don't show up until we deploy the product no matter how thorough our testing has been up until that point. In beta testing, we are monitoring the performance of the product in its operational environment and getting feedback on any quality issues that might occur. For example, the release may be functionally correct but performs too slow in the deployment environment to meet the client's expected quality levels.

We also want to keep monitoring the quality feedback from the operational environments after the product has been deployed. There inevitably are issues we can identify while the product is being used which can serve as inputs into the planning for the next release. Users often don't realize what some of their requirements are until they are using the product in their environment. Then we tend to hear things like "You know what would really be helpful, if I could only do..."

There are other reasons to monitor what happens when the product is in support:

1. **Support and maintenance issues.** We may discover that issues in the client environment create problems for support staff. Identifying these issues can provide data so that they can be accommodated in other products or releases.
2. **Behavioural economics.** Users are not rational all the time but they tend to be predictable. Understanding how they are using, or abusing, the product gives insight into ways to build products in the future that may have to deal with these same issues.
3. **Gathering data.** Process and testing maturity is data driven. There are two kinds of data we want to collect that feed back into our processes. The first is what sort of issues might exist in terms of where errors occurred and the second is how the real world errors in deployment correlate to our testing during development.

### 12.3.1 Process Metrics

We just touched on this in the last section. Basically we are collecting data during the deployment and during support on the processes themselves. We want to establish best practices for our development processes by gathering data on how well we met our success goals and what we learned from the project and the product in deployment that we can use to make future projects better.

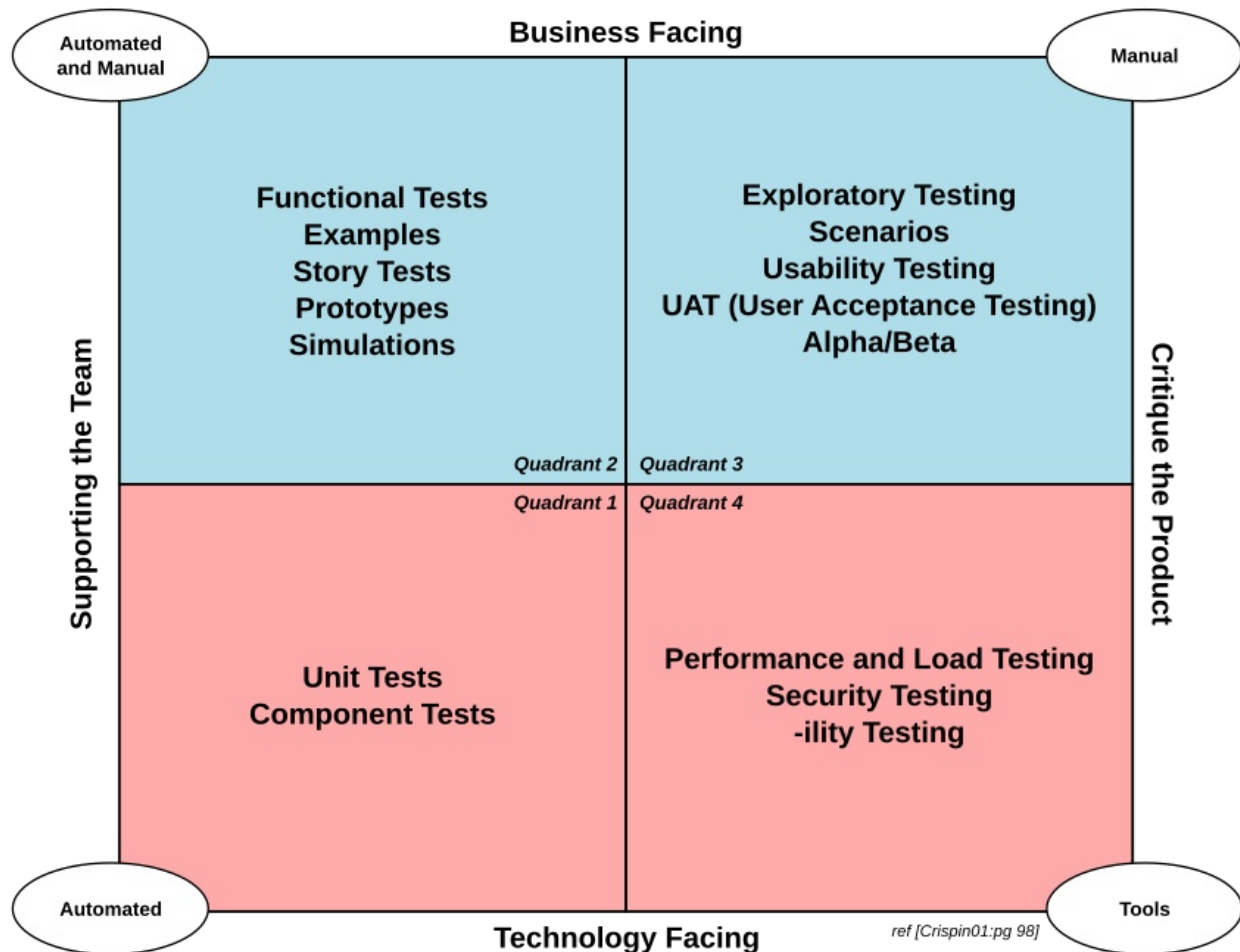


We do the same for testing, essentially looking at lessons learned from a testing perspective this project and how we can use that information to do a better testing job in future projects.

## 12.4 The Quadrants

The diagram above is adapted from [Crispin01] Agile Testing: a practical guide for testers and agile teams. This is the famous four quadrants of Agile testing.

The explanation by Crispin and Gregory [Crispin01: pg 99-100] is a lot better than anything I can do so I'll just quote it here for your reference. By the way, this book is highly recommended as background reading.



### Quadrant One:

*The lower left quadrant represents test-driven development, which is a core agile development practice.*

*Unit tests verify functionality of a small subset of the system, such as an object or method. Component tests verify the behaviour of a larger part of the system, such as a group of classes that provide some service [Meszaros, 2007]. Both types of tests are usually automated with a member of the xUnit family of test automation tools. We refer*

to these tests as *programmer tests*, *developer-facing tests*, or *technology-facing tests*. They enable the programmers to measure what Kent Beck has called the *internal quality* of their code [Beck, 1999].

A major purpose of Quadrant 1 tests is *test-driven development (TDD)* or *test-driven design*. The process of writing tests first helps programmers design their code well. These tests let the programmers confidently write code to deliver a story's features without worrying about making unintended changes to the system. They can verify that their design and architecture decisions are appropriate. Unit and component tests are automated and written in the same programming language as the application. A business expert probably couldn't understand them by reading them directly, but these tests aren't intended for customer use. In fact, *internal quality* isn't negotiated with the customer; it's defined by the programmers. Programmer tests are normally part of an automated process that runs with every code check-in, giving the team instant, continual feedback about their internal quality.

## Quadrant 2

The tests in Quadrant 2 also support the work of the development team, but at a higher level. These *business-facing tests*, also called *customer-facing tests* and *customer tests*, define *external quality* and the features that the customers want.

Like the Quadrant 1 tests, they also drive development, but at a higher level. With agile development, these tests are derived from examples provided by the customer team. They describe the details of each story. Business-facing tests run at a functional level, each one verifying a business satisfaction condition. They're written in a way business experts can easily understand using the business domain language. In fact, the business experts use these tests to define the external quality of the product and usually help to write them. It's possible this quadrant could duplicate some of the tests that were done at the unit level; however, the Quadrant 2 tests are oriented toward illustrating and confirming desired system behaviour at a higher level.

Most of the business-facing tests that support the development team also need to be automated. One of the most important purposes of tests in these two quadrants is to provide information quickly and enable fast trouble shooting. They must be run frequently in order to give the team early feedback in case any behaviour changes unexpectedly. When possible, these automated tests run directly on the business logic in the production code without having to go through a presentation layer. Still, some automated tests must verify the user interfaces and any APIs that client applications might use. All of these tests should be run as part of an automated continuous integration, build, and test process.

### Quadrant 3

*Business-facing examples help the team design the desired product, but at least some of our examples will probably be wrong. The business experts might overlook functionality, or not get it quite right if it isn't their field of expertise. The team might simply misunderstand some examples. Even when the programmers write code that makes the business-facing tests pass, they might not be delivering what the customer really wants. That is where the tests to critique the product in the third and fourth quadrants come into play. Quadrant 3 classifies the business-facing tests that exercise the working software to see if it doesn't quite meet expectations or won't stand up to the competition. When we do business-facing tests to critique the product, we try to emulate the way a real user would work the application. This is manual testing that only a human can do. We might use some automated scripts to help us set up the data we need, but we have to use our senses, our brains, and our intuition to check whether the development team has delivered the business value required by the customers.*

*Often, the users and customers perform these types of tests. User Acceptance Testing (UAT) gives customers a chance to give new features a good workout and see what changes they may want in the future, and it's a good way to gather new story ideas. If your team is delivering software on a contract basis to a client, UAT might be a required step in approving the finished stories.*

*Usability testing is an example of a type of testing that has a whole science of its own. Focus groups might be brought in, studied as they use the application, and interviewed in order to gather their reactions. Usability testing can also include navigation from page to page or even something as simple as the tabbing order. Knowledge of how people use systems is an advantage when testing usability.*

*Exploratory testing is central to this quadrant. During exploratory testing sessions, the tester simultaneously designs and performs tests, using critical thinking to analyze the results. This offers a much better opportunity to learn about the application than scripted tests. We're not talking about ad hoc testing, which is impromptu and improvised. Exploratory testing is a more thoughtful and sophisticated approach than ad hoc testing. It is guided by a strategy and operates within defined constraints. From the start of each project and story, testers start thinking of scenarios they want to try. As small chunks of testable code become available, testers analyze test results, and as they learn, they find new areas to explore. Exploratory testing works the system in the same ways that the end users will. Testers use their creativity and intuition. As a result, it is through this type of testing that many of the most serious bugs are usually found.*

### Quadrant 4

*The types of tests that fall into the fourth quadrant are just as critical to agile development as to any type of software development. These tests are technology-facing, and we discuss them in technical rather than business terms.*



*Technology-facing tests in Quadrant 4 are intended to critique product characteristics such as performance, robustness, and security. As we'll describe in Chapter 11, "Critiquing the Product using Technology-Facing Tests," your team already possesses many of the skills needed to do these tests. For example, programmers might be able to leverage unit tests into performance tests with a multi-threaded engine. However, creating and running these tests might require the use of specialized tools and additional expertise.*

*In the past, we've heard complaints that agile development seems to ignore the technology-facing tests that critique the product. These complaints might be partly due to agile's emphasis on having customers write and prioritize stories. Nontechnical customer team members often assume that the developers will take care of concerns such as speed and security, and that the programmers are intent on producing only the functionality prioritized by the customers.*

*If we know the requirements for performance, security, interaction with other systems, and other nonfunctional attributes before we start coding, it's easier to design and code with that in mind. Some of these might be more important than actual functionality. For example, if an Internet retail website has a one-minute response time, the customers won't wait to appreciate the fact that all of the features work properly. Technology-facing tests that critique the product should be considered at every step of the development cycle and not left until the very end. In many cases, such testing should even be done before functional testing.*

### **12.4.1 Automation**

Quadrants one and two are generally where we see most of the test automation taking place, with some in quadrant four. We will see automation issues in more detail in the next two modules but this section will have a general discussion on the issue of automation.

#### **Advantages of Automation**

I am a great supporter of test automation for a couple of fundamental reasons.

Automation enables continuous testing. Instead of expending resources in setting up and manually running tests, we can run them whenever we need immediate feedback and, once we have the tests set up, the resource cost of running the tests is negligible. This frees up my time as a tester to do other tasks if the developer just has to press a button to run the automated unit or acceptance tests.

Automated tests also removes a lot of the opportunities for human error. Automated testing is highly reliable in the sense we use reliable earlier that they are run exactly the same way every time. As well, if the reporting is also automated, then we remove the possibility of human error when examining the results of test execution.

## **Pitfalls for Automation**

Of course, like anything else, there are benefits and liabilities to using automation. In this section, I introduce some of the potential pitfalls we can fall into when doing automated testing.

Automated UI tests provides a high level of confidence, but they are slow to execute, fragile to maintain and expensive to build. Automation may not significantly improve test productivity unless the testers know how to test

### ***1. Test Script Errors***

Adding automation tools introduces a new source of testing error. Before, we knew that a test result depended on the quality of the test (remember the discussion on accuracy, etc) and whether or not an underlying fault existed. By maintaining strict quality for the test development and execution, we are able to achieve a high degree of confidence that test results are due to defects.

But what if there is a defect in the code that runs the test? Perhaps some of the tests are not being run or there is a logic error in the code that evaluates the result. Generally we want to perform the same sort of testing on the scripts and other code that run our tests as we do the actual code under development.

### ***2. Unstable Tests***

Tests are unstable when the feature description or acceptance criteria is unstable, often because the stakeholders have not fully settled on what that feature should do. What we want to try and avoid automating these tests too soon so that we constantly have to go back and rewrite the automated test, and then validate and verify the rewritten test.

### ***3. Interface Dependencies***

Very often automation tools operate through an interface, often a web or API interface. This means the automation script has to interact with the interface, so if the interface changes, then the test may have to be rewritten or else it might start returning incorrect results.

One way to mitigate against this is to create a special minimal testing interface through which all tests are run to avoid complications due to changes in the actual application interfaces. This is the approach generally recommended but it does require going through the effort of setting up a testing interface.

### ***4. Complacency***

There can be a tendency to just forget about what we are doing in the automated testing, which means that our testing effort may lose validity in the sense that we think we are testing something when in fact we are not. For example, not all of the tests may be running, or the test suite no longer provides test coverage or the requirements have changed but either the test cases have not or the automation of the test cases has not



been updated. Since the tests drive development, this could be a critical problem.

We mitigate this by testing the tests. This might involve a couple of test quality activities.

1. Regular test automation reviews where the tests being run are examined to ensure that they are actually being run and that the correct ones are being run.
2. Develop test data for the test cases. If we have tests that are supposed to pick up a certain defect, then we develop a mocked up version of the product which manifests a specific defect and only that defect. Running our automated tests against the mock should cause the test we are testing to fail and none of the others. In addition, we would have a mock that manifests no defects to ensure all the tests pass in the absence of any defects.

### **5. Set Up and Tear Down**

The results of a test case depend on two factors, the test input and the state of the system when the test is run. It is possible that an automation may run the tests correctly but may either not put the system in the correct state (set up) for the test or return the test environment to the state it was in before the test was run (tear down).

We generally mitigate this by running configuration tests on our test automation script to ensure they do the set up and tear down correctly.

### **6. Test Metrics**

A often used test quality metric is something like how many test cases a day are executed. While this might be an acceptable metric in a manual environment, it is meaningless in an automated environment. Instead we have to think about what sort of metrics are meaningful to us from a quality perspective.

### **7. “That’s odd..”**

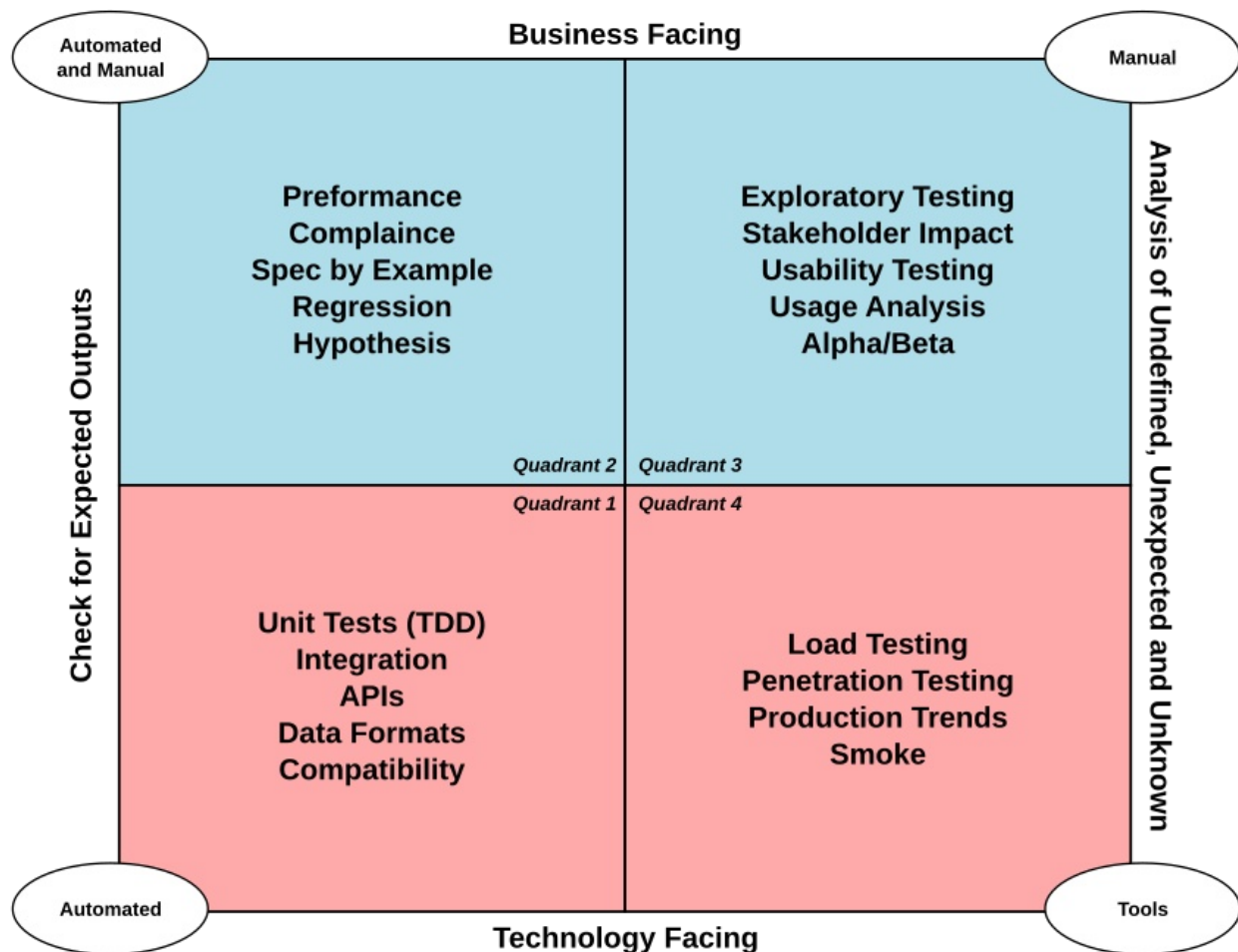
Almost every tester has some story about how they noticed some unexpected behaviour during a test that led to the discovery of a previously unsuspected defect. In my case, I noticed that when running a set of tests, that one of the tests ran many times slower than the rest, although they all should have taken about the same amount of time. All the tests passed, but the odd behaviour of that one test led to developing an exploratory set of test that identified a subtle bug in the code that managed the database queries that actually caused a momentary disconnection from the database.

In automated testing, these serendipitous observations may no longer occur because the test automation code never thinks to itself “That’s odd..” We mitigate this by spending more time doing exploratory testing and various sorts of analyses of metrics of our testing process. For example, it might not be particularly difficult to track average execution times for a set of tests and then identify test runs where those execution times show a significant deviation from the expected value.

For example, if I have a test that has been running consistently in 1ms, then after a new build I see that it is running in 120ms, that may suggest that some change in the code may have had an unexpected impact in how other code executes.

### 12.4.2 Alternative Models

The quadrants as described by Crispin and Gregory is the most well known version of the idea but it is by no means the only one. Alternative formulations exist such as this one due to Gojko Adzic<sup>4</sup>. We will not be exploring the topic of different models but will use the Crispin and Gregory version for the rest of the course.



## 12.5 Exploratory Testing

Exploratory Testing was defined by Cem Kaner, who invented the term as "a style of software testing that emphasizes the personal freedom and responsibility of the individual tester to continually optimize the quality of his/her work by treating test-related learning, test design, test execution, and test result interpretation as mutually supportive activities that run in parallel throughout the project." So what does this mean?

At its core, it epitomizes the "What would happen if..." of testing, or as some testers say, "poking the system with a sharp stick to see what happens." While this captures part of what exploratory testing does, it does not capture all of it. In this section, we will unpack Kaner's definition a bit.

### 12.5.1 Getting Off the Garden Path

Part of exploratory testing, especially with business logic and processes, is to explore the alternatives that are in the outlier and "invalid but looks valid" types of areas. For example, consider this case from Romania:

*Constantin Reliu asked the court in the town of Barlad to overturn a death certificate obtained by his wife after he had spent more than a decade in Turkey, during which time he was out of contact with his family. The court told him he was too late, and would have to remain officially deceased<sup>2</sup>.*

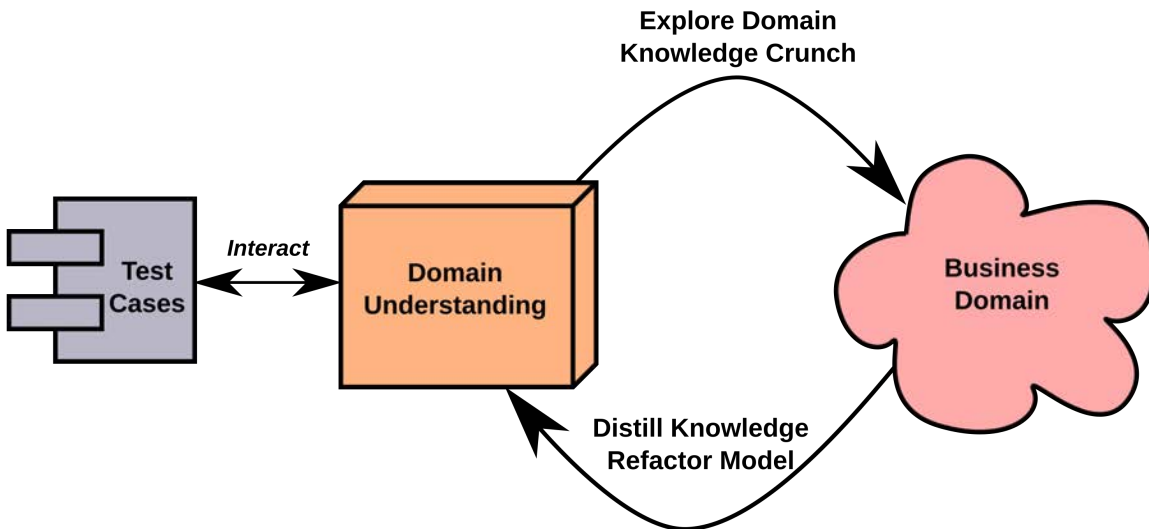
If we were thinking in terms of exploring off the garden path, we would ask the question "Is there anyway that a person who has been declared legally dead can not get the situation reversed?" Then we would go through the logic and see what would happen, probably while the client was protesting "That could never possibly happen!" To which I usually reply "Just humour me and show me why it's impossible."

These are different than boundary tests which usually try to find where errors are made in decisions about which category something belongs to, exploratory testing looks for the oddball cases that could occur where there are just gaping holes in the logic and requirements.

### 12.5.2 Knowledge Distillation

The term knowledge crunching was coined by Eric Evans in his book Domain Driven Design which was about building design models that accurately represented the problem or business domain. Knowledge crunching is the driver of the domain driven design process for three basic reasons:

## The Knowledge Crunching Iterative Process



1. Good domain models emerge through continuous refactoring because our understanding of the domain improves and is refined in an iterative manner. We are not going to take a one-time look at a complex domain, understand it all, and then churn out an elegant domain model. Our brains just don't work that way. This process has even been formally organized into something called the Stanford Design Process which is totally consistent with this kind of knowledge crunching effect.
2. Getting a deep insight into the domain requires a close and regular collaboration between the developers, testers and the domain experts. This collaboration has to be iterative to be effective because as we understand more about the domain, then we understand what sort of questions need to be asked, or where we need to drill down into the domain – things that we would not been aware of earlier in the process.

This dialog is iterative by necessity because it usually has to fit into an iterative development process to accommodate the reality of scheduling collaborations. Each time we collaborate, our latest insights into the domain allow the next collaboration to be more focused and productive.

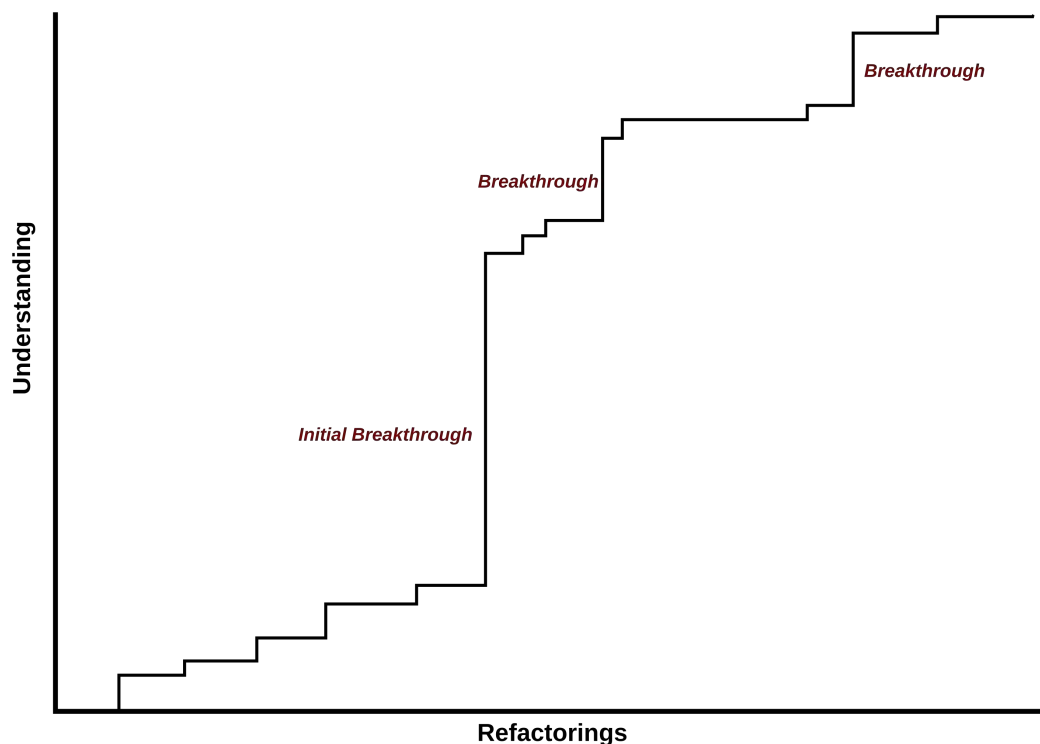
3. Third, in each iteration we can see how the domain model would suggest more of a solution. With a general idea of what a solution might look like, we can go back and explore the issues in the domain that would affect or impact the putative design we are considering. The exploratory tests start to emerge in a natural way from the deeper understanding of the domain, and we also can identify aspects which might be problematic early on and develop tests that focus in on them.

## Breakthrough Iterations

The iterative process of knowledge so far lends itself perfectly to this sort of insight process. Evans advises us to look out for that “breakthrough” iteration in which we suddenly get a whole new insight into the problem domain which results in us being able to refactor our understanding of it into a simpler, more effective and more elegant form.

*Often, though, continuous refactoring prepares the way for something less orderly. Each refinement of code and model gives developers a clearer view. This clarity creates the potential for a breakthrough of insights. A rush of change leads to a model that corresponds on a deeper level to the realities and priorities of the users. Versatility and explanatory power suddenly increase even as complexity evaporates.*

As this breakthrough occurs, it leads us to be able to develop a much more focused, elegant, effective and valuable set of tests, whether they are confirmatory or exploratory.



## The Experts

This section reproduces the explanations for exploratory testing cited by Crispin and Gregory. Notice the references to getting deeper understandings of the domain that echo what Evan said in Domain Driven Design.

### Exploratory Testing Explained

Michael Bolton is a trainer and consultant in rapid and exploratory testing approaches. He teaches a course called Rapid Software Testing, which he co-writes with senior author James Bach. Here’s Michael’s definition of exploratory testing.

*Cem Kaner didn't invent exploratory testing, but he identified and named it in 1983, in the first edition of Testing Computer Software, as an approach all testers use when their brains are engaged in their work. He and James Bach, the other leading advocate of the approach, have long defined exploratory testing as "simultaneous test design, test execution, and learning." Kaner also defines exploratory testing more explicitly as "a style of testing that emphasizes the freedom and responsibility of the individual tester to continually optimize the value of her work by treating learning, test design, test execution, and test result interpretation as activities that continue in parallel throughout the project." That's quite a mouthful. What does it mean?*

*The most important thing to remember about exploratory testing is that it's not a test technique on its own. Instead, it's an approach or a mindset that can be applied to any test technique. The second thing to remember is that exploratory testing is not merely about test execution; testers can also take an exploratory approach when they're designing new tests at the beginning of the iteration or analyzing the results of tests that have already been performed. A third important note is that exploratory testing isn't sloppy or slapdash or unprepared testing. An exploratory approach might require very extensive and elaborate preparation for certain tests—and an exploratory tester's knowledge and skill set, developed over years, is an often invisible yet important form of preparation. An exploratory test might be performed manually, or might employ extensive use of test automation—that is, any use of tools to support testing. So if exploratory testing isn't a technique, nor test execution, nor spontaneous, nor manual, what is it that makes a test activity exploratory? The answer lies in the cognitive engagement of the tester—how the tester responds to a situation that is continuously changing.*

*Suppose that a tester is given the mission to test a configuration dialog for a text editor. A tester using an exploratory approach would use specifications and conversations about the desired behavior to inform test ideas, but would tend to record these ideas in less detail than a tester using a scripted approach. A skilled tester doesn't generally need much explicit instruction unless the test ideas require some specific actions or data. If so, they might be written down or supplied to a program that could exercise them quickly. Upon seeing the dialog, the exploratory tester would interact with it, usually performing tests in accordance with the original test ideas—but she might also turn her attention to other ideas based on new problems or risks in the dialog as it appeared in front of her. Can two settings conflict in a way not covered by existing tests? The exploratory tester immediately investigates by performing a test on the spot. Does the dialog have a usability issue that could interfere with a user's work flow? The exploratory tester quickly considers a variety of users and scenarios and evaluates the significance of the problem. Is there a delay upon pressing the OK button? The exploratory tester performs a few more tests to seek a general pattern. Is there a possibility that some configuration options might not be possible on another platform? The exploratory tester notes the need for additional testing and moves on. Upon receiving new builds, the exploratory tester would tend to deemphasize repetition and emphasize variation in order to discover problems missed by older tests that are no longer revealing interesting infor-*



mation. This approach, which has always been fruitful, is even more powerful in environments where the need for repeated testing is handled by the developers' low-level, automated regression tests.

*Exploratory testing is characterized by the degree to which the tester is under her own control, making informed choices about what he or she is going to do next, and where the last outcome of the last activity consciously informs the next choice. Exploratory and scripted approaches are at the opposite poles of a continuum. At the extreme end of the scripted mind-set, the decision as to what to do next comes exclusively from someone else, at some point in the past. In the exploratory mind-set, the decision to continue on the same line of inquiry or to choose a new path comes entirely from the individual tester, in the moment in which the activity occurs. The result of the last test strongly informs the tester's choices for the next test. Other influences include the stakeholders for whom test information might be important, the quality criteria that are important to stakeholders, the test coverage that stakeholders seek, specific risks associated with the item being tested, the needs of the end user of the product, the skills of the tester, the skills of the developers, the state of the item under test, the schedule for the project, the equipment and tools that are available to the tester, and the extent to which she can use them effectively—and that's only a partial list.*

*No test activity performed by a thinking human is entirely scripted. Humans have an extraordinary capacity to recognize things even when people are telling them not to, and as a result we can be distracted and diverted—but we can learn and adapt astonishingly quickly to new information and investigate its causes and effects. Machines only recognize what they've been programmed to recognize. When they're confronted with a surprising test result, at best they ignore it; at worst, they crash or destroy data.*

*Yet no test activity performed on behalf of a client is entirely exploratory, either. The exploratory tester is initially driven by the testing mission, which is typically set out by the client early in the project. Exploratory work can also be guided by checklists, strategy models, coverage outlines, risk lists—ideas that might come from other people at other times. The more that the tester is controlled by these ideas rather than guided by them, the more testing takes on a scripted approach.*

*Good exploration requires continuous investigation of the product by engaged human testers, in collaboration with the rest of the project community, rather than following a procedurally structured approach, performed exclusively by automation. Exploration emphasizes individuals and interactions over processes and tools. In an agile environment, where code is produced test-first and is covered with automated regression tests, testers can have not only the confidence but also the mandate to develop new tests and seek out new problems in the moment. Exploration emphasizes responding to change versus following a plan. Exploratory approaches use variation to drive an active search for problems instead of scripted manual or automated test cases that merely confirm what we already knew.*

*Exploration emphasizes working software over comprehensive documentation. And to be effective, good exploration requires frequent feedback between testers, developers, customers, and the rest of the project community, not merely repetition of tests that were prepared at the beginning of the iteration, before we had learned important things about the project. Exploration emphasizes customer collaboration over negotiated contracts. Exploratory approaches are fundamentally agile.*

*Exploratory testing embraces the same values as agile development. It's an important part of the "agile testing mind-set" and critical to any team's success.*

### **Technique: Exploratory Testing and Information Evaluation**

*Jon Hagar, an experienced exploratory tester, learner, and trainer, shares some activities, characteristics, and skills that are vital to effective exploratory testing.*

*Exploratory testing uses the tester's understanding of the system, along with critical thinking, to define focused, experimental "tests" which can be run in short time frames and then fed back into the test planning process.*

*An agile team has many opportunities to do exploratory testing, since each development cycle creates production-ready, working software. Starting early in each development cycle, consider exploratory tests based on:*

- Risk (analysis): The critical things you and the customer/user think can go wrong or be potential problems that will make people unhappy.
- Models (mental or otherwise) of how software should behave: You and/or the customer have a great expectation about what the newly produced function should do or look like, so you test that.
- Past experience: Think about how similar systems have failed (or succeeded) in predictable patterns that can be refined into a test, and explore it.
- What your development team is telling you: Talk to your developers and find out what "is important to us."
- Most importantly: What you learn (see and observe) as you test. As a tester on an agile team, a big part of your job is to constantly learn about your product, your team, and your customer. As you learn, you should quickly see tests based on such things as customer needs, common mistakes the team seems to be making, or good/bad characteristics of the product.

*Some tests might be good candidates for automated regression suites. Some might just answer your exploratory charter and be "done." The agile team must critically think about what they are learning and "evolve" tests accordingly. The most important aspect here is to be "brain on" while testing, where you are looking for the "funny," unexpected, or new, which automated tests would miss. Use automation for what it is good at (repetitive tasks) and use agile humans for what we are good at (seeing, thinking, and dealing with the unexpected).*



*Several components are typically needed for useful exploratory testing:*

- **Test Design:** An exploratory tester as a good test designer understands the many test methods. You should be able to call different methods into play on the fly during the exploration. This agility is a big advantage of exploratory testing over automated (scripted) procedures, where things must be thought out in advance.
- **Careful Observation:** Exploratory testers are good observers. They watch for the unusual and unexpected and are careful about assumptions of correctness. They might observe subtle software characteristics or patterns that drive them to change the test in real time.
- **Critical Thinking:** The ability to think openly and with agility is a key reason to have thinking humans doing nonautomated exploratory testing. Exploratory testers are able to review and redirect a test into unexpected directions on the fly. They should also be able to explain their logic of looking for defects and to provide clear status on testing. Critical thinking is a learned human skill.
- **Diverse Ideas:** Experienced testers and subject matter experts can produce more and better ideas. Exploratory testers can build on this diversity during testing. One of the key reasons for exploratory tests is to use critical thinking to drive the tests in unexpected directions and find errors.
- **Rich Resources:** Exploratory testers should develop a large set of tools, techniques, test data, friends, and information sources upon which they can draw. The agile test team members should grow their exploratory resources throughout a project and throughout their careers.

*To help you understand a day in the life of an agile exploratory tester, here is a short tester's story:*

I arrived at 8:00 a.m. and reviewed what had happened the night before during automated testing. The previous night's automated tests found some minor but interesting errors. A password field on a login form had accepted a special character, which should have been rejected by the validation. I created an outline as a starting point for my "attack" (a top-level plan and/or risk list).

As I thought about my "plan of attack," I sketched a small state model of the problem on a flip chart and showed this to a developer and my team's customer rep. I designed a test incorporating their suggestions, using some data stress inputs that I expected the validation to reject (a 1 MG file of special characters). I executed my test with my stress input, and, sure enough, the system rejected them as expected. I tried a different data set and the system failed with a buffer overflow in the database. I was learning, and we were on the trail of a potentially serious security bug. As the day went on, I explored different inputs to the password field and worked with the team to get the bug fixed.

You can learn from automated test results as well as from exploratory testing. Each type of testing feeds into the other. Develop a broad range of skills so you'll be able to identify important issues and write tests to prevent them from reoccurring.

